暖城向新 热力不息

面向供热格局变革的城镇供热源荷匹配与规划寻优模型构建

摘　要

针对集中供热低碳重构、分散供热清洁替代和夏热冬冷地区供热新建中的源荷匹配问题，由于热源品位、建筑需求与存量资产差异明显，容量选择又与逐时调度、热状态及保供边界相互牵制，长期沿用的“先定热源、再算容量、按典型日校核”流程难以在动态工况下取得最优解。本文围绕资产配置、能量输配和跨时段调节，建立“公用模型内核＋场景适配器”的源—网—荷—储—电协同规划模型。

模型以建设、退役和容量为规划变量，以出力与热流为运行变量，以建筑与储热状态连接相邻时段，按节点、温位和小时建立带温位的能量守恒，联立设备转换、管段容量、储热递推、舒适边界与配电约束。在此内核上形成六项机制：温位下限与管段容量共同决定可供热量；存量管径与改造投资进入规划变量；燃煤退役规模由新增清洁容量、储热可信容量和备用容量共同兜底；建筑热状态与热源出力、储热充放、配电容量在逐时尺度上响应；规划解经有限管网模式回算，不合格模式转为新约束退回重解；同一内核按场景加载专用约束，实现三类供热格局的迁移比较。

本文以混合整数线性规划求解，乌鲁木齐联立投资与运行，固原、武汉锚定投资后冻结资产复算全季；以ε-约束逐档收紧碳帽生成成本—碳排候选解，并以条件风险价值描述气象不确定性；启停凸包重构与稀疏编译缓存降低求解负担。

按已核输入重跑三城算例，乌鲁木齐在4392 h窗口内收敛：等年值成本27.76万元、碳排放27.31 t CO_2、单位供热成本0.1007元/kWh，联合层相对间隙0.021%，失供率为零。该方案退役既有热电联产750 kW与燃气调峰锅炉900 kW，同步新建应急电锅炉908.44 kW、配电接入558.33 kW，并在候选干线上选定DN150管径。同一投资方案的年度回放中，整数变量由26280个减至8760个，两年度合计求解时间由602.483 s降至112.750 s，减少81.29%。风险层在置信水平0.8下取风险权重0.05，乌鲁木齐期望年化成本增加0.012万元、运行费用CVaR下降0.831万元，用于比较备用能力与尾部费用的取舍。

四项独立专项检验输入压缩与物理边界：六栋公开计量建筑的归一W_1误差中位数由12.3150%降至4.4775%，统计准入由3/6增至4/6；双楼原型在配电上限下四倍热容省电量费276.68元；72 h参考管网第三轮单位长度压降237.96 Pa/m、最远站供水温度57.45 °C，达到登记限值；四栋合成建筑的逐楼1R1C模型在8784 h服务年及2928 h冬季内无状态与舒适违约。

该模型可在给定资源、候选网络与服务边界下衔接规划与运行，为供热系统的建设退役、储热配置、配电增容和保供方案比较提供分析与决策依据。

关键词：城镇供热；源荷匹配；混合整数线性规划；温位匹配；多尺度耦合

目　录

一、　研究背景与文献综述　1
（一）　城镇供热转型背景 1
（二）　建筑热环境与居民用热需求 1
（三）　传统供热规划的局限 2
（四）　国内外研究进展 2
（五）　研究不足与本文切入点 3
二、　负荷预测与协同优化模型　4
（一）　基本假设 4
（二）　模型结构与决策变量 4
（三）　建筑负荷与热状态 4
（四）　协同约束 6
（1）　热源容量与能量转换 6
（2）　节点平衡与管网输配 7
（3）　储热与配电 8
（4）　目标函数与可靠性 9
（五）　场景适配 10
（1）　乌鲁木齐：既有集中供热系统低碳重构 10
（2）　固原：分散供热系统清洁替代 10
（3）　武汉：夏热冬冷地区综合供能新建 11
（六）　统一校核 12
三、　多场景多尺度源荷协同优化方法　13
（一）　多尺度变量与衔接 13
（二）　多目标协同优化 13
（三）　求解流程与收敛准则 15
（四）　计算效率与扩展性 16
四、　数据体系与模型验证　18
（一）　数据来源与参数性质 18
（二）　证据分层 18
（三）　主三城结果 20
（四）　工程验收与代码组织 20
五、　模型应用与决策分析　22
（一）　区域供热转型规划 22
（二）　项目方案比选 22
（三）　风险与政策输出 22
六、　模型创新、局限与结论　23
（一）　模型创新 23
（二）　模型局限 25

（三） 结论 25
参考文献 **26**
附录1 场景约束与校核式 **30**
A 公共模型补充约束 30
B 乌鲁木齐场景补充约束 35
C 固原场景补充约束 39
D 武汉场景补充约束 44
附录2 符号与缩略语说明 **49**
附录3：核心代码（Python详编） **50**
附录4：数据来源、实际规模与关键参数 **93**
附录5：补充对照实验与核验记录 **101**
附录6 推荐信 **112**

一、 研究背景与文献综述

（一） 城镇供热转型背景

城镇供热兼具公共服务与能源消费属性，热源、管网、建筑末端、电力系统和用户行为在其中相互制约。热能约占全球终端能源消费的一半，可再生热力的增长尚不足以支撑供热部门快速脱碳[26]。国家层面已把供热转型纳入“十五五”部署：规划建议要求推进煤电改造升级和散煤替代、大力发展新型储能、加快智能电网和微电网建设，并提高终端用能电气化水平[34]；规划纲要提出加快热力系统绿色低碳转型，因地制宜开展余热资源利用和非化石能源供热，有序推进供热计量改造和按热量收费[35]。供热规划因此需要由单一热源扩容转向源、网、荷、储协同配置。

我国各地区在气候、资源和既有资产上差异显著，同一套技术组合无法通用：北方大型城市以既有热网与热源的改造退出为主，县城及农村地区需要确定集中与分散的供热边界，夏热冬冷地区还要统筹冬季供热、夏季供冷、生活热水与配电容量。本文选取乌鲁木齐、固原、武汉三个参数化算例，分别代表存量集中供热重构、分散清洁替代和夏热冬冷地区新建三类格局，其主导约束各不相同。

（二） 建筑热环境与居民用热需求

老旧建筑的供热问题由建筑性能不足与运行失调共同引起：围护结构保温不足、空气渗透、末端老化和二次网失衡不仅抬高热负荷，还造成近端过热、远端欠热、回水温度偏高和泵耗上升。图1与图2给出改造运维与热工测量的现场对象。

图 1　　既有建筑供热系统的改造与运维现场

既有建筑节能水平差异较大[25]，仅改造热源难以保证室温与用热公平同步改善；热泵接入还可能抬升冬季电力峰值。若只比较年总热量，近端过热、远端欠热和配电瓶颈会被平均值掩盖。

围护结构及使用条件影响建筑热损系数与得热。图2引用新疆大学研究人员在乌鲁木齐开展既有建筑改造研究时的建筑和热流计实拍[33]，说明建筑调查与热工测量对应的现场对象。

（a）乌鲁木齐南郊既有旅馆

（b）现场使用的多通道热流计

图 2　乌鲁木齐既有建筑与热流测量现场。取自Liao等[33]原图1左、图5左，CC BY 4.0；等比例排版，未修改照片内容。

（三） 传统供热规划的局限

传统方法沿“先定技术路线、再核算容量、最后用典型日校核”的流程推进，难以反映热源、管网、建筑、储热和电网之间的关系：管网降温会影响末端改造，热泵扩容会增加配电投资，储热替代峰值容量又受充放热功率和时序状态限制。室外温度、资源可用率、能源价格和设备故障随工况变化，仅凭年平均负荷或单个设计日规划，无法同时保证全年经济性和寒潮期间的供热可靠性。

（四） 国内外研究进展

国内研究覆盖既有集中供热改造、分散清洁供热与建筑节能三类路径，实践涉及工业余热、大型热泵、低温热网和生物质等技术[2,7,9]，方法上以综合能源系统优化、多目标规划、鲁棒优化和需求响应为主[5,6,8,10]，但多数围绕单项设备展开，改造后热源与管网容量如何调整、集中与分散的边界如何确定，仍需统一量化。

欧洲已形成建筑节能、区域供热、热泵与低温热网相结合的技术路径，2024年空间供暖和生活热水约占欧盟家庭终端能源消费的77.1%[27]。北欧通过地方热力规划确定区域供热与个体供热的服务边界，第四代区域供热研究强调

需求侧柔性与低温热源利用[20,21,22]，其结论受资源与监管条件影响较大，对大规模热泵带来的配电峰值考虑不足。

（五） 研究不足与本文切入点

现有研究存在五项不足。第一，系统边界分散，热源、管网、建筑、储热和配电常由不同模型分别处理，总量平衡并不意味着温位、位置和输配能力同时满足要求。第二，规划与运行脱节，用年平均量或少量典型日确定容量所得方案，在寒潮或设备故障时缺少可靠容量。第三，低碳、经济与民生目标缺少统一权衡，年度清洁热占比不能代表极寒峰值的化石依赖程度。第四，物理精度与求解规模矛盾，连续水力和建筑热惯性直接纳入大规模整数模型会明显增加计算难度。第五，场景迁移困难，分别建模难以形成一致的数据与指标定义，城市、区域、用户三个尺度之间的耦合也缺少统一表达。

针对上述不足，本文以“公用模型内核＋场景适配器”为总体结构，将资产演化、带温位能量平衡、储热、建筑热状态、电力约束、成本碳排和可靠性纳入统一框架，按场景加载专用约束，形成“优化筛选—压力测试—数值校核”的流程。

二、 负荷预测与协同优化模型

（一） 基本假设

1. 同类建筑参数一致时聚合为典型建筑，外部数据按 1 h 步长连续输入。
2. 热源在允许工况和容量内运行，管网采用离散温位与固定温差近似，储能逐时递推且不得同时充放热。

（二） 模型结构与决策变量

供热规划需要同时确定资产配置与逐时运行方式：热源容量决定可生产的热量，管网温位和输送能力决定热量能否到达负荷节点，储热调节供需时差，建筑热惯性使当前供热行为影响后续室温。能量沿“余热或燃料—热源转换—区域热网—建筑或储热”流动，热泵、电锅炉和循环泵的用电共同占用配电容量（图3）。

决策变量分为三类。规划变量包括热源新增与退役容量、管网保留或改造状态、储热规模和配电增容，在完整供暖季内保持不变；运行变量包括各类热源出力、管段热流、储热充放功率和购电功率，随时段变化；状态变量包括建筑热状态和储热存量，由上一时段状态与当前运行决策共同确定。规划层决定系统资产，运行层判断其能否逐时满足需求，校核层把不符合项转化为新的温位、容量或运行模式限制反馈至规划层重新求解。

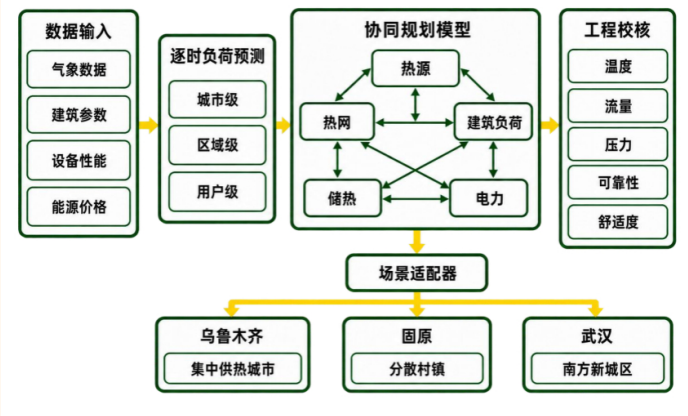

图 3　　源网荷储电协同规划总体结构

（三） 建筑负荷与热状态

建筑净供热负荷由围护结构传热、渗透换气和新风热损失构成，扣除太阳辐射及人员、设备等内部得热：

$$Q_{b,t}^{\text{load}} = Q_{b,t}^{\text{trans}} + Q_{b,t}^{\text{inf}} + Q_{b,t}^{\text{vent}} - Q_{b,t}^{\text{sol}} - Q_{b,t}^{\text{int}}. \tag{1}$$

其中 U_b、A_b、n_b^{inf}、V_b 为综合传热系数、围护结构面积、换气次数和建筑体积，$\theta_{b,t}^{a}$ 与 T_t^{out} 为室内外温度，渗透项按 $\rho_a c_a n_b^{\text{inf}} V_b \Delta\theta$ 逐时计算；负荷以 kW 计，参数与换算见附录1.A。

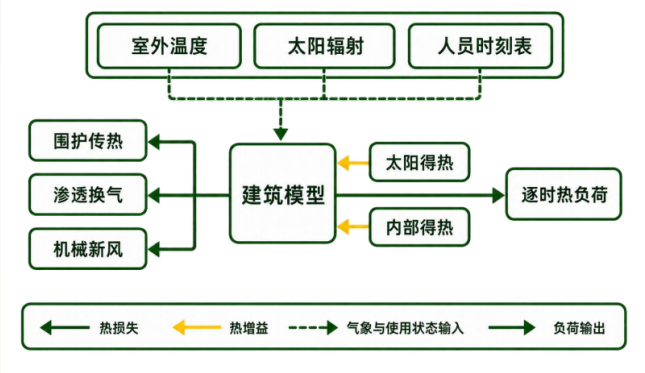

图 4　　建筑逐时负荷分解

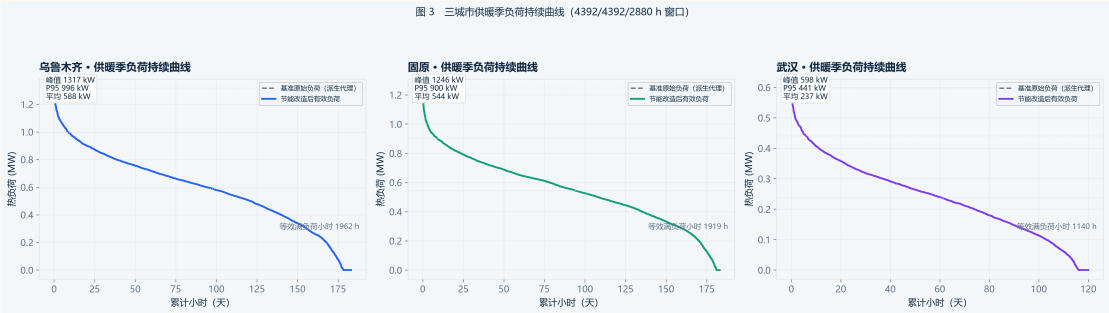

图 5　　三城供暖季负荷持续曲线

为描述提前供热对后续室温的影响，框架列示 2R2C 结构，以室内空气温度 $\theta_{b,t}^{a}$ 和建筑质量节点温度 $\theta_{b,t}^{m}$ 为状态变量，并受舒适区间与初末状态约束：

$$
\begin{aligned}
C_b^a \frac{\theta_{b,t+1}^a - \theta_{b,t}^a}{\Delta t} &= Q_{b,t}^h - U_b A_b(\theta_{b,t}^a - T_t^{\text{out}}) \\
&\quad - Q_{b,t}^{\text{inf}} - Q_{b,t}^{\text{vent}} + Q_{b,t}^{\text{int}} + Q_{b,t}^{\text{sol}}, \\
C_b^m \frac{\theta_{b,t+1}^m - \theta_{b,t}^m}{\Delta t} &= \frac{\theta_{b,t}^a - \theta_{b,t}^m}{R_b^{am}}, \\
\theta_{b,t}^{\min} \le \theta_{b,t}^a \le \theta_{b,t}^{\max}, &\qquad \theta_{b,1}^a = \theta_b^{a,0}, \qquad \theta_{b,T}^a \ge \theta_b^{a,\text{end}}.
\end{aligned}
\tag{2}
$$

其中 $Q_{b,t}^h$ 为建筑实际获得的供热功率，C_b^a、C_b^m、R_b^{am} 为空气节点热容、质量节点热容和节点间等效热阻，下标表示小时边界，$t \to t+1$ 与首末编号同处一个窗口。主三城采用聚合等效热状态，E08 以精确离散 1R1C 模型对四栋合成建筑单独验证，2R2C 为共用框架中的扩展结构；末态约束使建筑只能在舒适区间内调整供热时序。

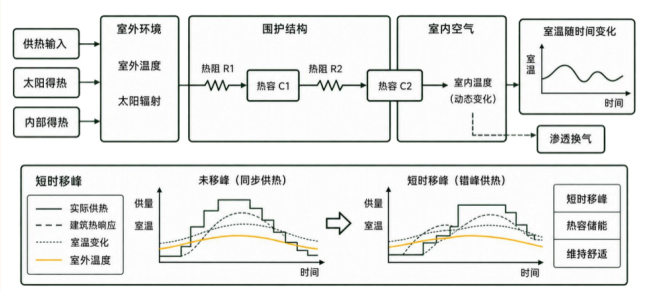

图 6　　建筑热惯性与跨时段移峰

（四） 协同约束

（1） 热源容量与能量转换

设 K_k^{exist}、K_k^{new}、K_k^{ret} 分别为设备 k 的既有、新增和退役容量，规划后容量与逐时出力满足

$$
K_k = K_k^{\text{exist}} + K_k^{\text{new}} - K_k^{\text{ret}}, \qquad \underline{Q}_k u_{k,t} \le \sum_g Q_{k,g,t} \le a_{k,t} K_k. \tag{3}
$$

其中 g 为供热温位，$u_{k,t}$ 为启停变量，$a_{k,t}$ 为逐时可用率，相邻时段出力另受爬坡约束；锅炉、热泵和电锅炉的转换关系为

$$Q_{k,t}^{\text{boiler}} = \eta_k^{\text{boiler}} F_{k,t}^{\text{fuel}}, \qquad Q_{k,t}^{\text{HP}} = \text{COP}_{k,t} P_{k,t}^{\text{HP}}, \qquad Q_{k,t}^{\text{EB}} = \eta_k^{\text{EB}} P_{k,t}^{\text{EB}}. \tag{4}$$

直接换热与热泵源侧取同一余热时合并计入资源池，避免重复取用。

（2） 节点平衡与管网输配

按节点、温位和时段分别建立热量守恒，避免总量平衡掩盖局部输配不足：

$$\begin{aligned} &\sum_{k\in K_n} Q_{k,g,t} + \sum_{e\in\delta^-(n)} \eta_{e,t} F_{e,g,t} + \sum_{s\in S_{n,g}} Q_{s,t}^{\text{dis}} \\ &\quad = \sum_{b\in B_{n,g}} Q_{b,t}^{h} + \sum_{s\in S_{n,g}} Q_{s,t}^{\text{ch}} + \sum_{e\in\delta^+(n)} F_{e,g,t}. \end{aligned} \tag{5}$$

其中 $F_{e,g,t}$ 为管段入口热功率，$\eta_{e,t}$ 为输送效率，$Q_{s,t}^{\text{ch}}$ 与 $Q_{s,t}^{\text{dis}}$ 为储热充、放热功率；图7给出各环节的热流、电力流与状态传递。

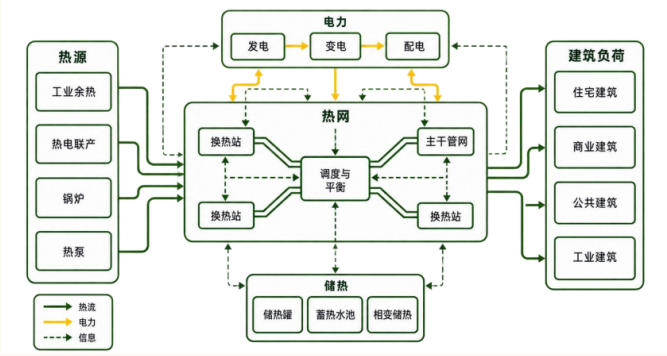

图 7　　源网荷储电统一模型

热源最低供水温度须满足

$$T_k^{\text{src,min}} \geq T_g^{\text{sup}} + \Delta T_g^{\text{hx}}, \tag{6}$$

不满足式(6)的低品位热源需经热泵升温后承担高温负荷。主模型以给定供回水温差关联热功率与流量并限制管段输送容量，候选方案确定后再复算连续温降、远端温度与压降。

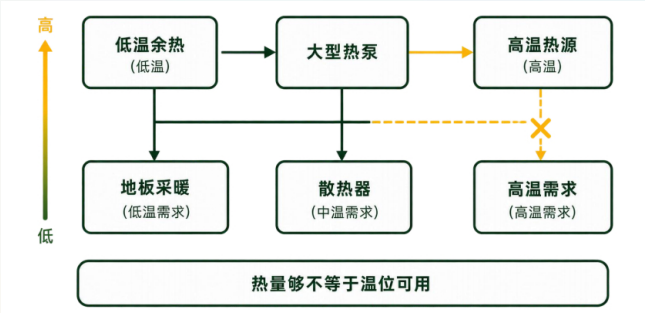

图 8　　温位匹配与梯级利用

（3） 储热与配电

储热存量按完整供暖季连续递推，并受容量、充放功率与初末存量边界限制：

$$\begin{cases} E_{s,t} = (1-\mu_s)E_{s,t-1} + \eta_s^{\mathrm{ch}} Q_{s,t}^{\mathrm{ch}} \Delta t - \dfrac{Q_{s,t}^{\mathrm{dis}} \Delta t}{\eta_s^{\mathrm{dis}}}, \\ 0 \le E_{s,t} \le K_s^E, \qquad 0 \le Q_{s,t}^{\mathrm{ch}}, Q_{s,t}^{\mathrm{dis}} \le r_s K_s^E. \end{cases} \tag{7}$$

其中 μ_s 为自损失率，η_s^{ch} 与 η_s^{dis} 为充、放热效率；模型以二元变量排除同时充放热并设置初末存量边界。热泵、电锅炉和循环泵共同占用区域配电容量：

$$(1+\sigma)\left(\sum_k \frac{Q_{k,t}^{\mathrm{HP}}}{\mathrm{COP}_{k,t}} + \sum_k \frac{Q_{k,t}^{\mathrm{EB}}}{\eta_k^{\mathrm{EB}}} + \sum_e P_{e,t}^{\mathrm{pump}}\right) \le a_t^{\mathrm{grid}} K^{\mathrm{grid}}. \tag{8}$$

该约束使热源电气化、建筑预热和储热调峰同受配电边界限制。

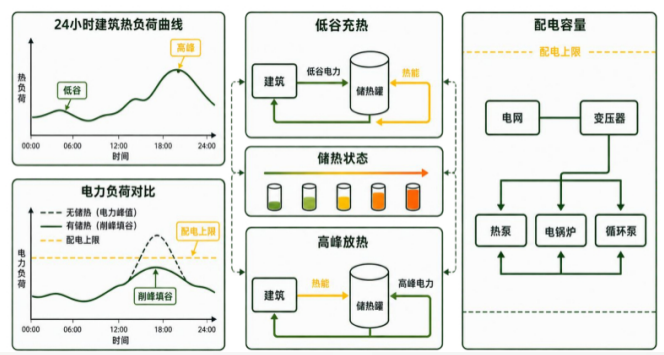

图 9　　储热与配电容量协同

（4） 目标函数与可靠性

模型以等年值成本 F^{cost} 最小为基础目标，并采用 ε 约束控制碳排放：

$$\begin{cases} \min F^{\mathrm{cost}}, \\ F^{\mathrm{CO_2}} \le \varepsilon, \\ U^{\mathrm{un}} = 0. \end{cases} \tag{9}$$

其中等年值成本含年化投资、固定运维、燃料、购电和启停费用，碳排放按同一组燃料消耗和购电量核算；失供率为失供热量与需求热量的时间加权比，推荐方案取零。改变碳排放上限并重复求解，可比较不同成本与排放的方案。

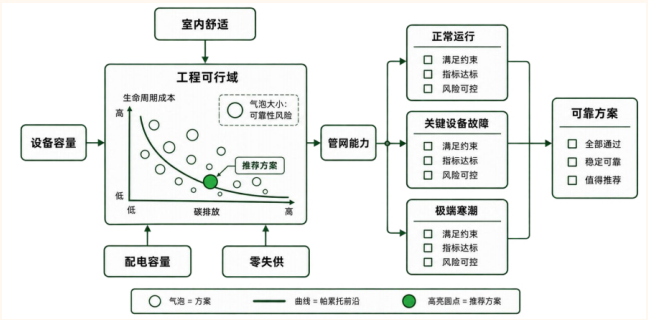

图 10　　目标函数与可靠性约束

（五） 场景适配

三类场景共享式(1)–式(9)构成的公共内核，并按既有资产、资源条件和供能对象加载专用约束（完整约束组与极端工况见附录）。

（1） 乌鲁木齐：既有集中供热系统低碳重构

燃煤退役容量须由新增清洁热源、储热有效容量和备用容量共同补足：

$$K^{\mathrm{ret}}_{\mathrm{coal},y} \le K^{\mathrm{clean,new}}_{y} + K^{\mathrm{storage,credit}}_{y} + K^{\mathrm{reserve}}_{y}. \tag{10}$$

工业余热取用量受资源可得性限制，低温余热经热泵升温后承担高温负荷；任一关键热源退出时，其余热源与储热仍须覆盖逐时需求：

$$\begin{cases} 0 \le Q^{\mathrm{waste}}_{t} \le \eta^{\mathrm{recovery}} Q^{\mathrm{available}}_{t}, \\ Q^{\mathrm{HP}}_{t} = \mathrm{COP}_{t} P^{\mathrm{HP}}_{t}, \\ \sum_{j \ne k} Q^{\mathrm{available}}_{j,t} + Q^{\mathrm{storage,dis}}_{t} \ge D_{t}. \end{cases} \tag{11}$$

式(11)将资源可得性、温位提升与 $N-1$ 保供纳入同一运行边界。

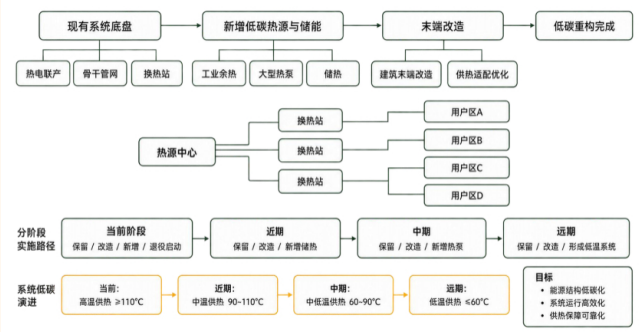

图 11　　乌鲁木齐既有集中供热低碳重构

（2） 固原：分散供热系统清洁替代

每个用户仅能由一个已建热源服务，并须同时满足服务半径与峰值容量限制：

$$\sum_i x_{i,b} = 1, \quad x_{i,b} \le z_i^{\text{site}}, \quad d_{i,b} x_{i,b} \le \bar{d}_i, \quad \sum_b D_b^{\text{peak}} x_{i,b} \le K_i. \tag{12}$$

生物质供热能力还受逐时到货与库存约束：

$$I_{i,t} = I_{i,t-1} + B_{i,t}^{\text{arrive}} - B_{i,t}^{\text{use}}, \qquad Q_{i,t}^{\text{bio}} = \eta_i^{\text{bio}} B_{i,t}^{\text{use}}. \tag{13}$$

只有同时满足两式的空间边界与燃料连续性，候选热源才能形成有效供热能力。

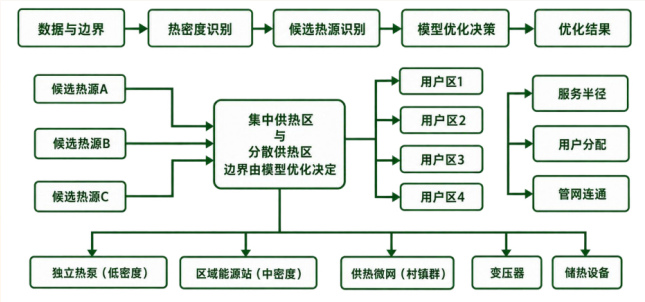

图 12　　固原分散供热选址与分配

（3） 武汉：夏热冬冷地区综合供能新建

楼栋和小区在独立楼宇、区域能源站、低温热网和第五代区域供热供冷系统间选择供能形态：

$$z_i^{\text{building}} + z_i^{\text{station}} + z_i^{\text{LTN}} + z_i^{\text{5GDHC}} = 1. \tag{14}$$

选定形态后，热源与储热须逐时满足采暖和生活热水需求；低温系统末端表面温度不得低于露点安全温度：

$$\begin{cases} Q_{i,t}^{\text{source}} + Q_{i,t}^{\text{storage,dis}} = D_{i,t}^{\text{heat}} + D_{i,t}^{\text{DHW}} + Q_{i,t}^{\text{storage,ch}}, \\ T_{b,t}^{\text{surface}} \ge T_{b,t}^{\text{dew}} + \Delta T_b^{\text{safe}}. \end{cases} \tag{15}$$

该组约束避免以牺牲生活热水或引入结露风险换取表面经济性。

图 13　　武汉四类供能形态选择

（六） 统一校核

模型以 1 h 为分辨率完成声明窗口计算，上一时段的建筑热状态、储热量和燃料库存作为下一时段初值，不在日期边界处重置。三类场景采用同一组验收条件：

$$\max_{n,g,t} |r^{Q}_{n,g,t}| \le \varepsilon_Q, \qquad \max_{s,t} |r^{E}_{s,t}| \le \varepsilon_E, \tag{16}$$

聚合热状态处于登记区间，管段流量利用率不超 1，配电容量满足，失供率为零。代表日仅用于生成候选容量；方案确定后固定规划变量，以原始逐时序列连续回放 T 小时，任一时段违反约束均返回规划层调整。

三、 多场景多尺度源荷协同优化方法

（一） 多尺度变量与衔接

模型由城市层、区域层和用户层构成。三个算例各取 3 个模型节点（2 个为聚合需求节点），乌鲁木齐、固原、武汉分别含 7、6、7 类热源和 3、2、1 条聚合管段。城市层确定机组新增与退役容量、温位模式、候选管径、储热规模和配电增容；区域层在 75/45 °C 两级温位上建立逐时能量平衡，阻止低温热源直接供应高温需求；用户层描述聚合节点的热惯性，表现为等效热状态递推

$$E_{n,t} = (1-\lambda_n)E_{n,t-1} + (q^{+}_{n,t} - q^{-}_{n,t})\Delta t, \qquad \Delta\theta_{n,t} = E_{n,t}/C_n, \tag{17}$$

其中 C_n 为节点 n 的等效热容，$q^{+}_{n,t}$ 与 $q^{-}_{n,t}$ 为相对基准负荷的增加与削减功率。原聚合算例采用该单一等效热状态，E08 的逐楼验证改用精确离散 1R1C 模型。三层通过容量、温位和建筑改造三类量耦合：用户侧负荷削减先使节点需求下降，优先减少边际热源出力，恢复负荷时由储热与可调热源共同响应，节点平衡逐时闭合。乌鲁木齐采用投资与运行联合 MILP；固原和武汉改用锚定两阶段方法，先确定投资模式再冻结投资开展全季运行校核。

图 14　　城市—区域—用户多尺度衔接

（二） 多目标协同优化

成本与碳排放采用 ε-约束法协调[32]，费用目标保留配置中的碳价与偏好项。模型以全生命周期净成本的等年值最小为目标并设置碳排放上限，逐档收紧上限后重新求解完整窗口，相邻两档的成本增量与减排量之比即边际减排成本；乌鲁木齐的成本端点为 27.49 万元、27.32 t CO_2，排放端点为 74.81 万

元、10.57 t CO_2。零失供、清洁热比例与 $N-1$ 容量裕度按硬边界处理，边界内的移峰由惩罚成本优化。

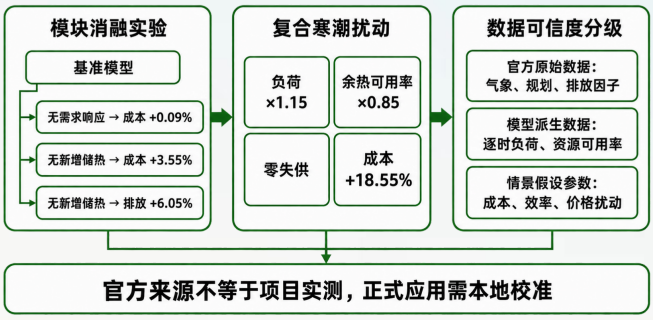

图 15　消融实验、扰动检验与数据可靠性

气象不确定性由独立风险层处理，采用

$$\min\ \mathbb{E}[C]+\lambda\,\mathrm{CVaR}_{\alpha}(C),\qquad \alpha=0.8, \tag{18}$$

其中 λ 取 0、0.02、0.05 和 0.10 四档，条件风险价值按 Rockafellar–Uryasev 线性化：

$$\mathrm{CVaR}_{\alpha}(C)=\min_{\eta,z_s}\left\{\eta+\frac{1}{1-\alpha}\sum_{s}p_s z_s\right\},\quad z_s\ge C_s-\eta,\quad z_s\ge 0. \tag{19}$$

$\lambda=0.05$ 为三个场景的统一参考值，该风险层按独立记录登记。

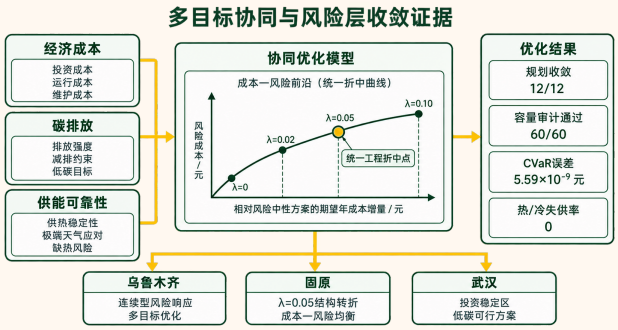

图 16　多目标协同与风险层收敛证据

（三） 求解流程与收敛准则

模型构造器采用 Python 3.11 实现，通过 SciPy 的 `milp` 调用 HiGHS 求解；约束按 CSR/CSC 稀疏格式编译和缓存，模型装配约需 1.6 s。机组启停采用凸包重建模，开机状态保留为二元变量，启停辅助变量连续化。窗口求解与复核顺序见算法1。

算法1　窗口求解与逐时复核流程

输入：场景配置、规划输入、连续窗口逐时数据与求解容差。

输出：投资及逐时运行方案，求解状态、间隙与校核记录。

1. 核对窗口、步长、单位与各模块启用状态。
2. 按第2章约束装配源网荷储电模型，联立守恒、容量与热状态条件并设初末边界。
3. 乌鲁木齐联合求解投资与运行；固原、武汉冻结投资模式后求解全季运行。
4. 按碳帽或 ε 档位重复求解并逐档保存。
5. 记录目标值、上下界、间隙、时间与可行性状态。
6. 以逐时输出复算守恒、状态、容量、资源与失供核对。
7. 导出可核结果并标注求解身份与逐时调度。

优化器一次求解整个声明窗口，求解状态、间隙、残差和后验校核分别登记，时限内可行解保留近似解身份。乌鲁木齐联合模型的相对间隙为 1.53×10^{-4}（531.1 s，启停二元 8784 个），固原运行层为 1.84×10^{-3}（64.0 s，二元 4392 个），武汉运行层间隙为 0（31.7 s），投资层记录 0.98%。

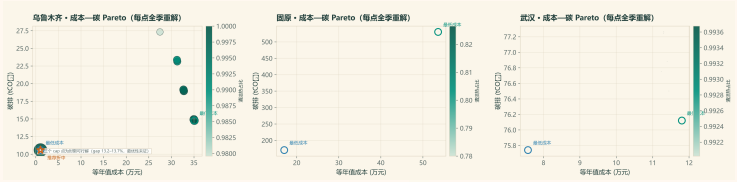

图 17　　三城成本与碳排放候选解

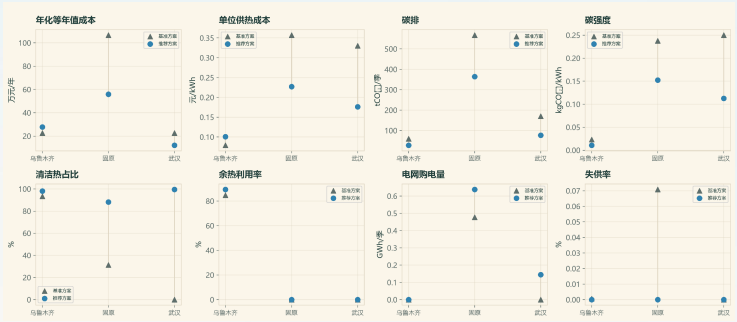

图 18　　基准与推荐方案指标对比

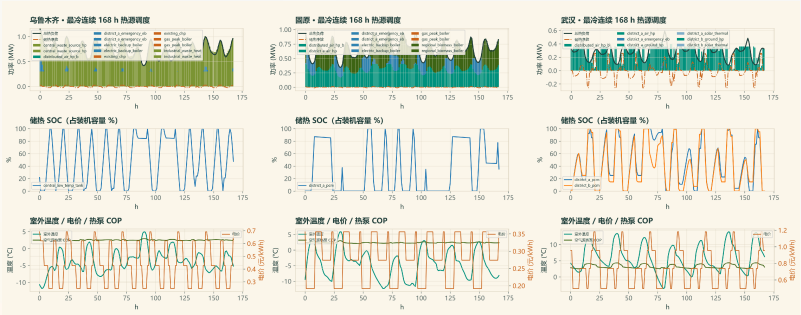

图 19　　最冷连续 168 h 调度与储热状态

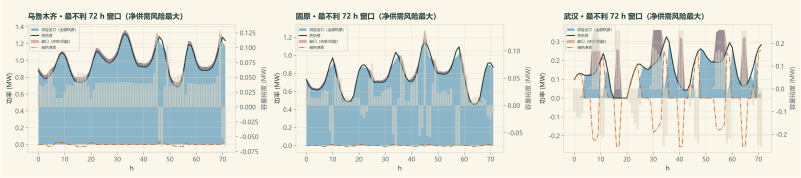

图 20　　最不利 72 h 容量裕度

（四） 计算效率与扩展性

效率改善来自启停凸包重建模和稀疏编译缓存：固原全年回放用时由 575.746 s 降至 86.903 s，两年度合计由 602.483 s 降至 112.750 s，减少 81.29%，年度回放中的整数变量由 26280 个减至 8760 个。单次窗口约含 31 万—45 万个变量。

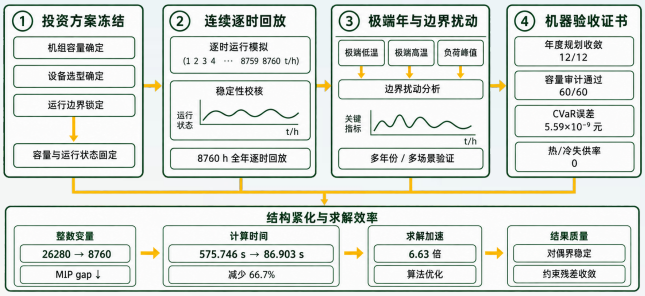

图 21　　逐时回放、边界稳定性与求解效率

四、 数据体系与模型验证

（一） 数据来源与参数性质

三城主算例的气象取自 NASA POWER 格点逐时产品，窗口依次为 2024-10-10 至 2025-04-11、2024-10-15 至 2025-04-16、2024-11-16 至 2025-03-16，按北京时间取零时、左闭右开，对应 4392、4392 和 2880 h，合计 11664 h，时间轴连续、无断时与重复。建筑负荷、设备工况、燃料价格和投资参数属派生或参数化输入，按来源可靠程度分为四类并作区间敏感性分析。单位统一为 kW、kWh、元（万元）、kg CO_2（t CO_2）、h 与百分数，热量与电量分列；字段定义与来源见附录（三）。

（二） 证据分层

主三城推荐结果与独立专项实验的对象、窗口与求解身份不同，分别报告；验证目标、核心结果与适用边界见表1。

表 1　　主算例与专项实验的对象、验证目标、结果与适用边界

对象	验证目标	核心结果	适用边界
主三城 4392/4392/2880 h	联合解与锚定投资运行解	乌鲁木齐重跑收敛：27.76 万元、27.31 t CO_2；固原、武汉取锚定投资解	参数化情景窗口
E01 六栋公开计量建筑	代表日压缩的峰值保留	W_1 中位数 12.3150%→4.4775%；准入 3/6→4/6	独立统计检验
E03 双楼原型 2880 h	热容与配电上限的交互	四倍热容省费 427.20/276.68 元，用电增 570.33/335.46 kWh	固定理想资产
E04 72 h 参考管网	有限模式的输配反馈	第三轮 237.96 Pa/m、57.45 °C 达登记限值	独立参考管网
E08 四栋合成建筑	逐楼室温与状态边界	8784/2928 h，间隙 0.1344%/0.4755%，违约 0	固定投资
E09 三城 27 条固定策略	代表日选取方法对照	频数法/软校准 9/12、11/12；四种硬法 12/12	开发确认样本

逐格结果与完整分母见图22、图23及附录（三）。

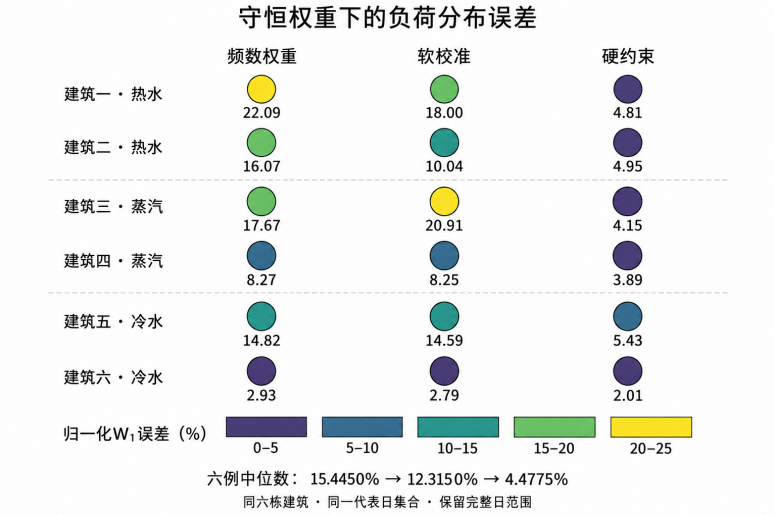

图 22　　同一代表日集合下的六楼负荷分布误差，颜色按归一 W_1 误差分箱

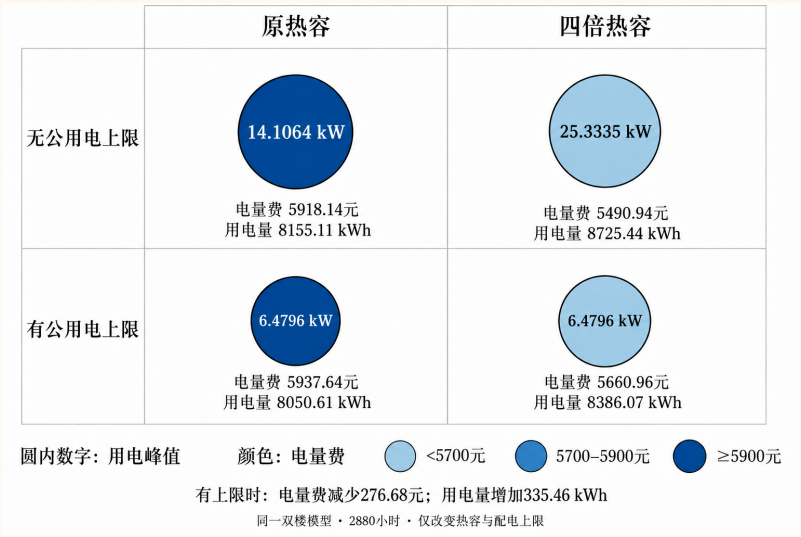

图 23　　热容与公用电上限的四格对照，圆内数字为用电峰值

（三） 主三城结果

乌鲁木齐、固原、武汉采用第3章说明的三种求解身份。乌鲁木齐在4392 h 窗口内收敛：等年值成本 27.76 万元、碳排放 27.31 t CO_2、单位供热成本 0.1007 元/kWh，联合层相对间隙 0.021%，失供率为零；方案退役既有热电联产 750 kW 与燃气调峰锅炉 900 kW，新建应急电锅炉 908.44 kW、配电接入 558.33 kW。三城逐时调度与已选投资分项随文交付。

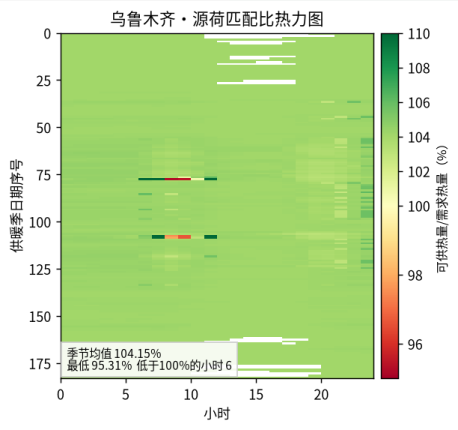

图 24　　乌鲁木齐供暖季逐时源荷匹配比

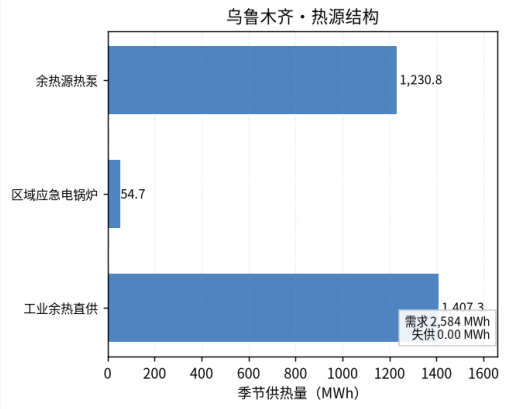

图 25　　乌鲁木齐季节热量来源结构

（四） 工程验收与代码组织

三类场景采用同一组验收条件：热量平衡与储热状态在容差内闭合、聚合热状态处于设定区间、管段流量利用率不超 1、配电容量满足、失供率为零。工程文件将配置与核心逻辑分离，主程序负责调度求解、函数库承担负荷与

设备计算，场景差异由配置文件描述。源码包可在独立环境和移位目录下运行同一实验并得到一致输出，被篡改的输入会被拒绝；目录与运行方式见附录（三）。

五、 模型应用与决策分析

（一） 区域供热转型规划

模型面向供热转型规划、项目比选、投资安排、风险评估和政策分析，输出容量配置、建设时序、运行指标和约束校核结果。在区域规划中，模型比较热源结构、管网温位、储热和建筑改造路径，依据等年值成本、碳排放、峰值化石依赖率和可靠性指标，确定各阶段资产的建设、保留、改造与退役安排。以乌鲁木齐为例，输入为既有热电联产机组、骨干管网与换热站清单、逐时气象与分时电价、余热可用率序列及候选管径，模型给出燃煤退役容量、余热与热泵容量、储热规模和管网降温方案，冻结该投资模式后复算全季运行，输出逐时热源出力、储热充放和购电曲线；决策者可据此比较退役节奏对峰值容量、运行成本和备用需求的影响。

（二） 项目方案比选

候选方案在相同负荷、气象、价格和可靠性边界下比较，评价设备投资、运行费用、温位适配、配电容量、建筑舒适和故障保供，并识别容量余量与关键瓶颈。改造时序与投资安排可按年度预算、施工能力和投产时间约束形成分年度建设清单，并分析项目延期对峰值容量、运行成本和备用需求的影响。

（三） 风险与政策输出

固定投资方案后，对寒潮、资源中断、价格变化、需求响应退出和关键故障重新调度，检查失供、建筑热状态、配电峰值和储热状态，确定备用容量与应急运行边界。通过调整碳约束、清洁热比例、补贴、能源价格和可靠性门槛，可生成成本与减排的候选解集，并按风险权重比较备用能力与尾部费用的取舍。模型结果用于规划决策，施工图、水力平衡、设备选型、地质勘察和现场调试仍由工程专业承担。

六、 模型创新、局限与结论

（一） 模型创新

创新一　温位匹配：热量送到才算数。按供热总量配置设备时，总量平衡并不保证热量能送到高温用户。模型把热源温位下限、管段输送容量和热源出力放在同一组约束内联立，用式(6)把温位不匹配的方案排除在可行域外，候选方案确定后再复算远端温度与压降。该约束参与三城方案的可行性判定，连续温降与末端适配由 72 h 参考管网专项实验承担。

创新二　存量管网：老管道的能力进规划。既有管段容量限制新增热源的接入能力，把管网视为无约束通道会使扩容方案在实施阶段受阻。模型以既有管径为起点，把候选管径和改造投资与热源容量一并纳入比较，使存量资产成为规划变量的一部分。乌鲁木齐本次解在 trunk_b 上选 DN150 而非 DN200，管径是可被目标函数区分的决策；施工条件与路由审批由工程阶段确定。

创新三　燃煤退役：替代容量先落实。按固定退役计划削减燃煤容量，可能在寒潮或余热中断时失去保供能力。模型把退役量与替代容量共同选择，以式(10)限制退出规模，并以式(11)要求任一关键热源退出后其余热源与储热仍覆盖逐时需求。乌鲁木齐本次解退役热电联产 750 kW 与燃气调峰锅炉 900 kW，同步新建应急电锅炉 908.44 kW、配电接入 558.33 kW。

创新四　楼宇蓄热：热状态进逐时调度。把建筑负荷当作给定曲线处理时，用户侧移峰对热源出力和配电的影响无法体现。模型用建筑热状态连接相邻时段，负荷转移先进入节点热平衡，再由热源出力、储热充放和配电容量共同响应。乌鲁木齐本次解的建筑热状态逐时变化（district_a 取值 −29.0—68.2 kWh，标准差 1.33 kWh），确实进入调度；逐楼室温由精确离散 1R1C 模型单独验证。

创新五　算完回算：管网不过关就退回重解。只给出最优解而不回看物理边界，会得到远端压力越限或温度不足的方案。模型把有限管网模式作为规划解的后验检查，逐轮把温度或压降不合格的模式转为新约束并重新求解。该闭环在 72 h 参考管网专项实验执行到第三轮，单位长度压降 237.96 Pa/m、最远站供水温度 57.45 °C 达到登记限值。

创新六　一套内核：三城只换配置。分场景独立建模会使数据定义和指标定义分叉，方案之间无法直接比较。模型以一套公用内核承载负荷分解、建筑

热状态、节点温位平衡、储热递推和配电约束，把存量热网、分散资源与间歇服务写成可加载的场景专用约束，并按城市、区域、用户三层组织变量与衔接量。同一求解入口按三份配置生成三城方案，求解身份见表1。

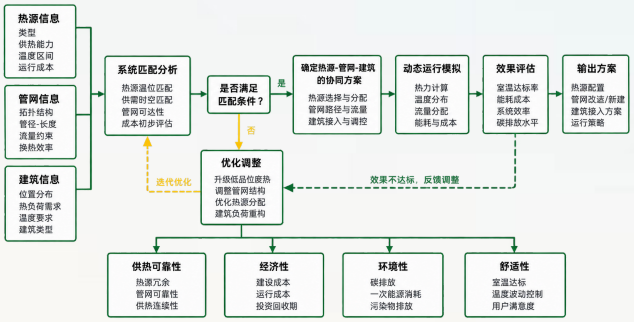

图 26　　创新一：温位与候选管网共同限制可供热量

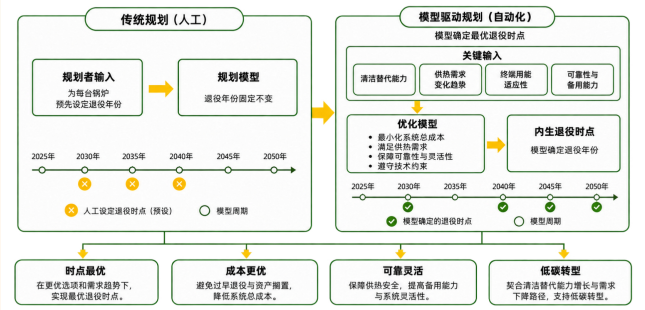

图 27　　创新三：退役规模由替代容量兜底

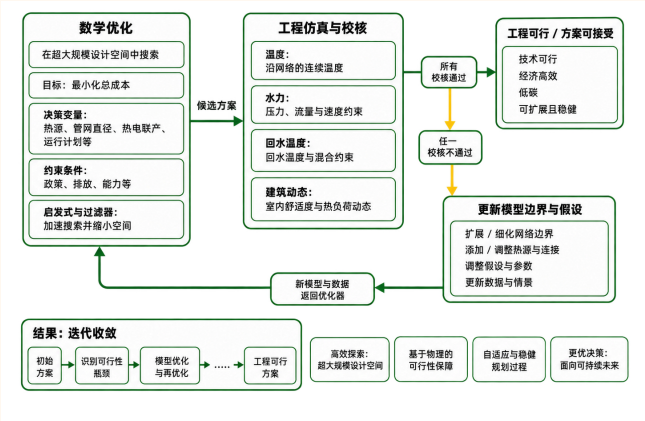

图 28　　创新五：规划解经管网回算反馈重解

（二） 模型局限

模型在固定温位与离散温位近似的框架下求解，连续温差、指数温降与非凸水力列入后续工程校核；建筑热状态采用聚合节点的单一等效热容，房间级温度由逐楼 1R1C 模型承担；热湿、噪声与地源多年动态可在同一框架内按需加载。模型结果用于规划比较，施工图、设备选型、地质勘察、环保审批和配电潮流校核由工程专业承担。

（三） 结论

本文将三类供热转型问题表述为资产配置、空间组织、带温位能流、建筑热状态与储热协同优化，并保留分阶段规划的处理方式。三个场景共享公用内核，分别加载既有热网改造、分散设施选址和夏热冬冷地区供热形态约束，乌鲁木齐获得投资与运行联合解，固原与武汉获得锚定投资后的全季运行结果。该模型体系可为供热系统的建设与退役、储热配置、配电增容和保供方案比较提供分析与决策依据。

参考文献

[1] 中国建筑科学研究院有限公司. 面向供热格局变革的城镇供热源荷匹配与规划寻优模型构建比赛方案[R]. 2026.

[2] 江亿, 付林, 夏建军, 谢晓云, 吴彦廷, 胡姗. 构建新型零碳热力系统[J]. 中国工程科学, 2025, 27(5): 1–18. DOI: 10.15302/J-SSCAE-2025.08.025.

[3] Lin Q, Lu S, Yue L, Li B, Zhu H. Research on low-carbon district heating system planning approach based on thermal energy grade and integrated demand response[EB/OL]. SSRN preprint 5082363, 2024/2025（未同行评审预印本）.

[4] Ceruti A, Lambert J, Spliethoff H. Integrating renewable energy and thermal storage in district heating networks: A design optimization approach[J]. Energy Conversion and Management, 2025, 345: 120323. DOI: 10.1016/j.enconman.2025.120323.

[5] Chen J, Lv H, Ding L, et al. Multi-objective optimization for the operation of district heating systems considering flexibility and economy[J]. Applied Thermal Engineering, 2025, 263: 125395. DOI: 10.1016/j.applthermaleng.2024.125395.

[6] Lyu Z, Huang M, Liu B, Liu Y. Robust optimization of regional integrated energy systems considering thermal comfort and uncertainty in heat networks[J]. Applied Thermal Engineering, 2025, 275: 126895. DOI: 10.1016/j.applthermaleng.2025.126895.

[7] 崔梦颖, 殷勇高, 赵栋霖, 黄钰希, 曹博文. 地埋管跨季节储热耦合热泵供暖系统性能与经济性分析[J]. 储能科学与技术, 2026, 15(3): 983–992. DOI: 10.19799/j.cnki.2095-4239.2026.0107.

[8] 崔杨, 姜涛, 仲悟之, 崔成伟, 赵钰婷. 考虑风电消纳的区域综合能源系统源荷协调经济调度[J]. 电网技术, 2020, 44(7): 2474–2482. DOI: 10.13335/j.1000-3673.pst.2019.2317.

[9] 李仲博. 基于数字孪生的城镇供热系统按需精准调控技术研究与应用[D]. 杭州: 浙江大学, 2023.

[10] Paule D, Pakere I, Pubule J, Blumberga D. Optimising multi-energy community hubs for decarbonisation and energy system flexibility[J]. Smart Energy, 2025, 20: 100214. DOI: 10.1016/j.segy.2025.100214.

[11] Zhang Y, Campana P E, Yang Y, et al. Energy flexibility from the consumer: Integrating local electricity and heat supplies in a building[EB/OL].
[12] Lu H, Khanna N, Feng W, et al. China’s Electrification Pathways: Findings from the China Energy Outlook 2022[R]. Lawrence Berkeley National Laboratory, 2022.
[13] 赵丽丽. 东北农村冬季清洁供暖技术节能改造应用技术研究[J]. 节能, 2025, 44(11): 88–91. DOI: 10.3969/j.issn.1004-7948.2025.11.018.
[14] 白孝轩, 谈润芃, 李朋, 等. 民用燃煤碳排放特征及电能替代减排路径优化[J/OL]. 环境科学研究, 2026. DOI: 10.13198/j.issn.1001-6929.2026.06.17.
[15] Wang S, Su H, Chen C, et al. Natural gas shortages during the “coal-to-gas” transition in China have caused a large redistribution of air pollution[EB/OL]. arXiv:2004.09585, 2020.
[16] Wang X, Xia L, Bales C, et al. A systematic review of recent air source heat pump systems assisted by solar thermal, photovoltaic and photovoltaic/thermal sources[EB/OL]. arXiv:2102.09797, 2021.
[17] Xiong C, Hassan M S. Renewable heat policy in China: Development, achievement, and effectiveness[J]. Sustainability, 2022, 14: 9297. DOI: 10.3390/su14159297.
[18] Fiorentini M, Baldini L. Control-oriented modelling and operational optimization of a borehole thermal energy storage[J]. Applied Thermal Engineering, 2021: 117518. DOI: 10.1016/j.applthermaleng.2021.117518.
[19] Takam P H, Wunderlich R. Model order reduction for the input–output behavior of a geothermal energy storage[J]. Journal of Engineering Mathematics, 2024, 148: 12. DOI: 10.1007/s10665-024-10398-4.
[20] Li H, Svendsen S. Energy and exergy analysis of low temperature district heating network[J]. Energy, 2012, 45(1): 237–246. DOI: 10.1016/j.energy.2012.03.056.
[21] Lund H, Werner S, Wiltshire R, et al. 4th Generation District Heating (4GDH): Integrating smart thermal grids into future sustainable energy systems[J]. Energy, 2014, 68: 1–11. DOI: 10.1016/j.energy.2014.02.089.

[22] Capone M, Guelpa E, Verda V. Potential for supply temperature reduction of existing district heating substations[J]. Energy, 2023, 285: 128597. DOI: 10.1016/j.energy.2023.128597.
[23] IPCC. 2006 IPCC Guidelines for National Greenhouse Gas Inventories, Volume 2: Energy[R]. Hayama: IGES, 2006.
[24] 国务院办公厅. 国务院办公厅关于转发国家发展改革委、住房城乡建设部《加快推动建筑领域节能降碳工作方案》的通知[EB/OL]. https://www.mee.gov.cn/zcwj/gwywj/202403/t20240318_1068599.shtml, 访问时间（2026年7月17日）.
[25] 国家发展和改革委员会. 加强建筑全过程节能降碳管理，推动建筑领域绿色低碳发展[EB/OL]. https://www.ndrc.gov.cn/xxgk/jd/jd/202403/t20240321_1365112.html, 访问时间（2026年7月17日）.
[26] International Energy Agency. Renewable heat: Renewables 2025[EB/OL]. https://www.iea.org/reports/renewables-2025/renewable-heat, 访问时间（2026年7月17日）.
[27] European Commission. Heating and cooling[EB/OL]. https://energy.ec.europa.eu/topics/energy-efficiency/heating-and-cooling_en, 访问时间（2026年7月17日）.
[28] Bacher P, Madsen H. Identifying suitable models for the heat dynamics of buildings[J]. Energy and Buildings, 2011, 43(7): 1511–1522. DOI: https://doi.org/10.1016/j.enbuild.2011.02.005.
[29] Miller C, Kathirgamanathan A, Picchetti B, et al. The Building Data Genome Project 2, energy meter data from the ASHRAE Great Energy Predictor III competition[J]. Scientific Data, 2020, 7: 368. DOI: https://doi.org/10.1038/s41597-020-00712-x.
[30] TICA. V-FORCE Full DC Inverter Modular Chiller (Heat Pump)[Z]. 产品样本，PDF第14–15页. 经销商存档：https://dec.com.pk/assets/images/products/tica/3-Inverter-modular-chiller.pdf. 采用已冻结副本.
[31] NASA POWER. Hourly API[EB/OL]. https://power.larc.nasa.gov/docs/services/api/temporal/hourly/. 三城逐时数据查询与原响应随证据保存.
[32] Mavrotas G. Effective implementation of the ε-constraint method in Multi-Objective Mathematical Programming problems[J]. Applied

Mathematics and Computation, 2009, 213(2): 455–465. DOI: 10.1016/j.amc.2009.03.037.

[33] Liao X, Wang W, Zhou Y. Investigating the Energy-Saving Effectiveness of Envelope Retrofits and Photovoltaic Systems: A Case Study of a Hotel in Urumqi[J]. Sustainability, 2023, 15(13): 9926. DOI: https://doi.org/10.3390/su15139926. 现场照片按CC BY 4.0许可引用.

[34] 中共中央. 中共中央关于制定国民经济和社会发展第十五个五年规划的建议[EB/OL]. 中国共产党第二十届中央委员会第四次全体会议2025年10月23日通过，新华社2025年10月28日受权发布. 国务院公报2025年第31号：https://www.gov.cn/gongbao/2025/issue_12386/202511/content_7047415.html.

[35] 中华人民共和国国民经济和社会发展第十五个五年规划纲要[EB/OL]. 2026年3月发布. 第四十七章第二节“推动重点领域节能降碳”：加快热力系统绿色低碳转型，因地制宜开展余热资源利用和非化石能源供热，有序推进供热计量改造和按热量收费.

附录1　场景约束与校核式

A　公共模型补充约束

A.1　共享余热资源核算

工业余热、污水余热和数据中心余热等低碳资源可能同时被直接换热设备与热泵利用。若分别为两类设备设置资源上限，同一份余热可能在模型中被重复计算。为此，本文按资源池统一核算源侧实际取热量。直接换热设备与热泵共同利用资源池 r 时，有

$$R_{r,t}^{\mathrm{use}} = \sum_{k \in D_r} \frac{Q_{k,t}}{\eta_r^{\mathrm{tr}}} + \sum_{k \in H_r} \frac{\left(1 - \dfrac{1}{\mathrm{COP}_{k,t}}\right) Q_{k,t}}{\eta_r^{\mathrm{tr}}} \le R_{r,t}. \tag{20}$$

其中，D_r 与 H_r 分别为直接换热和热泵集合，η_r^{tr} 为资源端至设备端的传热效率。对于直接换热设备，用户侧供热量经输送效率折算为资源消耗；对于热泵，用户侧供热量扣除输入电功后才是源侧取热量。式(20)保证所有设备的源侧取热量之和不超过时段 t 的可利用资源 $R_{r,t}$。

资源可利用量还应考虑生产节拍、检修计划和季节波动，可写为

$$R_{r,t} = R_r^{\mathrm{rated}} a_{r,t}^{\mathrm{prod}} a_{r,t}^{\mathrm{maint}}, \qquad 0 \le a_{r,t}^{\mathrm{prod}}, a_{r,t}^{\mathrm{maint}} \le 1. \tag{21}$$

其中，$a_{r,t}^{\mathrm{prod}}$ 和 $a_{r,t}^{\mathrm{maint}}$ 分别表示生产可用率和检修可用率。当资源侧停产或检修时，相应时段的资源上限同步下降，模型不能用全年平均余热量替代逐时资源边界。

A.2　设备性能域

热泵的制热量、输入功率和性能系数均随源侧温度、供水温度及部分负荷率变化。为避免采用固定性能系数高估低温工况能力，本文只在厂家同一型号的有效工况点内进行插值：

$$Q_{k,t}^{\mathrm{cat}} = \sum_i \lambda_{k,i,t} Q_{k,i}^{\mathrm{cat}}, \qquad P_{k,t}^{\mathrm{cat}} = \sum_i \lambda_{k,i,t} P_{k,i}^{\mathrm{cat}}, \qquad \sum_i \lambda_{k,i,t} = 1. \tag{22}$$

插值权重 $\lambda_{k,i,t}$ 非负，且仅连接同一型号相邻工况点。缺失点、越界点和跨型号组合不参与优化。工况有效时，实际出力上限和输入电功率为

$$0 \le Q_{k,t}^{\mathrm{HP}} \le \phi_{k,t} Q_{k,t}^{\mathrm{cat}}, \qquad P_{k,t}^{\mathrm{HP}} = \frac{P_{k,t}^{\mathrm{cat}}}{\phi_{k,t} Q_{k,t}^{\mathrm{cat}}} Q_{k,t}^{\mathrm{HP}}. \tag{23}$$

式中，$\phi_{k,t}$ 为除霜、老化、海拔或其他现场因素形成的综合折减系数。当环境温度或供水温度超出厂家性能域时，设备可用状态置为零；当现场修正因子小于1时，额定工况能力按比例下降。由此，容量规划使用的是可实现出力，而非铭牌容量。

A.3 设备启停、爬坡与最小运行时间

燃料锅炉、热泵和电锅炉除受容量上限限制外，还需满足启停与负荷变化约束。设备 k 的逐时出力满足

$$\underline{Q}_k u_{k,t} \le Q_{k,t} \le \overline{Q}_k u_{k,t}, \qquad -R_k^{\mathrm{down}} \le Q_{k,t} - Q_{k,t-1} \le R_k^{\mathrm{up}}. \tag{24}$$

其中，$u_{k,t}$ 为启停状态，R_k^{up} 与 R_k^{down} 分别为升、降负荷速率。启动变量 $v_{k,t}^{\mathrm{start}}$ 和停机变量 $v_{k,t}^{\mathrm{stop}}$ 满足

$$u_{k,t} - u_{k,t-1} = v_{k,t}^{\mathrm{start}} - v_{k,t}^{\mathrm{stop}}, \qquad v_{k,t}^{\mathrm{start}} + v_{k,t}^{\mathrm{stop}} \le 1. \tag{25}$$

对存在最小连续运行时间的设备，还应满足

$$\sum_{\tau=t}^{t+T_k^{\mathrm{on}}-1} u_{k,\tau} \ge T_k^{\mathrm{on}} v_{k,t}^{\mathrm{start}}. \tag{26}$$

上述约束防止优化结果通过每小时频繁启停或瞬时大幅调节获得不可实现的低成本。若设备实际调节性能未知，则应采用保守爬坡率并在敏感性分析中给出变化范围。

A.4 储热状态、互斥运行与期末边界

正文已给出储热状态递推。为保证状态物理可行，储热还需满足容量、充放功率及运行互斥约束：

$$\begin{cases} 0 \le E_{s,t} \le K_s^E, \\ 0 \le Q_{s,t}^{\mathrm{ch}} \le r_s^{\mathrm{ch}} K_s^E y_{s,t}^{\mathrm{ch}}, \\ 0 \le Q_{s,t}^{\mathrm{dis}} \le r_s^{\mathrm{dis}} K_s^E y_{s,t}^{\mathrm{dis}}, \\ y_{s,t}^{\mathrm{ch}} + y_{s,t}^{\mathrm{dis}} \le 1. \end{cases} \tag{27}$$

完整供暖季计算还需设置初末状态：

$$E_{s,1} = E_s^0, \qquad E_{s,T} \ge E_s^{\mathrm{end}}. \tag{28}$$

式(27)排除同一时段同时充放热，式(28)防止模型在最后数小时无偿耗尽储热。若研究采用周期运行假设，可令 $E_{s,T} = E_{s,1}$；若采用实际季初、季末库存，则应按运行制度分别给定。

A.5 管网连续温度与水力校核

主模型采用离散温位和固定供回水温差控制计算规模。候选容量方案确定后，必须恢复连续温度与水力关系进行工程复算。管段出口温度按连续温降关系计算：

$$T_{e,t}^{\mathrm{out}} = T_{e,t}^{\mathrm{env}} + \left(T_{e,t}^{\mathrm{in}} - T_{e,t}^{\mathrm{env}}\right) \exp\left(-\frac{U_e L_e}{\dot{m}_{e,t} c_p}\right). \tag{29}$$

最远节点温度裕度、流量利用率和回水条件分别满足

$$\Delta T^{\mathrm{margin}} = \min_t \left(T^{\mathrm{sup}}_{n,t} - T^{\mathrm{req}}_{n,t} - \Delta T^{\mathrm{hx}}_n \right) \geq 0, \qquad \lambda^{\mathrm{flow}}_e = \max_t \frac{|\dot{m}_{e,t}|}{\overline{\dot{m}}_e} \leq 1, \tag{30}$$

$$Q_{n,t} = \dot{m}_{n,t} c_p (T^{\mathrm{sup}}_{n,t} - T^{\mathrm{ret}}_{n,t}), \quad T^{\mathrm{ret}}_{n,t} \leq \overline{T}^{\mathrm{ret}}_n, \quad T^{\mathrm{sup}}_{n,t} - T^{\mathrm{ret}}_{n,t} \geq \Delta T^{\min}_n. \tag{31}$$

管段压降与泵功率进一步满足

$$\Delta p_{e,t} = f_e \frac{L_e}{D_e} \frac{\rho v^2_{e,t}}{2} + \sum_j \zeta_{e,j} \frac{\rho v^2_{e,t}}{2}, \qquad P^{\mathrm{pump}}_{e,t} = \frac{\dot{m}_{e,t} \Delta p_{e,t}}{\rho \eta^{\mathrm{pump}}_e}. \tag{32}$$

式中，f_e 为沿程阻力系数，D_e 为管径，$\zeta_{e,j}$ 为局部阻力系数。连续复算依次检查最远端温度、关键管段流量、回水温度、供回水温差、压降和泵功率。任一指标越界时，不直接修改计算结果，而是将对应管径、温位或运行模式限制反馈至规划模型重新求解。

A.6 生命周期成本与碳排放核算

生命周期成本由年化投资、固定运维、变动运维、燃料、电力、启停及未供能惩罚构成：

$$\begin{aligned} F^{\mathrm{cost}} = & \sum_k a^{\mathrm{ann}}_k C^{\mathrm{inv}}_k K^{\mathrm{new}}_k + \sum_k C^{\mathrm{fix}}_k K_k + \sum_t \sum_k C^{\mathrm{var}}_{k,t} Q_{k,t} \Delta t \\ & + \sum_t c^{\mathrm{fuel}}_t F^{\mathrm{fuel}}_t \Delta t + \sum_t c^{\mathrm{ele}}_t P^{\mathrm{imp}}_t \Delta t + \sum_{k,t} C^{\mathrm{start}}_k v^{\mathrm{start}}_{k,t} + C^{\mathrm{un}} \sum_t Q^{\mathrm{un}}_t \Delta t. \end{aligned} \tag{33}$$

其中，a^{ann}_k 为年化系数。碳排放与同一组燃料和购电量对应：

$$F^{\mathrm{CO_2}} = \sum_t \left(\sum_f \gamma_f F^{\mathrm{fuel}}_{f,t} + \gamma^{\mathrm{grid}}_t P^{\mathrm{imp}}_t - \gamma^{\mathrm{credit}}_t P^{\mathrm{exp}}_t \right) \Delta t. \tag{34}$$

成本与排放必须共享能耗数据，避免分别核算造成口径不一致。推荐方案中失供量为零，因此未供能惩罚仅用于帮助求解器识别不可行方案，不能替代零失供硬约束。

A.7 完整供暖季回放与验收

若容量规划采用代表日降低求解规模，代表日仅用于产生候选规划变量。候选方案确定后，固定热源、管网、储热与配电容量，按原始逐时序列重新计算完整供暖季。回放过程中，建筑温度、储热存量、燃料库存和设备启停状态均继承上一时段结果，不在日界或月界重置。

统一验收条件为

$$\max_{n,g,t} |r^{Q}_{n,g,t}| \le \varepsilon_Q, \quad \max_{s,t} |r^{E}_{s,t}| \le \varepsilon_E, \quad \lambda_e^{\text{flow}} \le 1, \quad U^{\text{un}} = 0. \tag{35}$$

除式(35)外，所有时段均须满足室内温度、设备出力、储热容量、管网温度与压降、配电容量及场景专用约束。任一时段不通过，即判定候选方案动态验证失败，并返回规划层调整容量或运行模式。

B 乌鲁木齐场景补充约束

B.1 既有热源分阶段退役

乌鲁木齐是在既有大型集中供热系统上进行低碳重构，退役决策具有明确的时间顺序。阶段 y 的燃煤热源退役量不得超过上一阶段已投运容量，达到政策退役期限后相应容量必须退出：

$$K^{\mathrm{ret}}_{\mathrm{coal},y} \le K^{\mathrm{exist}}_{\mathrm{coal},y-1}, \qquad y \ge y^{\mathrm{deadline}} \Rightarrow K^{\mathrm{exist}}_{\mathrm{coal},y} = 0. \tag{36}$$

正文式(10)进一步要求退役容量由新增清洁热源、储热有效容量和备用容量共同补足。其中，储热有效容量是设计峰值时段可持续释放的热功率，而非储热装置的额定储能量。只有替代能力在容量和持续时间上均足以覆盖退役缺口，退役方案才可进入逐时运行验证。

B.2 工业余热、大型热泵与配电边界

工业余热的可利用量同时受资源侧生产状态、回收效率和输送损失影响。余热温度低于高温热网要求时，须经大型热泵提升温位。相应约束为

$$\begin{gathered} 0 \le Q^{\mathrm{waste}}_t \le \eta^{\mathrm{recovery}} Q^{\mathrm{available}}_t, \quad Q^{\mathrm{HP}}_t = \mathrm{COP}_t P^{\mathrm{HP}}_t, \\ P^{\mathrm{HP}}_t + P^{\mathrm{base}}_t + P^{\mathrm{pump}}_t \le \overline{P}^{\mathrm{grid}}. \end{gathered} \tag{37}$$

式(37)第一项限制资源总量，第二项描述热泵热电转换，第三项保证热泵、循环泵和基础用电不超过区域配电容量。COP_t 应由资源侧温度、供水温度、温升和部分负荷率共同确定，不宜使用全年常数。若配电容量不足，模型需在配电增容、降低热泵规模、增配储热或保留其他调峰热源之间重新权衡。

B.3 温位兼容与直接供热判定

大型集中供热系统不仅要求热量守恒，还要求热能品质匹配。定义热源 k 与供热温位 g 的兼容参数

$$A_{kg}^{\text{grade}} = \begin{cases} 1, & T_k^{\text{src,min}} \geq T_g^{\text{sup}} + \Delta T_g^{\text{hx}}, \\ 0, & \text{其他情况}, \end{cases} \qquad 0 \leq Q_{k,g,t}^{\text{direct}} \leq A_{kg}^{\text{grade}} \overline{Q}_k. \tag{38}$$

当 $A_{kg}^{\text{grade}} = 0$ 时，对应热源不得直接承担该温位负荷，必须通过热泵或其他增温设备转换。该约束避免“热量足够即视为可用”的错误判断，尤其适用于低温工业余热接入75/45 °C等高温等级热网的情形。

B.4 骨干管网容量与最远端温度

热源替代会改变既有骨干管网的潮流方向和峰值流量。既有管段与新增管段共同承担的总流量满足

$$\dot{m}_{e,t}^{\text{total}} = \dot{m}_{e,t}^{\text{exist}} + \dot{m}_{e,t}^{\text{new}}, \qquad |\dot{m}_{e,t}^{\text{total}}| \leq \overline{\dot{m}}_e. \tag{39}$$

最远换热站供水温度及裕度满足

$$T_{n^{\text{far}},t}^{\text{sup}} \geq T_{n^{\text{far}},t}^{\text{req}} + \Delta T_{n^{\text{far}}}^{\text{hx}}, \qquad \Delta T_{n^{\text{far}}}^{\text{margin}} = \min_t \left(T_{n^{\text{far}},t}^{\text{sup}} - T_{n^{\text{far}},t}^{\text{req}} - \Delta T_{n^{\text{far}}}^{\text{hx}} \right) \geq 0. \tag{40}$$

若式(39)超限，则需要增大管径、增设联络线或调整热源空间布局；若式(40)不满足，则需要提高源侧温度、改善保温、增加流量或改造末端。两类问题不能通过增加系统总热量自动解决。

B.5 热源故障与备用能力

正文式(11)已给出关键热源退出后的逐时供需边界。为避免系统刚好满足负荷但没有调节余量，还需保留规定备用裕度：

$$\sum_{j \neq k} \left(\overline{Q}_j - Q_{j,t} \right) + \overline{Q}^{\text{storage,dis}} \geq R_t^{N-1}, \qquad \forall k \in K^{\text{critical}}. \tag{41}$$

每一台关键热源均需单独构造故障情景。故障设备的可用率置为零，其出力不得继续计入平衡；储热只能按故障发生时的实际存量提供支撑，不能直接

使用额定容量。若任一故障情景出现失供，则应增加备用热源、储热放热能力或管网互联能力。

B.6 寒潮零失供

寒潮通常同时造成室外温度下降、建筑负荷增加和工业余热可用量变化，因此三类输入需同步修正，并要求逐时零失供：

$$
\begin{aligned}
T_t^{\mathrm{out,cold}} &= T_t^{\mathrm{out}} - \Delta T^{\mathrm{cold}}, & D_t^{\mathrm{cold}} &= \alpha_D D_t, \\
Q_t^{\mathrm{waste,cold}} &= \alpha_W Q_t^{\mathrm{waste}}, \\
Q_t^{\mathrm{un,cold}} &= \max\left(0, D_t^{\mathrm{cold}} - \sum_k Q_{k,t}^{\mathrm{cold}} - Q_t^{\mathrm{storage,dis,cold}}\right) = 0.
\end{aligned}
\tag{42}
$$

寒潮校核必须保持原始连续时间顺序，使建筑热状态、储热存量和设备状态从寒潮前逐时传递至寒潮结束。零失供要求每个时段的 $Q_t^{\mathrm{un,cold}}$ 均为零，而不是仅要求全季平均失供率较低。

B.7 管网降温与建筑末端改造联动

降低热网供水温度有利于扩大余热利用范围并提高热泵性能，但老旧建筑散热器或换热器可能无法在低温工况下满足室温。建筑末端改造变量 z_b^{retro} 决定最低供水温度：

$$
T_b^{\mathrm{req}} = (1 - z_b^{\mathrm{retro}}) T_b^{\mathrm{old}} + z_b^{\mathrm{retro}} T_b^{\mathrm{new}}, \qquad T_{b,t}^{\mathrm{sup}} \geq T_b^{\mathrm{req}} + \Delta T_b^{\mathrm{hx}}. \tag{43}
$$

改造投资及室温还应满足

$$
\sum_b C_b^{\mathrm{retro}} z_b^{\mathrm{retro}} \leq B^{\mathrm{retro}}, \qquad \theta_{b,t}^{a} \geq \theta_b^{\min}. \tag{44}
$$

式(43)连接管网供水温度与建筑末端能力，式(44)限制改造规模并保持室内舒适度。只有管网温度、末端换热能力、投资预算和室温同时满足时，降温方案才具有工程可行性。

B.8　乌鲁木齐场景验收指标

乌鲁木齐候选方案除满足公共模型约束外，还需报告：燃煤热源分阶段退役量、清洁替代容量、余热利用率、热泵逐时性能系数、配电峰值、关键管段最大流量利用率、最远换热站最低温度裕度、回水温度、供回水温差、各关键热源 $N-1$ 校核结果、寒潮最大负荷时的备用裕度，以及末端改造面积与投资。任一指标越界均应返回规划层调整，不以全年平均指标替代最不利时段判定。

C 固原场景补充约束

C.1 热源选址、用户分配与服务半径

固原场景首先确定候选热源是否建设以及各村镇、片区由哪个热源服务。正文式(12)保证每个用户仅分配给一个已建热源，并同时满足服务半径和峰值容量。选址变量与分配变量满足

$$z_i^{\mathrm{site}} \in \{0,1\}, \qquad x_{i,b} \in \{0,1\}. \tag{45}$$

服务半径应使用道路或可敷设管线距离，而非简单直线距离。峰值容量还应考虑同时使用系数和备用裕度：

$$\sum_b \beta_{i,b} D_b^{\mathrm{peak}} x_{i,b} \le (1-\rho_i^{\mathrm{reserve}}) K_i. \tag{46}$$

其中，$\beta_{i,b}$ 为同时使用系数，$\rho_i^{\mathrm{reserve}}$ 为备用比例。若候选点不能满足距离、容量或土地条件，则相应用户分配变量必须为零。

C.2 微网拓扑与管网连通性

仅确定热源位置和用户归属并不能保证实际可供热，还需确保热源与用户之间存在连续可建的管网路径。候选管段建设变量 z_e^{pipe} 与热流满足

$$|F_{e,t}| \le \overline{F}_e z_e^{\mathrm{pipe}}, \qquad x_{i,b} \le \sum_{e\in\mathcal{P}_{i,b}} z_e^{\mathrm{pipe}}. \tag{47}$$

其中，$\mathcal{P}_{i,b}$ 为热源 i 至用户 b 的候选路径集合。第一项保证未建设管段不能传热，第二项要求已分配用户至少存在一条有效路径。对于树状微网，可进一步限制建设管段数以避免孤立支路；对于环状微网，则可通过联络线提高故障恢复能力。

区域节点 i 的逐时热量平衡为

$$\sum_{k} Q_{k,i,t} + Q_{i,t}^{\text{storage,dis}} + \sum_{e\in\delta^-(i)} \eta_e F_{e,t} = D_{i,t} + Q_{i,t}^{\text{storage,ch}} + \sum_{e\in\delta^+(i)} F_{e,t}. \tag{48}$$

该式将热源出力、管网输入、储热放热、用户负荷、储热充热和管网输出置于同一节点，避免区域总量平衡掩盖局部支路欠供。

C.3 生物质燃料供应、运输与库存

固原生物质系统的主要风险不是锅炉额定容量不足，而是燃料采购、运输和库存无法贯穿完整供暖季。正文式(13)给出库存递推，库存量、到货量及可用供热量还需满足

$$0 \le I_{i,t} \le \overline{I}_i, \qquad 0 \le B_{i,t}^{\text{arrive}} \le \overline{B}_{i,t}^{\text{arrive}}, \qquad Q_{i,t}^{\text{bio}} \le \eta_i^{\text{bio}} (I_{i,t-1} + B_{i,t}^{\text{arrive}}). \tag{49}$$

若燃料来自多个供应点 s，运输能力满足

$$B_{i,t}^{\text{arrive}} = \sum_{s} B_{i,s,t}^{\text{arrive}}, \qquad \sum_{s} d_{i,s} B_{i,s,t}^{\text{arrive}} \le \overline{L}_{i,t}^{\text{transport}}. \tag{50}$$

其中，$d_{i,s}$ 为供应点至热源的运输距离，$\overline{L}_{i,t}^{\text{transport}}$ 为时段运输能力。道路封闭、车辆检修或极端天气可通过降低该上限描述。为避免在供暖季末耗尽燃料，还需设置安全库存：

$$I_{i,t} \ge I_i^{\text{safe}}, \qquad t \in T^{\text{risk}}, \quad I_{i,T} \ge I_i^{\text{end}}. \tag{51}$$

库存不足时，生物质锅炉的可用出力必须同步下降，不能继续按额定容量供热。运输距离、到货能力和安全库存共同决定生物质方案的真实可靠性。

C.4 分布式设备与区域电力平衡

分布式热泵、电锅炉、光伏、储电和循环泵共同构成区域微网。设备出力满足

$$0 \le Q_{i,t}^{\mathrm{HP}} \le a_{i,t}^{\mathrm{HP}} K_i^{\mathrm{HP}}, \qquad 0 \le Q_{i,t}^{\mathrm{EB}} \le K_i^{\mathrm{EB}} u_{i,t}^{\mathrm{EB}}. \tag{52}$$

区域微网电力平衡为

区域微网电力平衡及关键用户孤岛保供约束为

$$P_{i,t}^{\mathrm{imp}} + P_{i,t}^{\mathrm{PV}} + P_{i,t}^{\mathrm{bat,dis}} = P_{i,t}^{\mathrm{base}} + P_{i,t}^{\mathrm{HP}} + P_{i,t}^{\mathrm{EB}} + P_{i,t}^{\mathrm{pump}} + P_{i,t}^{\mathrm{bat,ch}}, \tag{53}$$

并满足配电容量与储电状态约束：

$$P_{i,t}^{\mathrm{imp}} \le P_i^{\mathrm{grid,exist}} + K_i^{\mathrm{grid,new}}, \qquad E_{i,t}^{\mathrm{bat}} = E_{i,t-1}^{\mathrm{bat}} + \eta_i^{\mathrm{bat,ch}} P_{i,t}^{\mathrm{bat,ch}} \Delta t - \frac{P_{i,t}^{\mathrm{bat,dis}} \Delta t}{\eta_i^{\mathrm{bat,dis}}}. \tag{54}$$

上述约束把供热设备电气化带来的峰值负荷传递至配电规划。若配电容量成为瓶颈，模型可选择储热、储电、光伏、本地燃料热源或配电增容，但不得忽略循环泵和基础用电。

C.5 孤岛运行与关键用户保供

固原部分区域外部电网薄弱，医院、学校、养老设施及集中安置点等关键用户需在外部电网中断或单一热源故障时保持基本供热。可用热源、储热和防冻结约束为

$$\sum_{k \in K_i^{\mathrm{available}}} Q_{k,t} + Q_{i,t}^{\mathrm{storage,dis}} \ge D_{i,t}^{\mathrm{critical}}, \qquad \dot{m}_{e,t} \ge \dot{m}_e^{\min}, \qquad T_{e,t}^{\mathrm{ret}} \ge T_e^{\mathrm{freeze}}. \tag{55}$$

关键用户供热保障率满足

$$\frac{\sum_t Q_{i,t}^{\mathrm{critical,served}} \Delta t}{\sum_t D_{i,t}^{\mathrm{critical}} \Delta t} \ge \alpha_i^{\mathrm{reliability}}. \tag{56}$$

完全孤岛运行时，本地电源还需覆盖基本设备用电：

$$P_{i,t}^{\mathrm{local}} + P_{i,t}^{\mathrm{bat,dis}} \geq P_{i,t}^{\mathrm{HP}} + P_{i,t}^{\mathrm{EB}} + P_{i,t}^{\mathrm{pump}} + P_{i,t}^{\mathrm{critical,base}}. \tag{57}$$

孤岛情景中，外部购电置为零，只允许实际可用的本地电源、储电和燃料热源参与供能。储能初始状态取故障发生时的真实状态，不能假设每次故障前均为满储。

C.6 防冻结与极寒循环

低温环境下停泵或低流量运行可能造成支路、换热器和末端冻结。除式(55)给出的最低流量与回水温度外，极寒情景还应满足

$$T_{e,t}^{\mathrm{water}} \geq T_{e}^{\mathrm{freeze}} + \Delta T_{e}^{\mathrm{safe}}, \qquad u_{e,t}^{\mathrm{pump}} = 1, \quad t \in T^{\mathrm{extreme}}. \tag{58}$$

若管段停运，则需采用排空、防冻液或伴热措施，并将相应投资和运行能耗纳入成本。防冻结约束的作用是保证“少供热”不会转化为管网损坏风险。

C.7 运维可达性与居民可负担性

分散设备数量多、空间分布广，运维距离过长会降低故障处置能力。候选方案满足

$$d_{i}^{\mathrm{maint}} \leq \overline{d}_{i}^{\mathrm{maint}}, \qquad C_{b}^{\mathrm{heat}} \leq \overline{C}_{b}^{\mathrm{afford}}. \tag{59}$$

第一项限制运维站点到设备的可达距离或响应时间，第二项限制居民热费。对不满足可负担性要求的方案，应通过财政补贴、降低燃料物流成本、提高设备效率或调整服务范围进行修正，而不能仅以系统总成本较低判定为可实施。

C.8 固原场景验收指标

固原候选方案应报告热源选址结果、用户归属、最大服务半径、管网连通状态、峰值容量裕度、生物质最大日耗量、最小库存、运输能力利用率、配电

峰值、孤岛持续时间、关键用户保障率、最低循环流量、最低回水温度、运维响应距离及居民热费。任何用户未被唯一分配、任何已分配用户不存在管网路径、燃料库存出现负值或关键用户在故障情景中失供，均判定方案不可行。

D 武汉场景补充约束

D.1 供能形态与设备匹配

武汉无传统集中供热资产，规划需在独立楼宇、区域能源站、低温热网和第五代区域供热供冷系统之间选择。正文式(14)保证每个区域仅选择一种基本形态。设备建设变量还需服从形态选择：

$$x_{k,i} \le \sum_{m \in M_k} z_i^m, \qquad K_{k,i} \le \overline{K}_{k,i} \sum_{m \in M_k} z_i^m. \tag{60}$$

其中，M_k 为允许配置设备 k 的供能形态集合。该约束防止模型在选择独立楼宇方案后仍调用区域管网设备，或在未建设能源站时使用站内储热和集中泵组。若允许不同形态分阶段过渡，则应增加建设年份和退役年份变量，而不能在同一阶段同时启用互斥形态。

D.2 供热、供冷与生活热水耦合

武汉同时存在冬季供热、夏季供冷和全年生活热水需求。正文式(15)给出供热与生活热水平衡，供冷设备满足

$$Q_{i,t}^{\mathrm{cool}} = \mathrm{COP}_{i,t}^{\mathrm{cool}} P_{i,t}^{\mathrm{cool}}, \qquad 0 \le Q_{i,t}^{\mathrm{cool}} \le a_{i,t}^{\mathrm{cool}} K_i^{\mathrm{cool}}. \tag{61}$$

区域总电力需求为

$$P_{i,t}^{\mathrm{total}} = P_{i,t}^{\mathrm{base}} + P_{i,t}^{\mathrm{heat}} + P_{i,t}^{\mathrm{cool}} + P_{i,t}^{\mathrm{DHW}} + P_{i,t}^{\mathrm{pump}} + P_{i,t}^{\mathrm{dehumid}}. \tag{62}$$

式(62)将采暖、制冷、生活热水、输配和除湿用电统一计入配电峰值。冷热回收设备工作时，还需保证回收热量不超过制冷侧排热量和生活热水需求，避免同时高估制冷收益与余热利用收益。

D.3 间歇运行与预热恢复

南方建筑具有明显的间歇使用特征，办公、商业和公共建筑可在非使用时段降低设定温度，但重新投入使用前必须完成预热。设备启动状态满足

$$u_{b,t}-u_{b,t-1}\le v^{\mathrm{start}}_{b,t},\qquad \sum_{\tau=t}^{t+T^{\min}_b-1}u_{b,\tau}\ge T^{\min}_b v^{\mathrm{start}}_{b,t}. \tag{63}$$

使用开始时刻 t^{occ}_b 前的室温恢复满足

$$\theta^{a}_{b,t^{\mathrm{occ}}_b}\ge\theta^{\mathrm{occ,min}}_b,\qquad \theta^{a}_{b,t}\ge\theta^{\mathrm{setback,min}}_b,\quad t<t^{\mathrm{occ}}_b. \tag{64}$$

第一项防止频繁启停，第二项保证建筑投入使用时已达到舒适温度。非使用时段的温度下限仍需防止结露和过度降温，不能将停供时段负荷简单删除。

D.4 室内热湿舒适与最低新风

武汉冬季湿冷、夏季高温高湿，仅控制空气温度不足以反映使用品质。室内温湿度和最低新风约束为

热湿舒适、新风和结露约束为

$$\theta^{\min}_{b,t}\le\theta^{a}_{b,t}\le\theta^{\max}_{b,t},\qquad RH^{\min}_b\le RH_{b,t}\le RH^{\max}_b,\qquad V^{\mathrm{vent}}_{b,t}\ge\underline{V}^{\mathrm{vent}}_b, \tag{65}$$

供暖与供冷季可采用不同温度边界，新风带来的显热负荷为

$$Q^{\mathrm{vent}}_{b,t}=\rho_a c_a V^{\mathrm{vent}}_{b,t}\left(\theta^{a}_{b,t}-T^{\mathrm{out}}_t\right). \tag{66}$$

湿负荷则根据室内外含湿量差和新风量计算，并计入除湿设备负荷。最低新风量不能因能源成本升高而被削减；当湿度越界时，应增加除湿能力、调整送风状态或改变末端形式。

D.5 表面与管网防结露

低温辐射末端、冷水管道和换热设备表面温度低于露点时会产生结露，造成腐蚀、霉变和装修损坏。因此，正文的末端约束还需扩展至管网：

$$T_{b,t}^{\text{surface}} \geq T_{b,t}^{\text{dew}} + \Delta T_b^{\text{safe}}, \qquad T_{e,t}^{\text{sup}} \geq T_{e,t}^{\text{dew}} + \Delta T_e^{\text{safe}}. \tag{67}$$

露点温度由室内空气温度和相对湿度计算，ΔT^{safe} 为测量误差、局部温差和控制滞后预留的安全裕度。若式(67)不满足，应提高供水温度、降低湿度、增加保温或停用相应辐射末端，不能继续按理想无结露状态运行。

D.6 空气源热泵除霜与低温运行

空气源热泵在低温高湿条件下容易结霜，除霜期间制热能力下降并产生额外耗电。实际供热量和输入功率满足

空气源热泵除霜和地源年度平衡满足

$$Q_t^{\text{HP,actual}} = (1 - \phi_t^{\text{defrost}}) Q_t^{\text{HP,rated}}, \qquad P_t^{\text{HP,actual}} = P_t^{\text{HP,rated}} + P_t^{\text{defrost}}, \tag{68}$$

并要求

$$T_t^{\text{out}} \geq T_k^{\text{HP,min}} \quad \text{或} \quad u_{k,t}^{\text{HP}} = 0, \qquad 0 \leq \phi_t^{\text{defrost}} \leq \overline{\phi}^{\text{defrost}}. \tag{69}$$

除霜折减系数应由室外温度、相对湿度和厂家数据确定。若低于设备最低运行温度，则该设备不可用，需要由备用热源、储热或其他系统承担负荷。这样可避免按额定性能高估湿冷天气下的热泵能力。

D.7 地源系统年度热量平衡

地源系统具有跨季节累积效应，只计算供暖季会高估持续取热能力。年度取热与排热差满足

$$\left| \sum_t Q_t^{\text{extraction}} \Delta t - \sum_t Q_t^{\text{injection}} \Delta t \right| \leq \varepsilon^{\text{ground}}. \tag{70}$$

地温状态还需限制在允许范围内：

$$T_s^{\text{ground,min}} \le T_{s,t}^{\text{ground}} \le T_s^{\text{ground,max}}. \tag{71}$$

式(70)控制年度净取热量，式(71)控制长期地温漂移。若年度不平衡，可通过夏季回灌、冷却塔辅助散热、扩大埋管规模或与空气源设备混合运行修正。

D.8 噪声与邻近建筑约束

楼栋热泵、冷却塔和能源站可能靠近住宅、学校或办公建筑，设备容量可行不代表环境影响可接受。正常和夜间运行噪声满足

$$L_{b,t}^{\text{noise}} \le \overline{L}_b^{\text{noise}}, \qquad L_{b,t}^{\text{noise}} \le \overline{L}_b^{\text{noise,night}}, \quad t \in T^{\text{night}}. \tag{72}$$

多台设备同时运行时，应按声功率叠加而非算术相加。若夜间噪声越界，可采用低噪声设备、隔声屏障、距离衰减或运行时段调整，并将相应投资和能力折减纳入规划。

D.9 楼栋配电容量

武汉场景大量采用电驱热泵、制冷机和生活热水设备，楼栋既有配电容量可能成为决定性边界：

楼栋配电容量满足

$$P_{b,t}^{\text{HP}} + P_{b,t}^{\text{cool}} + P_{b,t}^{\text{DHW}} + P_{b,t}^{\text{base}} \le P_b^{\text{exist}} + K_b^{\text{grid,new}}. \tag{73}$$

若配置充电设施、除湿设备或其他新增电负荷，也应计入左侧。新增配电容量 $K_b^{\text{grid,new}}$ 需进入投资成本，且受变压器位置、接入条件和建设上限约束。模型不得只比较热源侧成本而忽略楼栋电气改造。

D.10 储热削峰与多服务竞争

采暖、供冷和生活热水峰值可能叠加，储热可在低负荷或低电价时段充热，在峰值时段放热。除公共储热约束外，多服务需求满足

$$Q_t^{\mathrm{heat}} + Q_t^{\mathrm{DHW}} \le Q_t^{\mathrm{source}} + Q_t^{\mathrm{storage,dis}}, \qquad E_{s,T} \ge E_s^{\mathrm{end}}. \tag{74}$$

储热放热应在不同服务之间显式分配，不能将同一放热功率同时计入采暖和生活热水。期末存量边界防止模型通过耗尽储热获得虚假削峰效果。

D.11 武汉场景验收指标

武汉候选方案应报告供能形态选择、设备容量、采暖与生活热水逐时平衡、供冷性能、使用开始时刻室温恢复、温湿度越界小时数、最低新风量、最小结露安全裕度、除霜期最大能力折减、年度地源取排热偏差、夜间噪声、楼栋配电峰值及储热削峰贡献。任何时段出现舒适度、结露、设备运行温度或配电容量越界，均判定方案不可行。

附录2　符号与缩略语说明

本表仅列出贯穿正文的核心符号。场景专用变量已在相应约束后就地说明，不再重复汇总。

表 2　　模型核心符号

符号	含义	单位
t, T	运行时段及完整供暖季总时段数	h
k, n, g, b, s, e	设备、节点、温位、建筑、储热装置和管段索引	–
K_k	热源或设备规划后可用容量	kW
$Q_{k,g,t}$	热源向温位 g 提供的热功率	kW
$F_{e,g,t}$	管段入口热功率	kW
$Q^{h}_{b,t}$	建筑实际获得的供热功率	kW
$\theta^{a}_{b,t}, \theta^{m}_{b,t}$	室内空气温度和建筑质量节点温度	°C
$E_{s,t}$	储热装置存量	kWh
$\mathrm{COP}_{k,t}$	热泵逐时性能系数	–
F^{cost}	生命周期成本	元
$F^{\mathrm{CO_2}}$	系统碳排放量	kg
U^{un}	供暖季失供率	–

附录3　核心代码（Python详编）

本附录沿用原有12段代码的顺序，补入共享热资源、逐楼热状态、配电边界、硬约束权重及紧凑启停实现，共17段、1066行原代码和224行补充注释。代码均从随包R8O固定源码连续节选，按输入输出、实现思路、关键注释和结果核对展开；正文模型与算法结构保持原样。

阅读顺序。第1段说明入口；第2–4段连接建筑、储热与节点热平衡；第5–6段完成稀疏求解及数值复核；第7–8段将规划资产交给连续供暖季运行；第9–12段说明多目标、权重诊断和管网反馈。新增第13–15段补齐跨模块的物理衔接，第16–17段展示权重优化与整数规模压缩的具体实现。

显示行号从每段1开始并包含新增注释，索引给出原文件准确行号。`code_annotated_r8p_a4`内保存原样节选、注释版及逐行映射，`code_annotation_manifest_a4.json`记录来源和指纹。删除新增注释后，原代码行及可执行token一致；可以独立解析的片段另核对AST。代码排版沿用原有配色、字体与框线，上下文片段仍须在完整工程中调用。

源码说明各接口的实现，实验是否执行及其结果以对应运行记录为准。代表日规划、固定投资复算、两阶段端点和独立碳帽扫描分别保留执行身份；下文在相关代码处说明开关和衔接条件。

表 3　　详编代码来源与正文方法对应

序号	文件与原行号	对应内容
1	`unified_cli.py`，93–118	单次求解入口与导出
2	`building_thermal.py`，31–46	逐楼1R1C精确离散系数
3	`model.py`，1391–1471	储热容量、运行与状态边界
4	`model.py`，1603–1697	节点温位逐时热平衡
5	`linear.py`，177–205	SciPy稀疏MILP求解接口
6	`linear.py`，40–105	独立数值与守恒残差复核
7	`experiments.py`，13–147	完整投资冻结与费用账本
8	`heating_season.py`，183–246	固定投资的连续供暖季复算
9	`model.py`，2829–2888	带主目标容差的两阶段端点
10	`experiments.py`，351–411	独立碳帽采样与非支配筛选

序号	文件与原行号	对应内容
11	representative_adoption.py，48–94	代表日硬能量约束可行性诊断
12	network_feedback.py，440–481	有限模式管网反馈与失败处理
13	physical_extensions.py，25–79	热源出力的上游余热共用约束
14	building_thermal.py，463–525	逐楼RC状态、舒适边界与节点交付量
15	model.py，2030–2089	逐时配电接入容量及可靠容量方法入口
16	hard_weight_calibration.py，13–97	硬能量守恒下的最小一维W1权重线性规划
17	model.py，938–1018	热源启停三行紧凑凸包与容量关联爬坡

正文算法1对应规划与固定投资复算；算法2对应独立碳帽采样与复核。第16段补充的是代表日权重优化本体，第11段则是等式与下界能否同时成立的诊断，两者作用分开。

算法2　独立费用与碳排放采样

输入：同一配置与时间序列、碳帽档数 $K \geq 2$。

输出：通过复核的非支配方案、边际减排费用与折中建议。

1. 清除原碳帽并令碳价为零，保留设备、运行与可靠性约束。
2. 分别以费用和排放为主目标求端点，据两端排放值等距设置 K 档碳帽。
3. 对每档碳帽重新构模求解，独立检查数值残差与碳帽是否成立。
4. 对通过的样本去重并剔除受支配点，计算相邻方案的边际减排费用，用归一化理想点距离给出折中建议。

1. 单次求解入口与导出

主入口中没有政策路径时的单次求解分支；配置可以按需展开相关天气场景，随后统一求解和导出。

输入：当前profile的配置路径、逐时输入路径、可选场景说明及求解预算。

输出：投资、逐时运行、汇总指标与场景结果文件。该节选依赖外层profile循环。

实现思路。该入口负责把配置、逐时输入和求解预算连接起来。需求列从节点配置取得，避免在场景展开时漏掉某一节点；随后将场景风险设置写入配置副本，再统一校验。HeatPlanningModel接收校验后的同一组输入，导出也使用模型实际采用的时序，因而输入、求解与结果文件可以一一对应。

来源：unified_cli.py，原第93–118行；26行原代码、10行补充注释。

```
else:
    # 配置与时序共同定义算例身份；本段不自动启动碳帽扫描。
    config, frame = load_inputs(config_path, timeseries_path)
    # 仅在提供场景文件时展开不确定场景；无文件则沿用读入时序。
    if scenario_specs_path:
        specs =
        ↪  yaml.safe_load(scenario_specs_path.read_text(encoding="utf-8"))
        # 按节点配置收集需求列，保持场景扩展与需求接口一致。
        demand_columns = [
            column for node in config["nodes"]
            for column in node.get("demand_columns", {}).values()
        ]
        frame = build_correlated_northern_scenario_bundle(
            frame, demand_columns, specs["scenarios"],
            # period_weight是代表时段权重，不等同于额外的物理小时步长。
            period_weight=float(specs.get("period_weight",
            ↪  config["planning"].get("representative_day_weight", 1.0))),
        )
        # 复制后修改风险设置，保留此前读入的配置对象。
        config = copy.deepcopy(config)
        config["planning"]["risk"] = dict(specs.get("risk", {}))
        # 场景展开后再次校验列名、参数和时序约定。
        validate_inputs(config, frame)
    # 构造公用模型；源、网、荷、储与配电模块由配置装配。
    model = HeatPlanningModel(config, frame)
    # mip_gap和time_limit限制求解预算，不是物理可行性容差。
    artifacts = model.solve(
        mip_gap=args.mip_gap,
        time_limit=args.time_limit,
    )
    # 导出所得解及其实际输入；无可行解会由上层模型抛出异常。
    export_results(profile_output, artifacts, config, model.ts)
```

```
    summary = {key: value for key, value in artifacts.summary.items() if not
    ↪ isinstance(value, dict)}
    # 场景统计另存，避免以全系统平均掩盖某个场景的失供。
    (profile_output / "scenario_results.json").write_text(
        json.dumps(artifacts.summary.get("scenario_results", {}),
        ↪ ensure_ascii=False, indent=2),
        encoding="utf-8",
    )
```

结果核对。阅读结果时先检查场景文件是否提供，再核对导出的时序、求解状态及逐场景结果。单次入口的结果不能仅凭文件存在就视为已执行碳帽扫描。

2．逐楼1R1C精确离散系数

独立逐楼热模型的完整系数函数。逐小时输入保持常值时，系数连接小时初末室温与交付冷热量。

输入：热损失系数H（kW/K）、有效热容C（kWh/K）和步长（h）。

输出：无量纲衰减系数a和温升系数b（K/kW）。函数不进行参数辨识。

实现思路。热平衡微分方程在一个时段内采用分段常值外温与净热输入。代码计算$a = \exp(-H\Delta t/C)$和$b = (1-a)/H$，使下一时段室温可直接写成当前室温、外温和净热输入的线性组合。系数由输入参数预先计算，优化时只含状态和供热变量，既保留热惯性，又不引入决策变量之间的乘积。

来源：building_thermal.py，原第31–46行；16行原代码、5行补充注释。

```
# 本函数属于独立1R1C分支，不替换历史聚合需求响应状态。
def exact_rc_coefficients(
    heat_loss_kw_per_c: float, thermal_capacity_kwh_per_c: float, dt_hours:
    ↪ float,
) -> tuple[float, float]:
    """Return a, b for T_end = a*T_start + (1-a)*Tout + b*(heat-cool+gains).

    expm1 avoids cancellation at small H*dt/C. H=0 is deliberately rejected:
    this module requires a positive envelope/ventilation heat loss
↪   coefficient.
    """
    # H、C与步长必须为正且有限，单位应先统一。
    h = _number(heat_loss_kw_per_c, "heat_loss_kw_per_c", positive=True)
    c = _number(thermal_capacity_kwh_per_c, "thermal_capacity_kwh_per_c",
    ↪ positive=True)
    dt = _number(dt_hours, "dt_hours", positive=True)
    # H乘步长除以C为无量纲指数，避免将秒与小时混算。
    decay = h * (dt / c)
```

```
    # expm1减小小指数下的数值相消；b乘净输入功率得到温升。
    a, b = exp(-decay), -expm1(-decay) / h
    # 系数低于矩阵数值分辨能力时明确报错，不返回无效热状态。
    if not isfinite(b) or b <= 1e-12:
        raise ValueError("RC coefficient b is outside LinearMILP numerical
        ↪  resolution; rescale units")
    return a, b
```

结果核对。核对单位时，H为kW/K，C为kWh/K，Δt为h，故b为K/kW。逐楼状态如何接入模型及舒适边界，见后面的RC装配代码。

3．储热容量、运行与状态边界

完整展示储热投资与逐时运行的这一连续代码块。既有与新增能量容量共同限制SOC和充放功率。

输入：储热位置、容量（kWh）、功率能量比（1/h）、效率、自损失及边界模式。

输出：容量、充热、放热、SOC变量和相应线性约束；严格互斥模式另含二元变量。

实现思路。代码先建立新增储热容量，再用同一容量同时约束SOC与充放功率，避免仅满足能量容量却忽略设备功率。严格模式下，充、放热分别受一个二元状态控制；线性化常数取最大允许容量对应的功率。状态递推采用物理步长和充放效率，代表时段权重只参与相应统计，不在此重复放大能量转移。

来源：`model.py`，原第1391–1471行；81行原代码、19行补充注释。

```
# 以下为build方法内部片段，依赖既有模型、时序与变量登记表。
# Storage investments and operation. Short-term storage can be cyclic;
# seasonal storage can use fixed initial/final boundary conditions.
for st_name, storage in self.storages.items():
    # existing和max_new均为能量容量kWh，不是充放功率kW。
    existing = float(storage.get("existing_capacity_kwh", 0.0))
    max_new = float(storage.get("max_new_capacity_kwh", 0.0))
    cap_idx = None
    # 只有允许新增容量时才建立投资变量；既有容量作为常量。
    if max_new > 0:
        cap_idx = p.add_var(f"storage_capacity[{st_name}]", ub=max_new)
        self.variables["storage_capacity"][st_name] = cap_idx
        self._record_investment_cost(
            cap_idx, "storage_capacity", st_name,
            ↪  float(storage.get("capex_yuan_per_kwh", 0.0))
```

```
        )

    # 功率能量比乘装机能量容量得到逐时充放功率上限。
    power_ratio = float(storage.get("power_to_energy_ratio_per_hour", 0.25))
    max_power = power_ratio * (existing + max_new)
    # 严格模式按配置启用；关闭时不能宣称二元互斥已实施。
    strict_storage_mode = bool(storage.get("strict_charge_discharge_mode",
    ↪  True))
    throughput_cost = float(
        storage.get("throughput_cost_yuan_per_kwh", 0.0)
    )
    # 每一真实时段建立充热、放热和SOC；状态连续性在后文连接。
    for t in self.times:
        ch = p.add_var(f"storage_charge[{st_name},{t}]")
        dis = p.add_var(f"storage_discharge[{st_name},{t}]")
        soc = p.add_var(f"storage_soc[{st_name},{t}]")
        self.variables["storage_charge"][(st_name, t)] = ch
        self.variables["storage_discharge"][(st_name, t)] = dis
        self.variables["storage_soc"][(st_name, t)] = soc
        # 可选吞吐费用作用于充放热量，不改变能量守恒方程。
        if throughput_cost > 0.0:
            self._record_time_component(
                "operating_cost", ch, throughput_cost, t
            )
            self._record_time_component(
                "operating_cost", dis, throughput_cost, t
            )

        # 新增容量以负系数进入上界，形成SOC与装机容量的线性关系。
        # 该行左端为SOC减新增容量，右端保留既有容量。
        soc_row = {soc: 1.0}
        ch_row = {ch: 1.0}
        dis_row = {dis: 1.0}
        if cap_idx is not None:
            soc_row[cap_idx] = -1.0
            ch_row[cap_idx] = -power_ratio
            dis_row[cap_idx] = -power_ratio
        p.add_constraint(soc_row, ub=existing)
        p.add_constraint(ch_row, ub=power_ratio * existing)
        p.add_constraint(dis_row, ub=power_ratio * existing)
        # M取候选最大功率常数；不使用投资变量乘二元变量。
        if strict_storage_mode:
            mode = p.add_var(f"storage_mode[{st_name},{t}]", ub=1.0,
            ↪  integer=True)
            self.variables["storage_mode"][(st_name, t)] = mode
            p.add_constraint({ch: 1.0, mode: -max_power}, ub=0.0)
            p.add_constraint({dis: 1.0, mode: max_power}, ub=max_power)

    # cyclic与fixed是两种边界身份，不能一律要求初末SOC相等。
    boundary_mode = str(storage.get("boundary_mode", "cyclic"))
    # 自损失按小时给定，再按实际步长取幂换算。
    loss_factor = (1.0 - float(storage.get("self_loss_per_hour", 0.0))) **
    ↪  self.dt
    eta_ch = float(storage.get("charge_efficiency", 0.95))
```

```
    eta_dis = float(storage.get("discharge_efficiency", 0.95))
    if boundary_mode not in {"cyclic", "fixed"}:
        raise ValueError(f"储热 {st_name} 的 boundary_mode 仅支持 cyclic 或 fixed")
    # 充放功率乘步长后才是能量；放热需除以放热效率。
    for t in self.times:
        row = {
            self.variables["storage_soc"][(st_name, t)]: 1.0,
            self.variables["storage_charge"][(st_name, t)]: -eta_ch * self.dt,
            self.variables["storage_discharge"][(st_name, t)]: self.dt / eta_dis,
        }
        # 状态仅在所属边界组内衔接，禁止跨无关场景串联。
        group_times = self.boundary_groups[self.time_boundary_group[t]]
        is_group_start = t == group_times[0]
        # 循环模式使用组内前驱；非首时段同样递推。
        if boundary_mode == "cyclic" or not is_group_start:
            prev = self.time_predecessor[t]
            self._add(row, self.variables["storage_soc"][(st_name, prev)], -loss_factor)
            p.add_constraint(row, lb=0.0, ub=0.0)
        else:
            # 固定边界以装机能量容量的声明比例给定初态。
            initial_fraction = float(storage.get("initial_soc_fraction", 0.5))
            if cap_idx is not None:
                row[cap_idx] = -loss_factor * initial_fraction
            # 首小时先计初始储热的自损，再计本小时充放。
            initial_energy = loss_factor * initial_fraction * existing
            p.add_constraint(row, lb=initial_energy, ub=initial_energy)
    # 固定模式期末是最低SOC比例约束，不强制等于初态。
    if boundary_mode == "fixed":
        final_fraction = float(storage.get("final_soc_min_fraction", 0.0))
        for group_times in self.boundary_groups.values():
            final_row = {self.variables["storage_soc"][(st_name, group_times[-1])]: 1.0}
            if cap_idx is not None:
                final_row[cap_idx] = -final_fraction
            # 期末库存下界与总装机容量保持同一比例。
            p.add_constraint(final_row, lb=final_fraction * existing)
```

结果核对。复核顺序为容量上界、充放互斥、逐时SOC递推、初末边界。循环边界在组内首尾相接，固定边界按初态比例起算并检查期末最低SOC。

4．节点温位逐时热平衡

将热源、双向管流、储热、需求响应和交付负荷接入同一节点温位等式；不以总量平衡代替局部平衡。

输入：已注册的出力与输配变量（kW）、管段效率、需求列及建筑模型覆盖范围。

输出：每个节点、温位、时段的热量等式。其他变量注册及指标统计见完整文件。

实现思路。节点平衡把每个变量在该节点的净作用写成稀疏系数行：本地产热及入流为正，出流与充热为负，到达端热流乘管段效率。储热、需求响应、计量和末端温位改造都在同一等式中登记。对启用逐楼RC的节点，需求由决策中的实际交付热量承接，避免再次加上同一建筑的固定负荷。

来源：`model.py`，原第1603–1697行；95行原代码、14行补充注释。

```python
# 每一行对应节点n、温位g、时段t，热流系数均采用功率口径。
# Node-grade-time heat balances.
for node_name, node in self.nodes.items():
    for grade_name, grade in self.grades.items():
        demand_column = node.get("demand_columns", {}).get(grade_name)
        for t in self.times:
            row: dict[int, float] = {}
            # 仅汇入安装在本节点且允许服务该温位的热源出力。
            for s_name, source in self.sources.items():
                idx = self.variables["source_heat"].get((s_name, grade_name,
                ↪  t))
                if idx is not None and source["node"] == node_name:
                    self._add(row, idx, 1.0)
            # 正反方向分开登记；管容量与方向互斥约束在此前装配。
            for e_name, pipe in self.pipes.items():
                forward = self.variables["pipe_flow_forward"][(e_name,
                ↪  grade_name, t)]
                # 离开本节点的热流以负号计入，尚未扣除下游管损。
                if pipe["from"] == node_name:
                    self._add(row, forward, -1.0)
                # 到达节点的热流乘效率；接收端承担输配损失。
                if pipe["to"] == node_name:
                    self._add(row, forward, self._pipe_efficiency(pipe, t))
                    activated =
                    ↪  self.variables["pipe_retrofit_flow_forward"].get(
                        (e_name, grade_name, t)
                    )
                    if activated is not None:
                        gain = self._pipe_retrofit_efficiency(pipe, t) -
                        ↪  self._pipe_efficiency(pipe, t)
```

```
                    self._add(row, activated, max(0.0, gain))
            # 允许反向输送时使用独立变量，符号按端点反转。
            reverse = self.variables["pipe_flow_reverse"].get((e_name,
            ↪  grade_name, t))
            if reverse is not None:
                if pipe["to"] == node_name:
                    self._add(row, reverse, -1.0)
                if pipe["from"] == node_name:
                    self._add(row, reverse, self._pipe_efficiency(pipe,
                    ↪  t))
                    activated_reverse = self.variables["
                    ↪  pipe_retrofit_flow_reverse"].get(
                        (e_name, grade_name, t)
                    )
                    if activated_reverse is not None:
                        gain = self._pipe_retrofit_efficiency(pipe, t) -
                        ↪  self._pipe_efficiency(pipe, t)
                        self._add(row, activated_reverse, max(0.0,
                        ↪  gain))
        # 放热增加节点供给，充热占用节点供给。
        for st_name, storage in self.storages.items():
            if storage["node"] == node_name and storage["grade"] ==
            ↪  grade_name:
                self._add(row,
                ↪  self.variables["storage_discharge"][(st_name, t)],
                ↪  1.0)
                self._add(row,
                ↪  self.variables["storage_charge"][(st_name, t)],
                ↪  -1.0)
        # 只有配置允许并注册失供变量时才进入等式；严格主模型可无此项。
        unmet = self.variables["unserved"].get((node_name, grade_name,
        ↪  t))
        if unmet is not None:
            self._add(row, unmet, 1.0)
        # 上调增加需求、下调减少需求，分别按相反符号登记。
        up = self.variables["demand_up"].get((node_name, grade_name, t))
        down = self.variables["demand_down"].get((node_name, grade_name,
        ↪  t))
        if up is not None:
            self._add(row, up, -1.0)
        if down is not None:
            self._add(row, down, 1.0)
        # RC覆盖节点以实际交付量闭合，不能再叠加同一建筑固定负荷。
        if self.building_thermal is not None and (node_name,grade_name)
        ↪  in self.building_thermal.covered_node_grades:
            delivered =
            ↪  self.building_thermal.node_heat[node_name,grade_name,t]
            self._add(row,delivered,-1.0)
            p.add_constraint(row,lb=0.0,ub=0.0)
            self._record_time_metric("demand_kwh",delivered,1.0,t)
            self._record_time_metric("building_heat_delivered_kwh",
            ↪  delivered,1.0,t)
            self._record_time_metric("demand_exergy_kwh",delivered,self
            ↪  ._exergy_factor(float(grade["temperature_c"])),t)
```

```
                reference = self._demand_value(node,demand_column,t) if demand_column else 0.0
                self.metric_constants["building_reference_heat_kwh"] += reference*self._op_weight(t)
                self.scenario_metric_constants[self._scenario(t)]["building_reference_heat_kwh"] += reference*self._physical_weight(t)
                self._record_metering_reference(node_name, grade, reference, t)
                continue
            # 未被RC覆盖的温位沿用登记需求列及其调节接口。
            demand = self._demand_value(node, demand_column, t) if demand_column else 0.0
            metering = node.get("heat_metering", {})
            existing_metering = float(
                metering.get("existing_coverage_fraction", 0.0)
            )
            saving_fraction = float(
                metering.get("saving_fraction_at_full_coverage", 0.0)
            )
            existing_metering_saving = (
                demand * existing_metering * saving_fraction
            )
            metering_idx = self.variables["heat_metering_coverage"].get(
                node_name
            )
            # 计量覆盖投资按登记节约系数调整需求，须与原需求保持同一单位。
            if metering_idx is not None and demand > 0.0 and saving_fraction > 0.0:
                self._add(row, metering_idx, demand * saving_fraction)
            # 末端改造在两个温位间转移符合条件的需求，不凭空产生热量。
            retrofit = node.get("temperature_retrofit")
            retrofit_idx = self.variables["terminal_temperature_retrofit"].get(node_name)
            if retrofit and retrofit_idx is not None:
                source_grade = str(retrofit["source_grade"])
                target_grade = str(retrofit["target_grade"])
                source_column = node["demand_columns"][source_grade]
                shift = (
                    float(retrofit.get("eligible_fraction", 1.0))
                    * self._demand_value(node, source_column, t)
                )
                if grade_name == source_grade:
                    self._add(row, retrofit_idx, shift)
                elif grade_name == target_grade:
                    self._add(row, retrofit_idx, -shift)
            # 固定需求分支以双侧相等界闭合；此前调节项已在系数行内。
            effective_constant_demand = demand - existing_metering_saving
            p.add_constraint(
                row,
                lb=effective_constant_demand,
                ub=effective_constant_demand,
            )
```

结果核对。检查时按节点、温位、小时逐行重算，重点核对管段首尾方向、入流效率及RC覆盖集合。全系统供需总量相等不能替代这些局部等式。

5．SciPy稀疏MILP求解接口

完整求解方法，将已装配的稀疏约束、边界及整数类型传给SciPy的milp接口。

输入：目标系数、约束矩阵、变量上下界、整数类型、间隙和时间预算。

输出：求解器原始结果。是否存在可行解及独立审计由上层模型处理。

实现思路。上层模型以变量索引装配目标系数、约束行和边界，本接口把它们整理为求解器接受的稀疏矩阵。整数类型与连续变量边界分别传入，因此容量、逐时运行和启停状态可以在同一个MILP中处理。矩阵缓存减少相同结构的重复整理，但费用或碳排目标切换时仍保存本次实际目标，供返回后的复核使用。

来源：`linear.py`，原第177–205行；29行原代码、6行补充注释。

```
# 本方法不构建热物理方程，只求解上层已装配的MILP。
def solve(self, mip_gap: float = 0.005, time_limit: float | None = 120.0):
    # 矩阵和边界可缓存；约束修改后由模型维护缓存失效。
    matrix = self.constraint_matrix()
    if self._compiled_constraints is None:
        self._compiled_constraints = LinearConstraint(
            matrix,
            np.asarray(self.row_lower),
            np.asarray(self.row_upper),
        )
    if self._compiled_bounds is None:
        self._compiled_bounds = Bounds(
            np.asarray(self.lower), np.asarray(self.upper)
        )
    if self._compiled_integrality is None:
        self._compiled_integrality = np.asarray(self.integrality)
    # 相对间隙控制停止条件，达到时间上限也可能仅返回可行候选。
    options: dict[str, float | bool] = {"mip_rel_gap": mip_gap, "disp":
    ↪  False}
    if time_limit is not None:
        options["time_limit"] = float(time_limit)
    # 记录本次实际目标，便于上层改回目标后仍能独立复算。
    self._last_solve_objective = np.asarray(self.objective,
    ↪  dtype=float).copy()
    self._last_solve_fun = None
    # 整数类型、边界与矩阵同时传入；不能只检查返回目标值。
    result = milp(
```

```
        c=self._last_solve_objective,
        integrality=self._compiled_integrality,
        bounds=self._compiled_bounds,
        constraints=self._compiled_constraints,
        options=options,
    )
    # 有目标值时保留它；无可行解的异常报告在HeatPlanningModel.solve中。
    if result.fun is not None:
        self._last_solve_fun = float(result.fun)
    return result
```

结果核对。原始求解结果还需由上层检查可行向量、状态、间隙及数值审计；达到时间上限与满足全部物理约束是两个不同检查项。

6．独立数值与守恒残差复核

完整数值审计方法，从全解重新计算每一约束行、变量边界、整数性及本次目标。

输入：求解向量、绝对容差和相对容差，以及已装配的原始矩阵。

输出：残差、最差约束和变量、整数偏差、目标复算误差及通过标记。

实现思路。审计不直接沿用求解器给出的目标和可行性标签，而是从返回向量重新计算Ax、变量界、整数偏差及目标值。约束行与变量边界残差采用绝对及缩放相对容差，整数偏差单独采用绝对容差，目标另按自身量级复算，避免不同检查项混用阈值。记录最差约束行与变量名，可将异常定位回具体模块。

来源：`linear.py`，原第40–105行；66行原代码、12行补充注释。

```
# 审计独立重算矩阵行；它不证明输入来自现场或未装配的工程约束成立。
def audit_solution(
    self, values: np.ndarray, *, absolute_tolerance: float = 1e-6,
    relative_tolerance: float = 1e-8,
) -> dict:
    """Recompute every row, bound and integer restriction from the full
    ↪ solution.

    Scaled violations divide by atol + rtol * row magnitude; <= 1 passes.
    The objective snapshot belongs to the last solve, including emissions
    objectives that a caller restores after the solver returns.
    """
    # 拒绝无效容差；不可通过任意放大容差掩盖物理超限。
    if not np.isfinite(absolute_tolerance) or not
    ↪  np.isfinite(relative_tolerance) or absolute_tolerance <= 0 or
    ↪  relative_tolerance < 0:
```

```
        raise ValueError("audit tolerances must be positive/nonnegative")
    x = np.asarray(values, dtype=float)
    report = {
        "numerical_audit_passed": False,
        "numerical_audit_tolerance": {
            "absolute": absolute_tolerance, "relative": relative_tolerance,
            "scaled_violation_limit": 1.0,
        },
        # 首先检查向量维数和有限性，缺失或非有限解立即不通过。
        "numerical_solution_finite": bool(x.shape == (len(self.names),) and
        ↪  np.isfinite(x).all()),
    }
    if not report["numerical_solution_finite"]:
        return report
    compile_seconds, cache_hit = self.last_matrix_compile_seconds,
    ↪  self.last_matrix_cache_hit
    matrix = self.constraint_matrix()
    self.last_matrix_compile_seconds, self.last_matrix_cache_hit =
    ↪  compile_seconds, cache_hit
    # activity为Ax；热平衡与储热递推等式也包含在这些行内。
    activity = np.asarray(matrix @ x)
    # 用各项绝对贡献估计量级，避免正负抵消使容差失真。
    magnitude = np.asarray(abs(matrix) @ np.abs(x))
    row_lo, row_hi = np.asarray(self.row_lower), np.asarray(self.row_upper)
    # 同时检查行下界与上界，等式并非仅看某一方向的残差。
    row_violation = np.maximum(0.0, np.maximum(row_lo - activity, activity -
    ↪  row_hi))
    row_scaled = np.zeros_like(activity)
    for bound, violation in ((row_lo, np.maximum(0.0, row_lo - activity)),
    ↪  (row_hi, np.maximum(0.0, activity - row_hi))):
        scale = np.maximum(np.maximum(1.0, magnitude),
        ↪  np.where(np.isfinite(bound), np.abs(bound), 0.0))
        row_scaled = np.maximum(row_scaled, violation / (absolute_tolerance
        ↪  + relative_tolerance * scale))
    # 变量上下界单独复核，防止行约束满足而容量或非负界被违反。
    lower, upper = np.asarray(self.lower), np.asarray(self.upper)
    bound_violation = np.maximum(0.0, np.maximum(lower - x, x - upper))
    bound_scaled = np.zeros_like(x)
    for bound, violation in ((lower, np.maximum(0.0, lower - x)), (upper,
    ↪  np.maximum(0.0, x - upper))):
        scale = np.maximum(np.maximum(1.0, np.abs(x)),
        ↪  np.where(np.isfinite(bound), np.abs(bound), 0.0))
        bound_scaled = np.maximum(bound_scaled, violation /
        ↪  (absolute_tolerance + relative_tolerance * scale))
    # 整数变量与最近整数比较，不将近似浮点数直接当作合法离散决策。
    integer_values = x[np.asarray(self.integrality) != 0]
    integer_violation = float(np.max(np.abs(integer_values -
    ↪  np.rint(integer_values)), initial=0.0))
    # 尚无本次目标快照时，不伪造目标复算误差。
    objective_error = None
    objective_pass = True
    # 按本次实际目标复算，适用于费用与排放目标切换。
    if self._last_solve_objective is not None and self._last_solve_fun is
    ↪  not None:
```

```
        recomputed = float(self._last_solve_objective @ x)
        objective_error = abs(recomputed - self._last_solve_fun)
        # 费用与排放目标均按当前目标各项贡献确定比较量级。
        objective_scale = max(1.0, float(np.abs(self._last_solve_objective *
        ↪  x).sum()), abs(self._last_solve_fun))
        objective_pass = bool(objective_error <= absolute_tolerance +
        ↪  relative_tolerance * objective_scale)
    report.update({
        "numerical_constraint_max_violation": float(np.max(row_violation,
        ↪  initial=0.0)),
        "numerical_constraint_max_scaled_violation":
        ↪  float(np.max(row_scaled, initial=0.0)),
        "numerical_constraint_worst_row": int(np.argmax(row_scaled)) if
        ↪  len(row_scaled) else None,
        "numerical_bound_max_violation": float(np.max(bound_violation,
        ↪  initial=0.0)),
        "numerical_bound_max_scaled_violation": float(np.max(bound_scaled,
        ↪  initial=0.0)),
        "numerical_bound_worst_variable":
        ↪  self.names[int(np.argmax(bound_scaled))] if len(x) else None,
        "numerical_integrality_max_violation": integer_violation,
        "numerical_objective_recompute_error": objective_error,
        "numerical_objective_checked": objective_error is not None,
        # 所有数值门槛共同决定通过，scaled violation不超过1才满足对应容差。
        "numerical_audit_passed": bool(
            np.isfinite(activity).all() and np.max(row_scaled, initial=0.0)
            ↪  <= 1.0
            and np.max(bound_scaled, initial=0.0) <= 1.0
            and integer_violation <= absolute_tolerance and objective_pass
        ),
    })
    return report
```

结果核对。检查numerical_audit_passed时同时保留最大缩放残差及最差行号；缩放残差不大于1表示在登记容差内，不能据此替代数据来源检查。

7. 完整投资冻结与费用账本

完整冻结函数。容量投资转入存量，离散改造保存固定取值，并关闭后续新增或退役自由度。

输入：原配置、规划投资表及规划费用摘要；默认不放宽可靠性。

输出：深复制后的固定配置，包含所选资产年化费用账本及复算政策身份。

实现思路。规划与运行复算通过资产冻结函数衔接。函数复制规划配置，将所选新增容量转为运行期存量，并扣除已退役容量；管径、末端改造等离散选择保留为固定取值，同时关闭再次投资的自由度。冻结覆盖热源、储热、管网、建筑侧改造、配电和燃料库存等登记资产，费用账本随所选资产保留。

来源：experiments.py，原第13–147行；135行原代码、16行补充注释。

```python
# 必须调用完整函数，不能仅冻结正文中示出的热源和储热两类。
def freeze_investment_plan(
    config: dict,
    nominal,
    *,
    relax_reliability: bool = False,
) -> dict:
    """Convert an optimized plan into fixed existing assets for replay.

    Reliability is retained by default. A diagnostic stress test may explicitly
    pass ``relax_reliability=True`` to quantify shortage, but that relaxed run
    is not evidence of compliance with the original service requirement.
    """
    # 深复制保护原规划配置，复算修改不回写原数据。
    fixed = copy.deepcopy(config)
    # 默认保留原服务可靠性；诊断性放宽会被单独标记。
    fixed["replay_service_policy"] = (
        "diagnostic_relaxed_reliability" if relax_reliability else "original_service_requirements"
    )
    # 冻结不抹去所选资产的年化费用；季节范围转换另由季节接口处理。
    # A frozen asset has no *new* investment variable, but its selected annual
    # capital charge still belongs in annual replay and affordability tests.
    planning_summary = getattr(nominal, "summary", {})
    components = planning_summary.get("components_yuan_per_year", {})
    fixed["fixed_investment_accounting"] = {
        "scope": "annual_selected_plan_charges",
        "components": {name: float(components.get(name, 0.0)) for name in ("annualized_capex", "fixed_om_cost")},
        "metrics": {name: float(planning_summary.get(name, 0.0)) for name in (
```

```
            "regulated_annualized_capex_yuan_per_year",
            "shared_non_heating_capex_yuan_per_year",
            "public_subsidy_yuan_per_year")},
    }
    if not hasattr(nominal, "summary"):
        # Investment-only callers can still inspect installed assets. A model
        # replay must refuse these unknown charges instead of assuming zero.
        fixed["fixed_investment_accounting"] = {"scope": "unavailable_no_planning_summary"}
    # 热源容量按既有加新增减退役转为存量，随后锁定两个投资上限。
    for source in fixed["sources"]:
        row = nominal.investments[
            (nominal.investments["category"] == "source_capacity")
            & (nominal.investments["asset"] == source["name"])
        ]
        addition = float(row["value"].iloc[0]) if not row.empty else 0.0
        # 退役量需要扣除，不可把已退役容量继续用于复算。
        retired = nominal.investments[
            (nominal.investments["category"] == "source_retirement")
            & (nominal.investments["asset"] == source["name"])
        ]
        retirement = float(retired["value"].iloc[0]) if not retired.empty else 0.0
        source["existing_capacity_kw"] = float(source.get("existing_capacity_kw", 0.0)) + addition - retirement
        source["max_new_capacity_kw"] = 0.0
        source["max_retire_capacity_kw"] = 0.0

    # 储热新增能量容量并入kWh存量，后续不再增加储热容量。
    for storage in fixed["storages"]:
        row = nominal.investments[
            (nominal.investments["category"] == "storage_capacity")
            & (nominal.investments["asset"] == storage["name"])
        ]
        addition = float(row["value"].iloc[0]) if not row.empty else 0.0
        storage["existing_capacity_kwh"] = float(storage.get("existing_capacity_kwh", 0.0)) + addition
        storage["max_new_capacity_kwh"] = 0.0

    # 管道改造以固定值保存；离散改造不必伪装成连续新增容量。
    for pipe in fixed["pipes"]:
        retrofit_row = nominal.investments[
            (nominal.investments["category"] == "pipe_retrofit")
            & (nominal.investments["asset"] == pipe["name"])
        ]
        if pipe.get("retrofit"):
            pipe["retrofit"]["fixed_value"] = (
                float(retrofit_row["value"].iloc[0]) if not retrofit_row.empty else 0.0
            )
            pipe["retrofit"]["capex_total_yuan"] = 0.0
        # 被选管径容量并入存量，再清空候选管径，防止复算重新选型。
```

```
        selected = nominal.investments[
            (nominal.investments["category"] == "pipe_choice")
            & (nominal.investments["asset"] == pipe["name"])
            & (nominal.investments["value"] > 0.5)
        ]
        if not selected.empty:
            option_name = str(selected["option"].iloc[0])
            option = next(x for x in pipe.get("diameter_options", []) if
            ↪  str(x["name"]) == option_name)
            pipe["existing_capacity_kw"] =
            ↪  float(pipe.get("existing_capacity_kw", 0.0)) +
            ↪  float(option["capacity_kw"])
        pipe["diameter_options"] = []

    # 计量覆盖率并入已覆盖比例，关闭新增覆盖自由度。
    for node in fixed["nodes"]:
        metering = node.get("heat_metering")
        if metering:
            row = nominal.investments[
                (nominal.investments["category"] ==
                ↪  "heat_metering_coverage")
                & (nominal.investments["asset"] == node["name"])
            ]
            addition = float(row["value"].iloc[0]) if not row.empty else 0.0
            metering["existing_coverage_fraction"] = min(
                1.0,
                float(metering.get("existing_coverage_fraction", 0.0))
                + addition,
            )
            metering["max_new_coverage_fraction"] = 0.0
            metering.pop("fixed_new_coverage_fraction", None)
            metering["capex_yuan_per_coverage_fraction"] = 0.0
        # 末端温位改造保存规划取值，避免逐时复算改变投资模式。
        retrofit = node.get("temperature_retrofit")
        if not retrofit:
            continue
        row = nominal.investments[
            (nominal.investments["category"] ==
            ↪  "terminal_temperature_retrofit")
            & (nominal.investments["asset"] == node["name"])
        ]
        retrofit["fixed_value"] = float(row["value"].iloc[0]) if not
        ↪  row.empty else 0.0
        retrofit["capex_total_yuan"] = 0.0

    # 电网接入增容同样冻结，不能靠新增配电容量通过复算。
    grid = fixed.get("grid_connection")
    if grid:
        row = nominal.investments[nominal.investments["category"] ==
        ↪  "grid_connection_capacity"]
        addition = float(row["value"].iloc[0]) if not row.empty else 0.0
        grid["existing_capacity_kw"] =
        ↪  float(grid.get("existing_capacity_kw", 0.0)) + addition
        grid["max_new_capacity_kw"] = 0.0
```

```
    # 生物质库存容量以kg登记；库存容量不同于逐小时到货速率。
    for chain in fixed.get("biomass_supply_chains", []):
        row = nominal.investments[
            (nominal.investments["category"] ==
            ↪  "biomass_inventory_capacity")
            & (nominal.investments["asset"] == chain["name"])
        ]
        addition = float(row["value"].iloc[0]) if not row.empty else 0.0
        chain["existing_inventory_capacity_kg"] = float(
            chain.get("existing_inventory_capacity_kg", 0.0)
        ) + addition
        chain["max_new_inventory_capacity_kg"] = 0.0
    # 可再生电源、储电功率与能量、备用电源均纳入冻结。
    microgrid = fixed.get("microgrid_islanding")
    if microgrid:
        fields = {
            "microgrid_renewable_capacity":
            ↪  ("existing_renewable_capacity_kw",
            ↪  "max_new_renewable_capacity_kw"),
            "microgrid_battery_power": ("existing_battery_power_kw",
            ↪  "max_new_battery_power_kw"),
            "microgrid_battery_energy": ("existing_battery_energy_kwh",
            ↪  "max_new_battery_energy_kwh"),
            "microgrid_backup_capacity": ("existing_backup_capacity_kw",
            ↪  "max_new_backup_capacity_kw"),
        }
        for category, (existing_key, max_key) in fields.items():
            row = nominal.investments[nominal.investments["category"] ==
            ↪  category]
            addition = float(row["value"].iloc[0]) if not row.empty else 0.0
            microgrid[existing_key] = float(microgrid.get(existing_key,
            ↪  0.0)) + addition
            microgrid[max_key] = 0.0
    # 仅显式诊断请求可放宽失供率和备用裕度；该分支不构成原要求验收。
    if relax_reliability:
        fixed["planning"]["max_unserved_rate"] = 1.0
        fixed["planning"]["max_scenario_unserved_rate"] = 1.0
        fixed["planning"]["firm_capacity_reserve_margin"] = 0.0
    # 返回完整固定配置；是否冻结成功另由投资冻结审计核对。
    return fixed
```

结果核对。复算前同时核对资产清单与新增上限，不能只锁定热源。资本费用是否计入及是否重复计入，应结合保存的费用账本和服务期口径检查。

8. 固定投资的连续供暖季复算

完整服务期复算函数，求规划方案或接收既有固定方案，保留小时顺序，并汇总季节范围检查。

输入：规划输入、连续时序、明确服务窗口、求解预算和可选已登记固定方案。

输出：季节运行解、储热检查、服务期证书及期间汇总；不生成年度认证。

实现思路。该函数先确定有时区的服务窗口，再准备规划方案或复用已冻结方案。完整时序作为运行输入，建筑与储热状态按其边界组传递。随后把资产冻结、供能能力、数值求解和储热校核合并为服务期证书，并额外加入窗口精确匹配检查，使“求解成功”和“覆盖声明供暖季”分别有可查记录。

来源：heating_season.py，原第183–246行；64行原代码、15行补充注释。

```
# 本接口检验声明供暖季，不等同于在全年窗口联合重求投资。
def run_fixed_investment_heating_season_replay(
    config: dict[str, Any], representative_timeseries: pd.DataFrame,
    chronological_source: pd.DataFrame, scope: dict[str, Any], *,
    timestamp_column: str = "timestamp", input_timezone: str | None = None,
    planning_time_limit: float = 180.0, replay_time_limit: float = 1800.0,
    planning_mip_gap: float | None = None, replay_mip_gap: float | None =
    ↪ None,
    allow_cyclic_state_assumptions: bool = False,
    fixed_plan: FixedInvestmentPlanArtifacts | None = None,
) -> HeatingSeasonReplayArtifacts:
    """Plan on the supplied planning horizon and validate only the stated
    ↪ season."""
    # 按有时区的左闭右开窗口切取时序，检查步长和完整覆盖。
    chronology, scope_audit = prepare_heating_season_timeseries(
        config, chronological_source, scope,
        ↪ timestamp_column=timestamp_column,
        ↪ input_timezone=input_timezone,
    )
    # 不支持的状态交接假设在昂贵求解前拒绝。
    # Reject unsupported state assumptions before a potentially expensive
    ↪ solve.
    _season_config(config, allow_cyclic_state_assumptions)
    # 无既有固定方案时先规划并完整冻结；有方案时复用该方案。
    plan = fixed_plan if fixed_plan is not None else
    ↪ prepare_fixed_investment_plan(
        config,representative_timeseries,planning_time_limit=
        ↪ planning_time_limit,planning_mip_gap=planning_mip_gap)
    # 季节转换保存假设；年度经济与资源约束另需年度复算。
    fixed_config, assumptions = _season_config(plan.fixed_config,
    ↪ allow_cyclic_state_assumptions)
    plan = replace(plan, fixed_config=fixed_config)
```

```
    _, scenario_audit = adapt_config_to_chronology_scenarios(fixed_config, chronology)
    preflight = AnnualReplayPreflight(
        chronological_timeseries=chronology, chronology_audit=scope_audit,
        representative_coverage_audit=pd.DataFrame(columns=["coverage_status"]),
        representative_recommendations=pd.DataFrame(), scenario_scope_audit=scenario_audit,
    )
    # 运行模型使用已冻结配置，require_full_year=False仅标明季节范围。
    result = replay_prepared_fixed_investment_plan(
        config, representative_timeseries, chronology, plan,
        timestamp_column=timestamp_column, replay_time_limit=replay_time_limit,
        replay_mip_gap=replay_mip_gap, require_full_year=False, preflight=preflight,
    )
    # 不把年度规划总量与季节总量直接比较，也不按小时比例外推全年费用。
    # Annual planning totals and seasonal totals are intentionally not compared.
    # 清空跨年度与季节的直接总量比较，避免口径错配。
    result.kpi_comparison = pd.DataFrame()
    # 独立复核储热状态、初末边界和对应时序。
    storage = seasonal_storage_postcheck(result.fixed_config, result.replay.dispatch, expected_timeseries=chronology)
    # 共同验收检查结合冻结、可靠容量、求解与储热证据。
    common = annual_replay_acceptance_audit(
        result.fixed_config, chronology_audit=scope_audit,
        representative_coverage_audit=preflight.representative_coverage_audit,
        investment_freeze_audit=result.investment_freeze_audit,
        fixed_investment_readiness_audit=result.fixed_investment_readiness_audit,
        planning_summary=result.planning.summary, replay_summary=result.replay.summary,
        storage_boundary_audit=storage,
    )
    # 删除年度专属门槛；季节证书不能因这些门槛被删除而晋级年度证书。
    gates = [gate for gate in common["gates"] if gate["gate"] not in {"continuous_full_calendar_year", "continuous_full_service_year", "representative_period_coverage"}]
    for gate in gates:
        gate["gate"] = gate["gate"].replace("annual_", "period_")
        gate["requirement"] = gate["requirement"].replace("全年", "本服务期")
    # 另加服务窗口精确匹配门槛，防止短片段冒充完整供暖季。
    gates.insert(0, {"gate": "exact_service_period", "passed": bool(scope_audit["exact_scope_match"]),
                     "observed": scope_audit, "requirement": "exact timezone-aware half-open service window"})
    # 保留全部失败门槛及假设，不把失败结果写成通过。
    # 证书由全部门槛共同决定，保留未通过项目名称。
    failed = [gate["gate"] for gate in gates if not gate["passed"]]
```

```
    certificate = {"schema": "heatplan.heating_season_certificate.v1", "scope_type": "heating_season",
                   "certification_pass": not failed, "gate_count": len(gates),
                   "passed_gate_count": len(gates) - len(failed), "failed_gate_count": len(failed),
                   "failed_gates": failed, "gates": gates, "assumptions": assumptions,
                   "annual_replay_certified": False}
    summary = {"scope": scope_audit, "heating_season_certified": not failed,
               "annual_replay_certified": False, "assumptions": assumptions,
               "solver_status": result.replay.summary["solver_status"],
               "unserved_rate": result.replay.summary["unserved_rate"],
               "numerical_audit_passed": result.replay.summary.get("numerical_audit_passed", False)}
    # 期间指标重新命名为over_period，避免per_year标签误导。
    for key in ("demand_kwh_per_year", "effective_heat_demand_kwh_per_year", "unserved_kwh_per_year",
                "emissions_kg_per_year", "total_service_emissions_kg_per_year", "electricity_kwh_per_year",
                "heating_cash_operating_cost_yuan_per_year", "cooling_demand_kwh_per_year"):
        if key in result.replay.summary:
            summary[key.replace("_per_year", "_over_period")] = result.replay.summary[key]
    return HeatingSeasonReplayArtifacts(result, scope_audit, assumptions, storage, certificate, summary)
```

结果核对。优先核对exact_service_period、failed_gates和heating_season_certified，再读费用与能量。期间指标使用over_period，按对应窗口解释。

9．带主目标容差的两阶段端点

完整端点方法，先求主目标，再施加单侧主目标上界，求解次目标并保留两阶段证据。

输入：主目标类型、绝对和相对容差，以及各阶段求解预算。

输出：次目标阶段的解与两阶段状态。限时可行值不等于全局词典序最优。

实现思路。两阶段端点与“先定投资、再算运行”用途不同。此处先求主目标，再把首阶段所得可行值加绝对、相对容差写成单侧上界，在该约束下优化次目标。排放表达含常量时，加入矩阵行的右端必须扣除常量；否则约束的实际范围会偏移。两阶段状态与预算分别保存，便于判断端点证据强度。

来源：model.py，原第2829–2888行；60行原代码、11行补充注释。

```python
# 与固定投资两阶段不同：此处两阶段切换目标，不自动冻结资产。
def solve_lexicographic(
    self, primary: str = "emissions", *, absolute_tolerance: float = 1e-7,
    relative_tolerance: float = 1e-8, mip_gap: float | None = None,
    time_limit: float | None = None,
) -> SolveArtifacts:
    """Two-stage endpoint with an explicit primary-objective allowance.

    Each stage uses the configured solve budget. A time-limited first
    stage only certifies a secondary solution within its incumbent bound,
    not a globally proven lexicographic optimum. The returned summary
    retains both stages' termination evidence. This instance keeps the
    primary bound, so use a fresh model for each independent endpoint.
    """
    # 只允许费用或排放主目标，容差必须有限且非负。
    if primary not in {"total_cost", "emissions"}:
        raise ValueError("primary must be total_cost or emissions")
    if not np.isfinite(absolute_tolerance) or not
    ↪  np.isfinite(relative_tolerance) or absolute_tolerance < 0 or
    ↪  relative_tolerance < 0:
        raise ValueError("lexicographic tolerances must be finite and
        ↪  nonnegative")
    # 第一阶段先取得实际可行候选值，保留其状态、间隙与下界。
    first = self.solve(mip_gap=mip_gap, time_limit=time_limit,
    ↪  objective_mode=primary)
    # 排放表达包含登记常量；添加上界时需扣除常量。
    if primary == "emissions":
        coefficients, constant = self.total_service_emissions_expression()
        value = float(first.summary["total_service_emissions_kg_per_year"])
        secondary = "total_cost"
    else:
        # 费用主目标使用当前原目标系数，不以事后费用统计替代。
        coefficients = {i:c for i,c in enumerate(self.problem.objective) if
        ↪  c}
```

```
        constant = 0.0
        value = self._dot(coefficients, first.values)
        secondary = "emissions"
    # 上界为首阶段值加绝对及相对容差；允许主目标进一步改善，不是双侧邻域。
    allowance = absolute_tolerance + relative_tolerance * abs(value)
    self.problem.add_constraint(coefficients, ub=value+allowance-constant)
    failed_second = None
    # 第二阶段用同样的预算求次目标；不据此宣称全局词典序最优。
    try:
        second = self.solve(mip_gap=mip_gap, time_limit=time_limit,
        ↪  objective_mode=secondary)
        endpoint_summary = second.summary
    # 次阶段无可行解时保留失败摘要，后文继续补足首阶段证据。
    except SolveFailure as error:
        failed_second = error
        endpoint_summary = error.summary
    # 同时报告首阶段可行值、容差和终止状态，防止只展示次阶段结果。
    endpoint_summary.update({
        "endpoint_method":
        ↪  "two_stage_lexicographic_with_reported_tolerance",
        "primary_objective": primary, "primary_incumbent_value": value,
        "primary_objective_allowance": allowance,
        "primary_solver_status": first.summary["solver_status"],
        "primary_mip_gap": first.summary.get("mip_gap"),
        "primary_numerical_audit_passed":
        ↪  first.summary["numerical_audit_passed"],
        "primary_solver_message": first.summary["solver_message"],
        "primary_solver_objective_value": first.summary["objective_value"],
        "primary_mip_dual_bound": first.summary.get("mip_dual_bound"),
        "primary_solver_requested_mip_gap":
        ↪  first.summary["solver_requested_mip_gap"],
        "primary_solver_time_limit_seconds":
        ↪  first.summary["solver_time_limit_seconds"],
        "primary_solver_elapsed_seconds":
        ↪  first.summary["solver_elapsed_seconds"],
        "endpoint_solver_elapsed_seconds":
        ↪  first.summary["solver_elapsed_seconds"]+endpoint_summary["」
        ↪  solver_elapsed_seconds"],
    })
    # 两阶段证据分别标识primary和secondary，可追踪求解身份。
    stages = [deepcopy(first.summary["solver_stages"][0]),
    ↪  deepcopy(endpoint_summary["solver_stages"][0])]
    stages[0]["stage_id"], stages[1]["stage_id"] = "primary", "secondary"
    endpoint_summary["solver_stages"] = stages
    endpoint_summary["solution_quality"] = solution_quality_from_summary(
        endpoint_summary, scope=first.summary["solution_quality"]["scope"])
    # 次阶段失败在补足证据后重新抛出，不静默返回首阶段当作成功端点。
    if failed_second is not None:
        failed_second.solution_quality =
        ↪  endpoint_summary["solution_quality"]
        raise failed_second
    return second
```

结果核对。联合查看主目标可行值、允许容差、下界和两次求解状态。次阶段失败要按失败处理；首阶段未取得全局证明时，不能只以次阶段目标值命名全局端点。

10．独立碳帽采样与非支配筛选

完整独立扫描接口。清除原碳帽、碳价置零，再按碳上限逐档重新构模和检查。

输入：同一配置和时间序列、至少两档采样；投资自由度由输入配置决定。

输出：样本内非支配集合、全部采样证据、边际减排费用及归一化折中建议。

实现思路。扫描采用同一基础配置和时序，每个碳帽重新建立模型，防止上一档约束残留。先清除碳价与原碳帽，再由两个端点确定采样范围；每个候选必须同时通过数值检查和实际碳帽检查。非支配筛选在已取得的可行点中进行，完整采样记录保留在结果属性中，方便核对被去重或支配的点。

来源：`experiments.py`，原第351–411行；61行原代码、16行补充注释。

```python
# 独立接口不自动运行于三城单次入口，也不代表历史扫描使用相同规则。
def epsilon_constraint_pareto(
    config_path: str | Path,
    timeseries_path: str | Path,
    points: int = 7,
) -> pd.DataFrame:
    """Sample a nondominated system cost/carbon frontier with explicit caps.

    The investment and operating model is unchanged across points. A hard reliability
    constraint prevents the emissions objective from creating artificial low-carbon
    solutions through heat curtailment.
    """
    if points < 2:
        raise ValueError("points 至少为 2")
    # 配置与时序只读取一次；各档复制同一基础配置，保持输入一致。
    base_config, frame = load_inputs(config_path, timeseries_path)
    base_config = copy.deepcopy(base_config)
    # 清除碳价与原碳帽；其余启用约束和偏好仍按配置保留。
    base_config["planning"]["carbon_price_yuan_per_kg"] = 0.0
    base_config["planning"].pop("emissions_cap_kg_per_year", None)

    # 两个端点分别新建模型，避免前一端点的主目标约束污染另一端点。
    cost_optimum = HeatPlanningModel(copy.deepcopy(base_config), frame).solve_lexicographic(primary="total_cost").summary
    emission_optimum = HeatPlanningModel(copy.deepcopy(base_config), frame).solve_lexicographic(primary="emissions").summary
```

```
    # 采用冷热服务及共享电力口径的登记系统总碳，单位kg CO2。
    emission_column = "total_service_emissions_kg_per_year"
    min_emissions = float(emission_optimum[emission_column])
    cost_optimum_emissions = float(cost_optimum[emission_column])
    # 端点次序超出容差则报错，不伪造可用扫描区间。
    if min_emissions > cost_optimum_emissions + 1e-6 +
    ↪  1e-8*abs(cost_optimum_emissions):
        raise RuntimeError("Carbon endpoint exceeds cost endpoint; improve
        ↪  solve budgets before defining an epsilon range")
    caps = np.linspace(min_emissions, cost_optimum_emissions, points)

    # 逐档保存全部可行候选，再统一进行非支配筛选。
    rows = []
    # 每档均由基础配置开始；本函数没有调用投资冻结。
    for cap in caps:
        config = copy.deepcopy(base_config)
        # A tiny numerical allowance avoids rejecting the emissions-optimal
        ↪  endpoint.
        effective_cap = float(cap) + 1e-7 + abs(float(cap))*1e-8
        # 实际碳帽包含微小数值余量，同时保存请求值供审查。
        config["planning"]["emissions_cap_kg_per_year"] = effective_cap
        # 自由投资输入时每档重求投资和运行；已固定配置才只检验固定资产。
        summary = HeatPlanningModel(config,
        ↪  frame).solve(objective_mode="total_cost").summary
        # 数值或碳帽复核失败立即终止，不能把失败点加入前沿。
        if not summary["numerical_audit_passed"] or not
        ↪  summary["emissions_cap_pass"]:
            raise RuntimeError("A sampled frontier incumbent failed
            ↪  independent numerical or carbon-cap checks")
        rows.append({"requested_emissions_cap_kg_per_year": float(cap),
        ↪  **summary})
    sampled = pd.json_normalize(rows)
    # 非支配关系只针对已采样可行解，不是完整连续Pareto前沿。
    result = nondominated_cost_emission_points(sampled)
    # 保留筛选前的完整分母，便于追查重复及被支配样本。
    result.attrs["all_sampled_points"] = sampled.to_dict(orient="records")
    result.attrs["cost_endpoint"] = cost_optimum
    result.attrs["carbon_endpoint"] = emission_optimum
    result["frontier_scope"] =
    ↪  "nondominated_subset_of_sampled_feasible_solutions"
    result["sampled_point_count"] = len(rows)
    result["removed_duplicate_or_dominated_count"] = len(rows)-len(result)
    result["marginal_abatement_cost_yuan_per_kg"] = np.nan
    # 相邻点费用差除以减排量，得到元/kg的边际减排费用。
    for i in range(len(result) - 1):
        delta_cost = result.loc[i, "objective_yuan_per_year"] - result.loc[i
        ↪  + 1, "objective_yuan_per_year"]
        delta_emission = result.loc[i + 1, emission_column] - result.loc[i,
        ↪  emission_column]
        # 减排差须超过数值容差，避免用近零分母放大比值。
        if delta_emission > 1e-6 + 1e-8*abs(result.loc[i + 1,
        ↪  emission_column]):
            result.loc[i, "marginal_abatement_cost_yuan_per_kg"] =
            ↪  delta_cost / delta_emission
```

```
    # 理想点距离依赖归一化规则，折中点不是额外物理或工程验收。
    cost_span = result["objective_yuan_per_year"].max() -
    ↪  result["objective_yuan_per_year"].min()
    emission_span = result[emission_column].max() -
    ↪  result[emission_column].min()
    normalized_cost = (result["objective_yuan_per_year"] -
    ↪  result["objective_yuan_per_year"].min()) / max(cost_span, 1e-9)
    normalized_emission = (result[emission_column] -
    ↪  result[emission_column].min()) / max(emission_span, 1e-9)
    result["distance_to_ideal"] = np.sqrt(normalized_cost**2 +
    ↪  normalized_emission**2)
    result["recommended_compromise"] = False
    result.loc[result["distance_to_ideal"].idxmin(),
    ↪  "recommended_compromise"] = True
    result["recommendation_method"] = "closest_normalized_ideal_point"
    return result
```

结果核对。相邻费用差除以减排量得到边际减排费用；推荐折中点由归一化理想点距离选取。投资是否重新优化仍取决于传入配置是否已冻结。

11．代表日硬能量约束可行性诊断

完整硬约束诊断函数，同时要求能量等式、代表天数守恒和极端日下界；与软权重候选生成分别登记。

输入：代表日能量矩阵M、目标b、原权重和权重下界（天）。

输出：经过独立复算的可行权重见证，或明确的不可行／求解失败状态。

实现思路。这段代码检查能否同时满足多列能量等式、代表天数守恒和各代表日下界。通过$w = \ell + z$把下界移入右端，求解非负变量z的零目标线性规划；能量行按目标量级缩放，以减小数值量级差异。取得候选后恢复原始权重，并在原单位下重新核对能量、总天数和下界。

来源：`representative_adoption.py`，原第48–94行；47行原代码、12行补充注释。

```
# 诊断Mw=b是否可行，不是scenarios.py中的软能量拟合SLSQP。
def diagnose_exact_weight_feasibility(
    design, targets, original_weights, lower_bounds, *, total_weight=None,
    numerical_tolerance: float = 1e-8,
) -> dict[str, Any]:
    """Diagnose M w = b, sum(w) = W, w >= lower using a standard LP.

```

```
    Feasible weights are a witness only. An infeasible result is explicit;
    numerical/solver failures are never reported as a proof of infeasibility.
    Rows are scaled to protect the LP from energy-unit magnitude differences.
    The witness is independently recomputed in original units afterwards.
    """
    # 矩阵列对应代表日，目标为声明能量口径；权重单位为天。
    matrix, target, initial, lower, total = _inputs(design, targets, original_weights, lower_bounds, total_weight)
    tolerance = float(numerical_tolerance)
    if not np.isfinite(tolerance) or tolerance <= 0:
        raise ValueError("numerical_tolerance must be positive and finite")
    # 这项诊断不修改来源数据，也不证明工程运行可行。
    report = {"method": "standard_linear_feasibility_HiGHS",
              "equations": "M w = b; sum(w) = W; w >= lower",
              "total_weight_days": total, "lower_bound_sum_days": float(lower.sum()),
              "zero_target_rows": np.flatnonzero(target == 0).tolist(),
              "source_data_modified": False, "engineering_operation_validated": False}
    # 所有下界之和超过总天数即可判不可行，不需优化求解。
    if lower.sum() > total + tolerance:
        return {**report, "status": "infeasible", "reason": "lower_bounds_exceed_total_weight", "witness_weights": None}
    # 先按能量量级缩放，并用w=lower+z转成非负变量。
    energy_scale = np.maximum(np.abs(target), 1.0)
    # Shift w = lower + z, z >= 0, and scale the conservation equation too.
    scale = np.concatenate((energy_scale, [max(total, 1.0)]))
    # 最后一行是总天数守恒，与每列负荷能量等式共同约束。
    equalities = np.vstack((matrix, np.ones(len(initial)))) / scale[:, None]
    rhs = np.concatenate((target - matrix @ lower, [total - lower.sum()])) / scale
    # 零目标线性规划只寻找可行见证，能量与总天数均为等式。
    result = linprog(np.zeros(len(initial)), A_eq=equalities, b_eq=rhs,
                     bounds=[(0.0, None)] * len(initial), method="highs")
    report.update(solver_status=int(result.status), solver_message=str(result.message))
    # 仅明确的LP不可行状态作为不可行结论。
    if result.status == 2:
        return {**report, "status": "infeasible", "reason": "LP_infeasible", "witness_weights": None}
    # 求解或数值失败单独登记，不冒称数学上已经证伪。
    if not result.success or result.x is None:
        return {**report, "status": "solver_failure", "reason": "LP_did_not_return_a_feasible_witness", "witness_weights": None}
    # 恢复原单位权重并重新计算能量、天数与下界残差。
    witness = lower + np.asarray(result.x, dtype=float)
    # 见证恢复后使用原矩阵复算，不只信任缩放后的求解值。
    energy_residual = matrix @ witness - target
    conservation = abs(float(witness.sum() - total))
    bound_violation = float(np.maximum(lower - witness, 0).max())
    normalized_residual = float(np.max(np.abs(energy_residual) / energy_scale))
```

```
    # 见证必须通过独立有限性和残差检查；通过也不自动代表已采用。
    verified = (np.isfinite(witness).all() and normalized_residual <=
    ↪  tolerance
                and conservation <= tolerance * max(total, 1.0) and
                ↪  bound_violation <= tolerance)
    report.update(status="feasible" if verified else "solver_failure",
                  reason="verified_LP_witness" if verified else
                  ↪  "LP_witness_failed_independent_residual_check",
                  witness_weights=witness.tolist(),
                  ↪  energy_residual_original_units=energy_residual.tolist
                  ↪  (),
                  maximum_normalized_energy_residual=normalized_residual,
                  conservation_residual_days=conservation,
                  ↪  maximum_lower_bound_violation=bound_violation)
    return report
```

结果核对。该函数给出可行权重见证，不负责最小化分布距离。其status区分可行、不可行及求解失败；最小W_1的具体优化实现见后面的新增代码。

12．有限模式管网反馈与失败处理

完整有限目录反馈函数。适用域为登记的独立泵送星形支路；发现不合格模式后加禁用约束并重求。

输入：支持范围内的配置、连续时序、求解预算、轮数上限和可选逐轮记录回调。

输出：通过硬校核的候选及逐轮历史；失败则随异常保留原因和已有割约束。

实现思路。反馈循环在登记的有限模式目录内工作：求得候选后，独立检查其水力、温度与服务条件；将新增不合格模式在全部时段的选择变量固定为零，再求下一轮。已排除模式集合防止重复加割；通过、异常、无法产生新割和轮数耗尽都有独立出口，逐轮回调可保存问题定位所需的中间结果。

来源：network_feedback.py，原第440–481行；42行原代码、15行补充注释。

```
# 此接口不是三城聚合网络的通用连续非线性水力优化器。
def solve_with_network_feedback(config, timeseries, *, mip_gap=0.,
↪  time_limit=60.,
                                max_rounds=None, on_iteration: Callable |
                                ↪  None = None):
    # 模式目录和适用配置在构模入口先验证，禁用模式必须来自该有限集合。
    model, catalog, flags = build_feedback_model(config, timeseries)
    # 默认轮数有限；不会承诺任意网络的全局收敛。
    if max_rounds is None:
```

```
        max_rounds = len(catalog) + 1
    # 拒绝无效轮数、时间预算和间隙。
    if isinstance(max_rounds, bool) or int(max_rounds) != max_rounds or max_rounds < 1:
        raise ValueError("max_rounds must be a positive integer")
    if not math.isfinite(time_limit) or time_limit <= 0 or not math.isfinite(mip_gap) or not 0 <= mip_gap < 1:
        raise ValueError("invalid solve budget/gap")
    # 逐轮历史及已排除模式单独记录，便于辨识失败发生在哪一步。
    history, eliminated = [], set()
    # 每轮在已有割约束下求解，然后接受独立模式校核。
    for iteration in range(int(max_rounds)):
        try:
            # 费用最小可行候选仍需通过温度、水力及服务范围检查。
            solution = model.solve(time_limit=time_limit, mip_gap=mip_gap)
            dispatch, audit = audit_feedback_solution(config, model, solution, catalog)
        # 求解或审计异常保存至history，再抛出FeedbackFailure。
        except Exception as error:
            record = {"iteration": iteration, "status": "failed", "type": type(error).__name__, "reason": str(error),
                      "solver_evidence": getattr(error, "summary", None), "previous_cuts": sorted(eliminated)}
            history.append(record)
            # 回调可逐轮落盘，让拒绝方案与失败原因可追溯。
            if on_iteration:
                on_iteration(record, None, None)
            raise FeedbackFailure(str(error), history) from error
        # invalid_modes来自独立硬校核，不由软惩罚值直接替代。
        invalid = set(audit["invalid_modes"])
        record = {"iteration": iteration, "status": "accepted" if audit["passed"] else "rejected",
            "objective_yuan": solution.summary["objective_yuan_per_year"],
            "solver_status": solution.summary["solver_status"], "mip_gap": solution.summary.get("mip_gap"),
            "solution_quality": solution.summary["solution_quality"], "audit": audit,
            "added_mode_cuts": sorted(invalid - eliminated), "previous_cuts": sorted(eliminated)}
        history.append(record)
        if on_iteration:
            on_iteration(record, solution, dispatch)
        # 全部校核通过才返回；最优性范围限于有限目录和登记容差。
        if audit["passed"]:
            solution.summary["network_feedback"] = {"iterations": iteration + 1, "eliminated_modes": sorted(eliminated),
                "audit": audit, "optimality_domain": "finite catalog MILP within recorded solver tolerances; not continuous nonlinear network optimum",
                "soft_penalty_is_not_cash_cost": True}
            return FeedbackArtifacts(model, solution, dispatch, history, audit, catalog)
        # 没有可新增禁用模式时明确失败，避免重复无效求解。
        if not invalid - eliminated:
```

```
            raise FeedbackFailure("non-engineering audit failure; no valid
            ↪  mode cut available", history)
        # 对每个时段将不合格模式的选择变量置零，再重新求解。
        for virtual in sorted(invalid - eliminated):
            for t in model.times:
                model.problem.add_constraint({flags[virtual, t]: 1.}, ub=0.)
        # 登记已排除集合，下轮只对首次发现的不合格模式加割。
        eliminated.update(invalid)
    # 轮数耗尽仍未验收则失败，不把最后一轮写成合格方案。
    raise FeedbackFailure("feedback iteration limit reached without
    ↪  engineering acceptance", history)
```

结果核对。比较每轮audit、added_mode_cuts与previous_cuts，确认新割有物理依据。最终接收条件是硬校核通过，软惩罚只参与目标。

13．热源出力的上游余热共用约束

完整的共享上游热资源装配函数。把直供换热器与热泵已有出力变量共同映射到同一资源池，约束逐时实际取热，而不是让每台设备分别重复使用全部余热。

输入：明确登记的资源池名称、成员热源、可用热功率常量或时序列（kW）、传热效率（无量纲），以及热源逐温位出力变量与逐时COP。

输出：共享资源取热变量shared_heat_resource_use（kW）、等式连接及可用功率上界；资源行同时登记以供结果核验。

实现思路。本函数把正文热源出力$Q_{s,g,t}$（kW）折算为资源池上游取热。直供项为Q/η，热泵项为$Q(1-1/\mathrm{COP})/\eta$；η为无量纲传热效率，压缩机电功不算余热。程序用取热变量与全部成员、全部温位的折算和建立等式，再以逐时可用热功率限制其上界，因此换热器与热泵不能重复占用同一资源。该约束衔接已有热源出力和节点热平衡，仅在配置登记资源池时启用。

来源：physical_extensions.py，原第25–79行；55行原代码、14行补充注释。

```
# 本函数使用已注册热源出力，不新增热源设备。
def add_shared_heat_resources(model):
    """Bound upstream heat drawn jointly by direct exchangers and heat
    ↪  pumps.

    Q_upstream = Q_direct / eta + Q_HP * (1 - 1/COP) / eta.
    eta is the explicitly declared resource-to-device transfer efficiency.
    Compressor electricity is excluded. Cooling/rejection is not allocated
    ↪  to
```

```
    this pool; such sources require a separate, explicitly defined resource model.
    """
    # 资源行保留可用上界、出力系数与取热变量。
    model.shared_heat_resource_rows = []
    names, assigned = set(), set()
    # 只有配置登记的资源池才会增加这组约束。
    for pool in model.config.get("shared_heat_resources", []):
        name = str(pool["name"])
        members = list(pool["source_names"])
        # 资源名称和成员须唯一，避免资源账本重复。
        if not name.strip() or name in names or not members or len(members) != len(set(members)):
            raise ValueError("shared heat resource names/members must be unique and nonempty")
        names.add(name)
        # 同一热源不可分属多个资源池而重复计量。
        if set(members) - set(model.sources) or assigned.intersection(members):
            raise ValueError(f"resource {name}: unknown source or source assigned to multiple pools")
        assigned.update(members)
        # 可用热功率以kW登记，常量与时序二选一。
        scalar = "available_heat_kw" in pool
        column = "available_heat_column" in pool
        if scalar == column:
            raise ValueError(f"resource {name}: specify exactly one available_heat_kw/column")
        # 效率无量纲，输出热量折回上游时需除以它。
        efficiency = _nonnegative(pool["transfer_efficiency"], f"resource {name} efficiency")
        if not 0 < efficiency <= 1:
            raise ValueError("resource transfer_efficiency must be in (0, 1]")
        for source_name in members:
            source = model.sources[source_name]
            # 此资源口径仅接受直供换热器与供热热泵。
            if source.get("type") not in {"direct_heat", "heat_pump"}:
                raise ValueError(f"resource {name}: only direct_heat and heat_pump are supported")
            # 制冷或排热设备须另建资源守恒，不能套用。
            if any(key[0] == source_name for key in model.variables["cooling_output"]) or source.get("heat_rejection_enabled"):
                raise ValueError(f"resource {name}: cooling/rejection needs a separate resource balance")
        for t in model.times:
            available = _nonnegative(
                model._ts_value(t, pool["available_heat_column"]) if column else pool["available_heat_kw"],
                f"resource {name} available heat at {t}",
            )
            terms = {}
            for source_name in members:
```

```
                source = model.sources[source_name]
                for grade_name, grade in model.grades.items():
                    # 按温位汇总同一成员已有的供热功率变量。
                    idx = model.variables["source_heat"].get((source_name,
                    ↪  grade_name, t))
                    if idx is None:
                        continue
                    # 直供取热比例为1；热泵需扣除压缩机电功。
                    fraction = 1.0
                    if source.get("type") == "heat_pump":
                        cop = float(model._cop(source, grade, t))
                        if not isfinite(cop) or cop <= 1:
                            raise ValueError("a resource heat pump requires
                            ↪  finite COP > 1")
                        # 热泵上游取热为供热功率乘以1减COP倒数。
                        fraction -= 1.0 / cop
                    # 再除传热效率，得到每kW供热对应的上游取热。
                    model._add(terms, idx, fraction / efficiency)
            # 取热变量上界为逐时可用资源，等式绑定总取热。
            use = model.problem.add_var(f"shared_heat_resource_use[{name},{
            ↪  t}]", ub=available)
            model.variables["shared_heat_resource_use"][(name, t)] = use
            model.problem.add_constraint({use: 1.0, **{i: -c for i, c in
            ↪  terms.items()}}, lb=0.0, ub=0.0)
            model.shared_heat_resource_rows.append((name, t, available,
            ↪  terms, use))
```

结果核对。核验shared_heat_resource_use的逐时kW值及资源行登记的可用上界、COP和折算系数；适用于明确登记的直供与供热热泵共享资源池。制冷、排热支路不属于本口径。

14．逐楼RC状态、舒适边界与节点交付量

完整的逐楼1R1C约束装配函数，与前述精确离散系数函数互补。室温变量直接承载逐时舒适上下界，实际冷热交付量进入热状态递推，并按节点温位聚合后交给主模型平衡。

输入：通过校验的逐楼热损失系数、有效热容、离散系数a和b、室外温度与得热时序、舒适温度及冷热功率上界、分组边界和互斥模式。

输出：逐楼末态室温（摄氏度）、实际供热和制冷功率（kW）、可选冷热模式二元变量、节点交付量映射以及RC递推等式与末态边界。

实现思路。逐楼末态室温T_t（摄氏度）受逐时舒适上下界约束，供热Q_h与制冷Q_c均为kW。以此前计算的a（无量纲）、b（K/kW）装配$T_t - aT_{t-1} - bQ_h + bQ_c = b(HT_{\text{out}} + G)$；$H$为kW/K，$G$为内得热与太阳得热之和。固定首态移入右

端，循环组连接组内前驱，固定末态另受温度界约束。聚合交付量接入正文节点平衡；冷热互斥二元变量仅按配置启用。

来源：building_thermal.py，原第463–525行；63行原代码、15行补充注释。

```python
# 逐楼热状态是独立RC分支，节点平衡须消费其交付量。
def add_building_thermal_constraints(model: Any) -> BuildingThermalBlock | None:
    """Register on model.problem / model.variables and return load variable maps.

    Needs config, ts, dt, problem and variables, optionally boundary_groups.
    Adds no source equipment, costs, unmet-load variables or reliability credits.
    The caller MUST consume node_heat/node_cooling in physical supply balances.
    """
    # 先校验逐楼输入与分组边界，未启用则返回空。
    inputs = validate_building_thermal_inputs(model.config, model.ts, dt_hours=model.dt,
                                              boundary_groups=getattr(model, "boundary_groups", None))
    if inputs is None:
        return None
    categories = ("building_temperature", "building_heat", "building_cooling", "building_heat_mode",
                  "building_heat_by_node", "building_cooling_by_node")
    # 拒绝重复注册，避免同一建筑出现两套状态变量。
    if any(model.variables.get(category) for category in categories):
        raise ValueError("building thermal variables already registered")
    maps = {category: {} for category in categories}
    temperature, heat, cooling, mode, node_heat, node_cooling = (maps[category] for category in categories)
    p = model.problem
    heat_members: dict[tuple[str, str, int], list[int]] = defaultdict(list)
    cooling_members: dict[tuple[str, int], list[int]] = defaultdict(list)
    for building in inputs.buildings:
        for t in range(len(inputs.frame)):
            key = building.name, t
            # 室温单位为摄氏度，逐时舒适区间直接作为变量界。
            temperature[key] = p.add_var(f"building_temperature[{building.name},{t}]",
                                         lb=building.minimum[t], ub=building.maximum[t])
            # 冷热交付均为非负kW，各自受末端功率上界限制。
            heat[key] = p.add_var(f"building_heat[{building.name},{t}]", ub=building.heat_upper[t])
            cooling[key] = p.add_var(f"building_cooling[{building.name},{t}]", ub=building.cooling_upper[t])
            # 按建筑所属节点和供热温位收集实际交付功率。
            heat_members[building.node, building.heat_grade, t].append(heat[key])
```

```
            cooling_members[building.node, t].append(cooling[key])
            # 仅互斥模式且冷热均可用时引入二元开关。
            if inputs.simultaneous_mode == "exclusive" and
            ↪  building.heat_upper[t] > 0 and building.cooling_upper[t] >
            ↪  0:
                mode[key] =
                ↪  p.add_var(f"building_heat_mode[{building.name},{t}]",
                ↪  ub=1.0, integer=True)
                # 模式为1允许供热；模式为0允许制冷。
                p.add_constraint({heat[key]: 1.0, mode[key]:
                ↪  -building.heat_upper[t]}, ub=0.0)
                p.add_constraint({cooling[key]: 1.0, mode[key]:
                ↪  building.cooling_upper[t]},
                ↪  ub=building.cooling_upper[t])
        # 状态递推只连接所属边界组，避免跨场景串联。
        for group, times in inputs.boundary_groups.items():
            condition = building.boundary[group]
            for position, t in enumerate(times):
                key = building.name, t
                # a无量纲，b为K每kW；供热升温而制冷降温。
                row = {temperature[key]: 1.0, heat[key]: -building.b,
                ↪  cooling[key]: building.b}
                # H乘室外温度与内、太阳得热合成递推常量项。
                rhs = building.b * (building.heat_loss_kw_per_c *
                ↪  inputs.outdoor_temperature[t]
                                    + building.internal_gain[t] +
                                    ↪  building.solar_gain[t])
                # 固定组首时段把初始室温乘a移到等式右端。
                if position == 0 and condition.mode == "fixed":
                    rhs += building.a * condition.initial_temperature_c
                else:
                    previous = temperature[building.name, times[position -
                    ↪  1]]
                    # Addition is essential for single-step cyclic groups
                    ↪  (same index).
                    # 循环组接组内前驱，单时段同索引须累加系数。
                    row[previous] = row.get(previous, 0.0) - building.a
                p.add_constraint(row, lb=rhs, ub=rhs)
            # 固定边界另约束末态室温，不强制循环相等。
            if condition.mode == "fixed":
                p.add_constraint({temperature[building.name, times[-1]]:
                ↪  1.0},
                                 lb=condition.final_min_temperature_c,
                                 ub=condition.final_max_temperature_c if
                                 ↪  condition.final_max_temperature_c is
                                 ↪  not None else np.inf)
    # 多楼以等式汇总交付量，单楼直接复用变量索引。
    for category, members, target in (("building_heat_by_node",
    ↪  heat_members, node_heat),
                                      ("building_cooling_by_node",
                                      ↪  cooling_members, node_cooling)):
        for key, indices in members.items():
            if len(indices) == 1:
```

```
                target[key] = indices[0]  # Alias avoids a redundant row/
                ↪ variable.
            else:
                aggregate = p.add_var(f"{category}[{','.join(map(str,
                ↪ key))}]", ub=sum(p.upper[i] for i in indices))
                target[key] = aggregate
                row = {index: -1.0 for index in indices}
                row[aggregate] = 1.0
                p.add_constraint(row, lb=0.0, ub=0.0)
    model.variables.update(maps)
    return BuildingThermalBlock(inputs, temperature, heat, cooling, mode,
    ↪ node_heat, node_cooling)
```

结果核对。读取逐楼轨迹的temperature_end_c、实际冷热交付与RC残差，并结合舒适摘要的低温、超温及末态偏差核验；对应登记的RC建筑与边界组，不能用聚合平均温度替代逐楼边界。

15．逐时配电接入容量及可靠容量方法入口

build方法内部的连续上下文片段。完整展示逐时电网接入容量、可用率及微电网净负荷约束，并保留后续可靠容量方法的参数准备和合法性检查；两种可靠容量方法的实际装配位于后续源码。

输入：已注册的热泵、电锅炉、循环泵等电负荷系数，既有和新增接入容量上限（kW）、增容造价（元/kW）、备用裕度、可用率时序，以及可选微电网变量。

输出：可选新增接入容量及投资账项、逐时净电负荷容量上界；同时准备可靠本地电源容量行并校验firm_capacity_method的方法身份。

实现思路。程序读取统一电负荷登记表，将热泵、电锅炉和泵耗等形成P_t（kW）。令备用倍率$m = 1 + r$、接入可用率$\alpha_t \in [0, 1]$，逐时约束为$m(P_t + P_{\text{ch}} - P_{\text{local}} - P_{\text{dis}}) \le \alpha_t(K_0 + K_{\text{new}})$，微电网项仅在其启用时加入。新增容量$K_{\text{new}}$（kW）同时进入投资账本，使正文供热方案受配电接入限制。本节末部只准备可靠容量方法入口，后续静态分支另行装配。

来源：model.py，原第2030–2089行；60行原代码、14行补充注释。

```
# Local electricity connection adequacy for heat pumps and circulation
# pumps. This prevents a heat-only optimum that cannot be connected to
# the distribution grid without reinforcement.
# 以下依赖build内既有变量，配置接入模块才装配。
```

```
grid = self.config.get("grid_connection")
if grid:
    # 既有和新增接入容量均为kW，新增量有投资上界。
    existing_grid = float(grid.get("existing_capacity_kw", 0.0))
    max_new_grid = float(grid.get("max_new_capacity_kw", 0.0))
    grid_idx = None
    # 不允许增容时仅使用既有容量，不建立新增变量。
    if max_new_grid > 0:
        grid_idx = p.add_var("grid_connection_capacity", ub=max_new_grid)
        self.variables["grid_connection_capacity"]["system"] = grid_idx
        # 增容造价按元每kW进入既有投资年化账本。
        self._record_investment_cost(
            grid_idx, "grid_connection_capacity", "system",
            ↪  float(grid.get("capex_yuan_per_kw", 0.0))
        )
    # 裕度无量纲，以1加备用比例放大净电负荷。
    margin = 1.0 + float(grid.get("reserve_margin", 0.0))
    for t in self.times:
        grid_availability = 1.0
        # 若登记可用率时序，则逐时截取到0至1。
        if grid.get("availability_column"):
            grid_availability = self._clip_scalar(
                self._ts_value(t, grid["availability_column"]),
                0.0,
                1.0,
            )
        # 读取统一电负荷行，热泵和泵电耗不另算一遍。
        row = self._electric_load_row(t, margin=margin)
        # 微电网分支仅在配置启用时修正接入净负荷。
        if microgrid:
            for category, sign in (
                # 储电充电增加负荷，本地发电与放电减少负荷。
                ("microgrid_battery_charge", 1.0),
                ("microgrid_renewable_use", -1.0),
                ("microgrid_backup_generation", -1.0),
                ("microgrid_battery_discharge", -1.0),
            ):
                self._add(row, self.variables[category][t], margin * sign)
        # 新增容量移到左端且乘可用率，保持线性上界。
        if grid_idx is not None:
            self._add(row, grid_idx, -grid_availability)
        # 净负荷须小于可用率乘既有加新增接入容量。
        p.add_constraint(row, ub=existing_grid * grid_availability)

    # Ensure that electrically supplied firm heat capacity is actually
    # deliverable through the grid connection in a contingency.  The
    # legacy method assumes every installed electric heat source runs
    # simultaneously.  ``portfolio_dispatch`` instead selects a
    # feasible firm-heat portfolio for the peak (and each configured
    # source outage), avoiding double-counting mutually substitutable
    # heat pumps and backup boilers.
    # 以下准备静态可靠容量检查；同时率与裕度不同。
    simultaneity = float(grid.get("firm_simultaneity", 1.0))
    existing_firm_local = 0.0
```

```
    firm_local_row: dict[int, float] = {}
    if microgrid:
        # 本地可再生可靠容量按登记出力因子折算。
        renewable_firm = float(microgrid.get("renewable_firm_output_factor",
        ↪  0.15))
        for asset, factor in (("renewable", renewable_firm),
        ↪  ("battery_power", 1.0), ("backup", 1.0)):
            idx, existing, _ = microgrid_specs[asset]
            existing_firm_local += factor * existing
            if idx is not None:
                firm_local_row[idx] = -factor

    # 选择可靠容量方法；本片段尚未装配该方法约束。
    firm_method = str(grid.get("firm_capacity_method",
    ↪  "all_installed_electric"))
    if firm_method not in {"all_installed_electric", "portfolio_dispatch"}:
        raise ValueError(
            "grid_connection.firm_capacity_method 仅支持 "
            "all_installed_electric 或 portfolio_dispatch"
        )
```

结果核对。核验逐时电负荷行、接入可用率和grid_connection_capacity_kw，并记录firm_grid_capacity_method；本节证明接入约束的装配方式，可靠容量核验应追踪后续实际选择的分支。

16．硬能量守恒下的最小一维W1权重线性规划

完整展示实际代表日保持不变、仅优化权重的主算法；以能量和总天数为硬等式，在权重下界内最小化归一化的一维经验分布W1，并独立重算残差。

输入：完整等间隔实测日序列、已选实际代表日矩阵、原天数权重、逐日权重下界与正数数值容差；代表日值与观测值须同量纲。

输出：通过独立复核的权重、归一化W1及原量纲W1、能量/天数/下界/目标残差；另明确返回不可行、零能量无定义或求解失败身份。

实现思路。设实测经验分布为F，第j个固定代表日的CDF贡献为C_{ij}，原总天数为W。代码用$w = \ell + z$把下界转为$z \geq 0$，同时硬性保持加权总能量与总天数；两组不等式使辅助变量包住各支撑区间的CDF差，目标按区间宽度和实测均值加权，得到最小归一化一维W1。其优势是守恒条件不会被拟合目标让步，数据与代表日也不被改写；所得最优权重仍须独立重算残差。

来源：hard_weight_calibration.py，原第13–97行；85行原代码、15行补充注释。

```python
def minimum_w1_energy_weights(observed, representative_profiles, original_weights,
                              lower_bounds, *, numerical_tolerance=1e-8):
    """Minimize normalized 1-D W1 with exact energy and day conservation.

    Each representative row is one equally sampled actual day; observations
    comprise complete equally sampled days. All values must be finite and
    nonnegative. With zero observed mean the normalized metric is undefined.
    Returns explicit success, infeasibility, or failure with no data mutation.
    """
    # 只读入观测与固定代表日；优化对象是天数权重。
    values = np.asarray(observed, dtype=float)
    profiles = np.asarray(representative_profiles, dtype=float)
    original = np.asarray(original_weights, dtype=float)
    lower = np.asarray(lower_bounds, dtype=float)
    if values.ndim != 1 or not values.size or profiles.ndim != 2 or not all(profiles.shape):
        raise ValueError("observed must be nonempty 1D and representative profiles nonempty 2D")
    # 每行是一个实际代表日；观测须由完整同长日组成。
    m, hours = profiles.shape
    if original.shape != (m,) or lower.shape != (m,) or len(values) % hours:
        raise ValueError("weight/profile dimensions or complete-day observation length are invalid")
    if any(not np.isfinite(x).all() or (x < 0).any() for x in (values, profiles, original, lower)):
        raise ValueError("observations, profiles, weights and lower bounds must be finite and nonnegative")
    tolerance = float(numerical_tolerance)
    if not np.isfinite(tolerance) or tolerance <= 0:
        raise ValueError("numerical_tolerance must be positive and finite")
    # 原权重总天数乘每日采样数须等于观测长度。
    total = float(original.sum())
    if not np.isclose(total * hours, len(values), rtol=0, atol=tolerance):
        raise ValueError("original day weight does not represent the observed complete-day count")
    # 报告明确保留未验证工程运行的身份。
    report = {"method": "standard_LP_hard_energy_minimum_1D_CDF_W1",
              "energy_matching": "hard_equality_with_independent_residual_check",
              "source_data_modified": False, "selected_profiles_modified": False,
              "engineering_operation_validated": False, "total_weight_days": total,
              "original_weights": original.tolist(), "lower_bounds": lower.tolist()}
    # 下界总和超过总天数时直接返回不可行。
    if lower.sum() > total + tolerance:
        return {**report, "status": "infeasible", "reason": "lower_bounds_exceed_total_weight", "weights": None}
    # 零均值时归一化W1无定义，不能伪造零误差。
    mean = float(values.mean())
```

```
    if mean == 0:
        return {**report, "status": "zero_energy_undefined", "reason": "normalized_W1_and_relative_energy_undefined", "weights": None}
    # 合并取值支撑点，将CDF差积分化为区间加权和。
    support = np.unique(np.concatenate((values, profiles.ravel())))
    left, widths = support[:-1], np.diff(support)
    observed_cdf = np.searchsorted(np.sort(values), left, side="right") / len(values)
    # A day weight contributes count(profile <= support) / (H*W) to the CDF.
    # 每列给出一个代表日单位天数的CDF贡献。
    cdf = np.column_stack([np.searchsorted(np.sort(p), left, side="right") for p in profiles]) / (hours * total)
    day_energy, target = profiles.sum(axis=1), float(values.sum())
    n = len(left)
    # 令权重等于下界加非负余量，显式保证逐日下界。
    shifted_target = target - float(day_energy @ lower)
    # 两行硬等式分别锁定总能量与总天数，并按量级缩放。
    equality = csr_matrix(np.vstack((day_energy / max(target, 1), np.ones(m) / max(total, 1))))
    equality = hstack((equality, csr_matrix((2, n))), format="csr")
    rhs = np.array([shifted_target/max(target, 1), (total-lower.sum())/max(total, 1)])
    if n:
        # 两组不等式让辅助量包住CDF误差的正负两侧。
        residual = observed_cdf - cdf @ lower
        cdf_sparse = csr_matrix(cdf)
        absolute = -eye(n, format="csr")
        inequalities = vstack((hstack((cdf_sparse, absolute)), hstack((-cdf_sparse, absolute))), format="csr")
        upper = np.concatenate((residual, -residual))
    else:
        inequalities, upper = None, None
    # 区间宽度除以实测均值是绝对误差辅助量的目标系数。
    objective = np.concatenate((np.zeros(m), widths / mean))
    result = linprog(objective, A_ub=inequalities, b_ub=upper,
                     A_eq=equality, b_eq=rhs,
                     bounds=[(0, None)] * (m+n), method="highs")
    report.update(solver_status=int(result.status), solver_message=str(result.message),
                  solver_iterations=int(result.nit), support_intervals=n)
    # 只有明确的LP不可行状态才登记为不可行。
    if result.status == 2:
        return {**report, "status": "infeasible", "reason": "hard_energy_or_day_bounds_infeasible", "weights": None}
    if not result.success or result.x is None:
        return {**report, "status": "solver_failure", "reason": "LP_did_not_return_optimum", "weights": None}
    # 恢复原权重后按原量纲独立重算守恒与CDF目标。
    weights = lower + np.asarray(result.x[:m])
    energy = float(day_energy @ weights - target)
    conservation = abs(float(weights.sum() - total))
    bound = float(np.maximum(lower - weights, 0).max())
    normalized_w1 = float(widths @ np.abs(cdf @ weights - observed_cdf) / mean)
```

```
    objective_residual = abs(normalized_w1 - float(result.fun))
    # 有限性和四类残差全部通过才交付权重。
    verified = (np.isfinite(weights).all() and abs(energy)/max(target, 1) <=
    ↪  tolerance
                and conservation <= tolerance*max(total, 1) and bound <=
                ↪  tolerance
                and objective_residual <= tolerance*max(normalized_w1, 1))
    return {**report, "status": "ok" if verified else "solver_failure",
            "reason": "independently_verified_hard_energy_and_CDF_objective"
            ↪  if verified else "LP_failed_residual_check",
            "weights": weights.tolist() if verified else None,
            "energy_residual_original_units": energy,
            "relative_energy_residual": energy/target,
            "day_conservation_residual": conservation,
            "maximum_lower_bound_violation": bound,
            "normalized_W1": normalized_w1,
            "W1_original_units": normalized_w1*mean,
            "solver_objective_residual": objective_residual,
            "caveat": "Minimum W1 on fixed selected profiles subject to
            ↪  these statistical constraints only; chronology and
            ↪  engineering operation unvalidated."}
```

结果核对。先查status=ok及权重非空，再核对能量、天数、下界与目标残差均通过，并保存求解器状态和全部残差供后续复核。比较normalized_W1衡量分布拟合；权重成功仅是固定代表日的统计匹配，时序和工程运行证据须另行查验。

17．热源启停三行紧凑凸包与容量关联爬坡

连续展示可选机组组合的变量、开停出力边界、三行状态转换凸包和双向爬坡约束；开机状态为二元，启停事件保持连续而被相邻状态唯一确定。

输入：显式启用的机组组合配置、最大候选容量、既有容量和新增容量变量、最低出力比例、逐时热出力变量及已登记前驱。

输出：每时段一个二元开机状态、两个连续启停变量，以及出力、转换和爬坡线性约束；该段依赖build外层登记表与既有容量约束。

实现思路。以u_t为二元开机状态，s_t, d_t为连续启停事件，三行约束为$s_t - d_t = u_t - u_{t-1}$、$s_t + d_t \le u_t + u_{t-1}$及$s_t + d_t \le 2 - u_t - u_{t-1}$。相邻状态确定后，事件唯一取零或一；两条和约束与转换等式共同蕴含四条逐项上界，保持同一紧松弛而每时段少两行，并省去两个事件整数变量，从而减少模型规模。热出力边界与容量关联爬坡接续装配，启动或停机时通过最大候选容量留出转换余量。

来源：model.py，原第938–1018行；81行原代码、15行补充注释。

```python
# 仅显式启用且候选容量为正时装配此块。
if source.get("unit_commitment", False) and maximum_capacity > 0:
    # 读入最低出力比例与上下爬坡比例。
    min_fraction = float(source.get("minimum_output_fraction", 0.0))
    ramp_up = float(source.get("ramp_up_fraction_per_hour", 1.0))
    ramp_down = float(source.get("ramp_down_fraction_per_hour", 1.0))
    cap_idx = self.variables["source_capacity"].get(s_name)
    for t in self.times:
        # 每时段只给开机状态添加整数身份。
        on = p.add_var(f"source_on[{s_name},{t}]", ub=1.0, integer=True)
        # Startup and shutdown are exact transition indicators once
        # the adjacent on/off states are binary.  Keeping these two
        # auxiliaries continuous removes two thirds of the unit-
        # commitment integer variables without relaxing the model;
        # the linking constraints below uniquely force them to 0/1.
        # 启停事件为连续变量，相邻二元状态将其唯一锁定。
        startup = p.add_var(f"source_startup[{s_name},{t}]", ub=1.0)
        shutdown = p.add_var(f"source_shutdown[{s_name},{t}]", ub=1.0)
        self.variables["source_on"][(s_name, t)] = on
        self.variables["source_startup"][(s_name, t)] = startup
        self.variables["source_shutdown"][(s_name, t)] = shutdown
        # 登记的热电联产最小电出力随开机状态生效。
        if t in minimum_electricity_by_time:
            p.add_constraint(
                {
                    self.variables["chp_electricity_output"][(s_name, t)]:
                    ↪  1.0,
                    on: -minimum_electricity_by_time[t],
                },
                lb=0.0,
            )
        # 启停费用按事件口径入账，避免按普通功率计费。
        self._record_time_component(
            "startup_shutdown_cost", startup,
            ↪  float(source.get("startup_cost_yuan", 0.0)), t, event=True
        )
        self._record_time_component(
            "startup_shutdown_cost", shutdown,
            ↪  float(source.get("shutdown_cost_yuan", 0.0)), t, event=True
        )

        # 各温位热出力合计受开机状态与最大候选容量限制。
        upper_on = {idx: 1.0 for idx in heat_by_time[t]}
        self._add(upper_on, on, -maximum_capacity)
        p.add_constraint(upper_on, ub=0.0)

        # 启用最低出力时，把新增容量系数接入线性下界。
        if min_fraction > 0:
            lower_on = {idx: 1.0 for idx in heat_by_time[t]}
            if cap_idx is not None:
                self._add(lower_on, cap_idx, -min_fraction)
```

```
            self._add(lower_on, on, -maximum_capacity)
            p.add_constraint(lower_on, lb=min_fraction * existing_capacity -
            ↪  maximum_capacity)

    for t in self.times:
        # 前驱来自已登记时序边界，本段据此连接相邻状态。
        prev = self.time_predecessor[t]
        on = self.variables["source_on"][(s_name, t)]
        prev_on = self.variables["source_on"][(s_name, prev)]
        startup = self.variables["source_startup"][(s_name, t)]
        shutdown = self.variables["source_shutdown"][(s_name, t)]
        # Exact three-row state-transition hull.  The two sum
        # inequalities imply all four individual upper links
        # (startup <= on, startup <= 1-prev, shutdown <= prev,
        # shutdown <= 1-on) when combined with the equality.  This
        # is the same tight relaxation with two fewer rows per hour.
        # A one-row cyclic group has prev_on == on. Accumulate
        # coefficients so its transition cancels exactly.
        # 转换等式为启动减停机等于当前开机减前驱开机。
        transition = {startup: 1.0, shutdown: -1.0, on: -1.0}
        # 累加同一索引系数，单时段循环的状态项正确抵消。
        self._add(transition, prev_on, 1.0)
        p.add_constraint(transition, lb=0.0, ub=0.0)
        # 第一条和约束限制事件之和不超过两期开机之和。
        active_sum = {startup: 1.0, shutdown: 1.0, on: -1.0}
        self._add(active_sum, prev_on, -1.0)
        p.add_constraint(active_sum, ub=0.0)
        # 第二条和约束补上关机侧界，合计三行保留紧松弛。
        inactive_sum = {startup: 1.0, shutdown: 1.0, on: 1.0}
        self._add(inactive_sum, prev_on, 1.0)
        p.add_constraint(inactive_sum, ub=2.0)

        # 上爬坡关联装机容量；启动事件提供候选最大容量余量。
        ramp_up_row = {idx: 1.0 for idx in heat_by_time[t]}
        for idx in heat_by_time[prev]:
            self._add(ramp_up_row, idx, -1.0)
        if cap_idx is not None:
            self._add(ramp_up_row, cap_idx, -ramp_up)
        self._add(ramp_up_row, startup, -maximum_capacity)
        p.add_constraint(ramp_up_row, ub=ramp_up * existing_capacity)

        # 下爬坡对称处理，停机事件提供相应变化余量。
        ramp_down_row = {idx: 1.0 for idx in heat_by_time[prev]}
        for idx in heat_by_time[t]:
            self._add(ramp_down_row, idx, -1.0)
        if cap_idx is not None:
            self._add(ramp_down_row, cap_idx, -ramp_down)
        self._add(ramp_down_row, shutdown, -maximum_capacity)
        p.add_constraint(ramp_down_row, ub=ramp_down * existing_capacity)
```

结果核对。解后检查启停转移残差与事件小数偏差，均须符合数值容差，再结合全模型约束审计确认出力与爬坡。统计二元状态数、连续事件数及事件总量，可核查变量压缩；不能只凭费用下降判断机组组合成立。

附录4　数据来源、实际规模与关键参数

本附录按“实际进入结果的输入、只作背景或方法参考的资料、未准入资料”登记数据。规模按城市、建筑、时间窗口和目录工况分别计算；同一天气的原表、派生表和重复求解不叠加为独立样本。公开天气不称当地传感器实测，情景负荷不称计量热负荷，公共热原型与四楼合成参数不称现场校准。下列D01–D15与随文复核清单中的来源编号一致。

4.1　实际使用的数据与规模

原三城算例采用11664个连续城市小时输入行；多年风险层保留2020–2024年15组年度气象序列。外部计量确认另取六栋2017年建筑，设备专项另取两型号165对目录工况。三类材料分别用于检查连续服务、气象扰动、数据压缩和设备转换关系，不合并成一个现场实测数据集。

表 4　　实际输入对象、规模与结果用途

数据类别	确用规模	来源及处理	结果用途与边界
D01 原三城全季	乌鲁木齐4392 h；固原4392 h；武汉2880 h。合计11664行。	NASA逐时天气＋18 °C基温度时缩放的情景负荷；源容量、价格与资源曲线按配置设定。	三场景规划、锚定投资全季校核及相关策略对照；负荷不是当地热量表读数。
D02 多年风险层	3城×5年＝15组；每城43848 h，合计131544个城市小时。风险权重12案、容量就绪检查60案年。	2020、2024各8784 h，其他年各8760 h；历史年度气象链标注LST，负荷由情景规则派生。	既有CVaR扫描与5气象年容量边界检查；60次检查不增加独立数据量。
D03 六楼计量确认	六栋；52560个日历建筑小时；有效计量51955 h，联合天气有效51589 h；完整日46104 h。	BDG2作者清洗版热水、蒸汽、冷水各两栋；保留缺失，按完整24 h日筛选。	1921个完整建筑日压缩为54个代表日（1296 h）；用于权重守恒、峰值/爬坡与W1检验。
D04 固定热泵专项	TCAV035BHE（83对），TCAV065BHE（82对），共165对Q, P；3城各2880 h，固定整机LP六案。	TICA标识目录的第三方分发副本；20份公共原型的合成尺度为960 m^2。	以实际目录轴约束制热量和输入电功率；同服务范围对照能源费，不含采购和投资最优。

续下页

表4（续）

数据类别	确用规模	来源及处理	结果用途与边界
D05–D06 公共原型与双楼	公共原型48 m²；双楼四案，每案2楼×2880 h，同一武汉天气。	BESTEST600规定几何及派生1R1C参数；B楼热容倍率和共享配电上限为研究设定。	分开检查热容轴、配电轴与每楼状态；四案没有新增四份独立天气或现场建筑。
D07 四楼热状态	四栋合成建筑共44000 m²；服务年8784 h，冬季2928 h；分别35136、11712个建筑状态小时。	武汉公开天气＋分别登记的面积、H、C、占用和舒适边界。	C4四楼逐时状态和建筑、站点、源端账本；冬季位于服务年内，不加总为独立样本。
D08 参考管网	72 h；近、远两支路，每支5个有限稳态模式；软惩罚关闭/开启两案。	取乌鲁木齐共同天气窗口前72 h；几何、水物性、限值与模式均为设计参数。	末端温位、压降和源热–管损–送热–泵电核对；未作实测管网或冷启动验证。
D09–D10 费用与排放	2024年11月至2025年3月，5个月×4时段＝20个完整电价；电力平均因子按各计算轴分别登记。	湖北公布能源价在账户适用条件下使用；排放为2023年平均因子参考。	专项共用湖北价格/因子作为方法轴；原三城仍保留自己的历史情景价格与国家平均因子。

4.2 时间窗口、单位与计量筛选

所有逐时状态更新的时间步为1 h。三城原窗口按北京时间小时标签读取，起点含、终点不含；如下表列出的终点均为排他端点。固原窗口是论文情景运行边界，不能以4392 h代替当地行政供暖期。武汉窗口借用了2024年武昌及周边项目服务公告，也不是武汉全市法定供暖期。

表 5　连续运行窗口及计数口径

对象	开始（含）	结束（不含）	小时	窗口身份
乌鲁木齐原全季	2024-10-10 00:00	2025-04-11 00:00	4392	历史情景输入
固原原全季	2024-10-15 00:00	2025-04-16 00:00	4392	历史情景输入；不等同11月1日起公告
武汉原全季／设备共同轴	2024-11-16 00:00	2025-03-16 00:00	2880	设备三城专项共同采用这一个窗口
C4四楼服务年	2023-07-01 00:00	2024-07-01 00:00	8784	跨闰日服务年；不等于2023日历年

续下页

表5（续）

对象	开始（含）	结束（不含）	小时	窗口身份
C4四楼冬季	2023-11-15 00:00	2024-03-16 00:00	2928	服务年内子窗口
72 h参考管网	2024-11-16 00:00	2024-11-19 00:00	72	设备共同轴的前三日

NASA请求采用城市参考点：乌鲁木齐(43.8256°N, 87.6168°E)、固原(36.0159°N, 106.2426°E)、武汉(30.5928°N, 114.3055°E)。温度为°C，相对湿度原字段为%，风速为m/s。太阳辐照按存档字段单位处理：旧年度表标为kWh/(m²·h)，后续UTC表标为Wh/m²的小时累计量；不能只改字段名而跳过单位核对。NASA温度等气象与辐射使用不同网格产品，不能把一个统一空间分辨率转写为全部字段或现场精度。

多年风险结果对应原LST气象链；2026年10月保存的UTC重建包属于后续核验材料。C4使用武汉2023、2024年旧天气，并做了专项时间转换核对。时钟、单位与文件身份核对保证读入一致，不能据此声称气象精度或本地热负荷已完成校准。

六栋计量建筑均未参与此前开发样本；site仍与开发数据有重合。使用作者已规范化的小时kWh字段，除以1 h得到小时平均kW，不再按原始燃料或MJ重做一次单位换算。计量与联合温度覆盖率均须达到95%；非负、有限且24 h完整的日才参与压缩，其余有效小时仍保存在损失核对中。热水通道的采暖与生活热水分界并未逐楼查明。

表 6　六栋2017年计量样本的实际筛选数量

通道	建筑ID	有效计量 h	联合天气 h	完整日N	代表日M
热水	`Bobcat_education_Alissa`	8641	8617	320	8
热水	`Eagle_assembly_Ian`	8638	8636	345	10
蒸汽	`Bull_assembly_Brandon`	8566	8531	304	9
蒸汽	`Cockatoo_assembly_Genoveva`	8647	8405	276	9
冷水	`Bobcat_education_Angela`	8721	8697	338	9
冷水	`Bull_assembly_Dorethea`	8742	8703	338	9

K=6是基础聚类请求数，补入极端日后实际M为8–10。六楼合计1921个完整日，保留54个代表日；不同权重方法使用相同已选24 h曲线。硬权重候选以

总权重天数与保留域能量作等式约束，独立复核残差，再比较峰值、爬坡及负荷持续曲线的W1。研究统计门槛下的4/6准入只支持该压缩检验，不能转作工程调度、费用、碳或室温认证。

4.3 关键输入参数与来源关系

表7给出影响供热量、状态、费用和工况边界的主要数值。原情景参数仍按原配置登记；有公开来源的原值和派生值分别说明。没有追到项目测量或单项报价的参数保留情景身份，不给它补上“规范默认”或“厂家实测”的名义。

表 7　　主要数值输入、计算关系与资料身份

参数组	实际登记数值	使用方式	来源与范围
原三城温位	乌鲁木齐、固原75/45 °C；武汉45 °C。	供热品位与需求通道配置	D01情景温位；不是已测管网供回水。
原三城设计室外温度	乌鲁木齐−26、固原−26、武汉−7 °C。	既有设计/保供校核输入	D01历史情景值；不冒充D11规范表。
原三城财务轴	折现率5%，资产寿命20年；时间步1 h。	容量费用年化、运行量累计	D01研究配置；不是实际融资合同。
原三城碳价／储备	碳价0.10/0.10/0.12元/kg；保供容量储备25%/15%/12%；关键峰段12/12/6 h。	按乌鲁木齐／固原／武汉顺序	D01情景政策和保供轴；不称行政统一标准。
原三城电价范围	0.246848–0.691；0.19173–0.35607；0.4953–1.1795元/kWh。	三个历史输入CSV的实际最小、最大值	D01保留原情景价；与D09另表公布价不混用。
旧源与接入费用	空气热泵1700–1850、地源3100–3150、电锅炉560–650元/kW；电接入新增容量费用820元/kW。	所列为旧三城配置中对应技术的资本参数	D01情景报价轴，未建立当地项目逐项原始报价关系；不转为本轮新采购事实。
公共原型H、C	48 m^2，129.6 m^3；H=0.1043 kW/K，C=0.535576 kWh/K；内增益0.2 kW。	BESTEST600规定几何，围护/渗透项与室内侧材料归并	D05公开原型派生；不是地方楼体辨识。
热泵合成尺度	20份原型，960 m^2；H=2.086 kW/K，C=10.71152 kWh/K。	对原型H、C线性复制为研究尺度	D04/D05；不称20户实测或已识别等效楼。

续下页

表7（续）

参数组	实际登记数值	使用方式	来源与范围
热泵服务与资产边界	室温20–27 °C，初末20 °C；出水45 °C；1整模块；电热90 kW、独立制冷80 kW、配电100 kW。	固定设备六案共享服务包络；独立冷机EER=3，电热效率1	D04固定理想资产；目录没有提供实际部分负荷/启停曲线。
两型号工况	035型号83对，065型号82对Q,P；目录环境轴−26至55 °C。	逐点核对；Q,P分别插值，再计算COP=Q/P	D04打印轴内且所需角点齐全才准入，空白角点和外推拒绝；湿度/化霜信息未补齐。
双楼2×2参数	A：H=0.1043，C=0.535576；B：H=0.15645，C=0.535576或2.142304。H单位kW/K，C单位kWh/K。	B楼H保持1.5倍，C设1倍或4倍；共享电上限另设开/关	D06设计轴；没有更换实际墙体或现场测量。
双楼公用电上限	6.4796375 kW；按$2.5H_A[20-\min(T_{\text{out}})]$预先确定。	四案同2880 h，初末20 °C、服务20–27 °C	D06；原理想热/冷设备各1000 kW、效率/EER均1，共享配电是唯一联系。
参考管网几何	近支路单程30 m、远支路150 m；内径0.015 m；粗糙度5×10^{-5} m。	供回双腿热损分别计算	D08设计值，不代表某现场DN管径和竣工长度。
参考管网热边界	源供65、地温5、末端供温下限55、回温上限45 °C；每腿散热系数0.3 W/(m·K)。	两支路同一硬边界；送热与热损分别计量	D08设计限值；不能宣称规范自动给定。
参考管网水物性／泵	密度985 kg/m³，比热4.18 kJ/(kg·K)，黏度0.00055 Pa·s；泵效率0.72。	两支路独立泵，局部阻力系数2。	D08固定设计参考；不是随温实测物性。
参考管网模式／容量	各支路质量流量0.03、0.06、0.08、0.12、0.18 kg/s；模式回温40 °C；源40 kW，配电50 kW。	每支每小时最多一个稳态开态，另有占空边界	D08有限候选模式；不作连续全水力可行域认证。
参考管网硬／软限值	压降300 Pa/m；流速3 m/s；单支泵0.25 kW、瞬时热输入30 kW；温差目标20 °C、软带1 °C。	硬限筛除模式；软惩罚0或0.2元/(°C·开态h)	D08设计机制测试；软惩罚与源/泵能源费单列。

续下页

表7（续）

参数组	实际登记数值	使用方式	来源与范围
电力平均排放轴	原三城0.5306；公共原型等专项共用湖北0.4044 $kgCO_2/kWh$。	2023年国家／湖北年度平均因子	D10；不是2024/25逐时边际排放。新疆0.6021、宁夏0.6187为待重跑参考，未替换旧结果。

H由已登记围护传热、渗透等项求和；C由选定室内侧材料的$Ad\rho c_p$归并并换算为kWh/K。它们的量纲与1R1C方程一致，但源于规定原型与归并假设，未用三城实楼室温及实际供热输入作辨识。四楼C4与公共原型双楼是两组独立设计，不能混称同一批建筑。

表 8　C4四栋合成建筑的面积和热参数

建筑	面积m^2	H kW/K	C kWh/K	热损W/(m^2K)	热容Wh/(m^2K)
`apartments_a`	18000	16.2	1260	0.9	70
`office_a`	7000	7.7	315	1.1	45
`apartments_b`	14000	14	980	1.0	70
`retail_b`	5000	6	175	1.2	35
合计	44000	43.9	2730	–	–

上表全部标记为`synthetic_assumption_not_calibrated`。初温20 °C，基础上下界18–28 °C；实际计算同时使用每楼逐时占用、增益及温度边界列。面积、H、C与舒适规则提供可复算的异质设计对象，不能作为四栋当地建筑的实测档案。

4.4　条件电价与费用边界

专项实验选择湖北普通工商业代理购电、单一制、不满1千伏的公布能源价行。适用账户需满足已登记供电容量区间与至少12个月的选择期限等条件，论文未核实任一具体账户的资格或接入。公布的完整能源价直接代入小时电量；不再乘一次峰谷系数。配电模型上限不是账户登记容量，设备380 V标签也不能代替计费供电电压身份。

表 9　条件参考轴的公布能源电价（元/kWh）

月份	尖峰	高峰	平段	低谷	原表定位
2024-11	1.1724	1.0188	0.7762	0.5187	11月通知第1页
2024-12	1.2908	1.0368	0.7927	0.5187	12月通知第1页
2025-01	1.1995	0.9744	0.7581	0.5153	2025年合订通知第12页
2025-02	1.0553	0.9313	0.7353	0.5273	2025年合订通知第11页
2025-03	1.1144	0.9726	0.7484	0.5105	2025年合订通知第10页

本窗口小时归属为：尖峰18、19时；高峰16、17、20–23时；平段6–11、14、15时；低谷0–5、12、13时，各时间戳标记随后1 h。电价含公布列中的线损、输配、基金及系统运行等能源价组成；实际功率因数调整、其他服务、结算修正、采购、维护与配电投资不在专项能源费中。热泵三城对照共用该条件轴以隔离天气/设备差异，不预测乌鲁木齐、固原的本地账单。

4.5　原始来源与仅参考资料

来源链接指向数据或公布材料本身，详细文件路径、物理页码、原响应、筛选规则与SHA-256保存在`review/data_inventory_r8p.json`。指纹只核对保存身份，不认证资料准确性、现场适用性或方案最优。下面将实际数值来源与背景参考并列，避免以下载库存扩大论文使用量。

表 10　来源链接、确用内容与参考边界

来源	材料与定位	实际用途／未采用范围
D01–D02 NASA POWER	官方逐时接口说明；各城市请求、原响应及UTC/LST身份随清单保存。	实际气象输入；温度、湿度、风、辐照；负荷与太阳热可用率是后续派生。
D03 BDG2	Miller等，Scientific Data 7:368（2020）；作者固定版本数据仓库。	确用六栋2017年计量通道；原型建筑H/C、室温、设备COP与当地账单不由此获取。
D04 TICA目录	24页产品目录分发副本，物理第15页Q, P网格；第14页额定工况另记。	两型号83/82对打印值经逐点核对；原刊端该具体表版本尚未认证。

续下页

表10（续）

来源	材料与定位	实际用途／未采用范围
D05–D06 BESTEST	1995年NREL/TP-472-6231公共原型报告，物理第34、36、39、41、48–50、53页。	规定几何和材料用于H/C派生；没有当地辨识或BESTEST软件认证。
D09 五个月电价	2024年11月表、12月表及2025年合订表。	前两份为政府转载；合订表为国网湖北公布。选同一单一制、不满1kV行，物理页码见表。
D09 账户与时段规则	鄂发改价管〔2024〕77号及〔2023〕163号原件政府转载。	77号第2页范围/时段、第3页生效日；163号第2页账户选择条件。按历史窗口作条件引用。
D10 电力因子	生态环境部、国家统计局2025年第47号公告及2023年因子原附件。	实际国家0.5306／专项湖北0.4044；省级重跑建议表不是全部旧结果的已用输入。
D11 设计气候表	GB 50736–2012附录A刊载原表，1971–2000气候统计。	仅参考：供暖设计温度乌鲁木齐−19.7、固原−13.2、武汉−0.3 °C；不替换原情景值，不宣布现行规范审查完成。
D12 城市规划	乌鲁木齐2025–2027供热规划草案；固原2021–2025清洁取暖规划；蔡甸2020年项目审批材料。	场景背景参考；规划面积/容量不自动等于数值原型存量、竣工资产或本轮实绩。
D13 OSM空间资料	OpenStreetMap来源与许可；原CSV导出缺快照日期、查询范围和版本元数据。	三城建筑质心、道路等可作空间参考；现有审计未执行优化，不计入本论文已用管段或实楼H/C。
D14 UTC重建包	保存三城15个年度、3个季节NASA请求和坐标索引，区别于旧年度LST输入。	用于来源与时钟核验；原返回含边界补齐，按最终窗口裁剪；不能据库存数量宣称已新跑年度实验。
D15 未准入与待补	39页营销资料、跨型号采购价格、没有原始凭证的口述性能；当地实楼测温、实际热输入、资产报价和连接审批。	不补作实测参数。目录空点、外推点、无适用账户或不明来源项保留拒绝/待核状态。

完整设备存量、费用、资源曲线与候选管段见结构化清单。公开计量、目录工况和热原型分别检验压缩误差、设备转换与状态守恒；情景参数保留原身份，工程实施前另核现场凭证。

附录5　补充对照实验与核验记录

本附录整理已保存的六组专项证据，本轮仅核对身份、汇总量和文字适用范围，不重新求解。原三城主案例、原算法章节的消融与压力测试仍留在原位置。各组输入、设备、窗口和费用口径独立；公开计量、公开天气驱动的设计原型与合成数学场景分别标注，不能将其改善率相加或视为同一工程收益。以下E编号沿用随附证据索引；精确路径、SHA-256及本轮只读核对见`review/experiment_evidence_r8p.json`。

表 11　　补充验证的对象、分母与已有结果

证据与模块	冻结对象与比较条件	结果及限制
E01 代表日权重	六栋BDG2海外计量建筑，同8–10个实际代表日，五类输出共30条记录	软/硬归一W_1中位数12.3150%/4.4775%；六例硬LP成功，统计准入3/6增至4/6，两例原峰丢失拒绝。仅验证完整日统计压缩。
E02 厂家曲线	三城、两型号固定单模块，同资产、同服务2880小时，六组调度/恒温对照	电量费下降1.1013%–1.5970%，用电均增加；厂家工况与理想部分负荷假设下的有限LP，不含采购投资或当地实账。
E03 建筑热惯性	双楼连续2880小时，纯热容倍率和公用电上限四格	增大热容省费但增加电量与固定因子碳排；公用上限使热容省费少150.52元。四格gap为0，独立守恒与状态核验通过。
E04 管网反馈	72小时参考管网，无/有设计惩罚各三轮，交付191.5475 kWh相同	每组保留两轮不合格模式，第三轮压降237.96 Pa/m、远端供水57.45 °C，满足300 Pa/m、55 °C；无惩罚电量费178.64增至224.26元。有限稳态星型设计网络。
E08 四楼连续	武汉合成参数数学场景，固定规划投资；8784小时服务年与2928小时声明冬季	两组逐楼状态/舒适/初末检查均零违约；运行gap为0.1344%和0.4755%。不能证明投资层或真实现场性能。
E09 策略选取	已见三城，各九条固定策略，六权重、四预算；27条连续轨迹、72条选择	频数/软校准实际合格9/12、11/12；四种硬方法均12/12且库内遗憾为0。新候选没有胜过硬W_1的选策证据，费用与能量估计排序不同。

5.1　E01：六栋真实计量负荷的完整分母

2017年热水、蒸汽、冷水各两栋，均未用于原开发建筑集合；站点均已见，故不声称未见站点泛化。先按计量及联合温度覆盖率至少95%、非负有效记录等规则筛选，再冻结六栋，求解或统计门槛失败后不换样本。来源准入共

记录66次判断，接受6次、拒绝60次，理由随选取记录保存。已有九例只作开发参考，不并入本六例的分母。基础聚类$K = 6$、种子42，加极端锚点后实际代表日为8–10个；三种候选均使用同一组真实日，不填缺失小时、不改测值。

归一误差为$W_1/\overline{L}$，$\overline{L}$采用保留完整日的平均负荷。硬候选采用成熟线性规划，在日数守恒、完整日能量等式、极端日权重至少1日及普通日至少10^{-6}日条件下最小化一维CDF积分误差；软候选仍用原正则化10^{-4}，不能称能量硬保证。采用规则为预登记研究门槛：相对能量误差不超过0.1%、W_1不劣于未校准极端日、保留范围峰值/最大爬坡及原有效观测峰全部保留。它不是官方工程验收要求。

表 12　六栋建筑全部权重候选及采用结果，W_1列单位为%

建筑短名/类别	日数	代表日	频数	软	硬	软/硬采用
Alissa/热水	320	8	22.0936	18.0043	4.8097	是/是
Ian/热水	345	10	16.0723	10.0363	4.9463	否/否
Brandon/蒸汽	304	9	17.6675	20.9090	4.1453	否/是
Genoveva/蒸汽	276	9	8.2654	8.2519	3.8920	是/是
Angela/冷水	338	9	14.8176	14.5937	5.4304	否/否
Dorethea/冷水	338	9	2.9310	2.7885	2.0065	是/是
全六例中位数	–	–	15.4450	12.3150	4.4775	3/6、4/6

短名对应完整ID：Alissa为`Bobcat_education_Alissa`，Ian为`Eagle_assembly_Ian`，Brandon为`Bull_assembly_Brandon`，Genoveva为`Cockatoo_assembly_Genoveva`，Angela为`Bobcat_education_Angela`，Dorethea为`Bull_assembly_Dorethea`。六例硬LP均成功，最大绝对相对能量残差3.58×10^{-15}，这是保留范围内的数值守恒。Brandon软校准虽然改善能量偏差，却使W_1比频数更差而被拒绝；硬方法没有该劣化。

表 13　完整日筛选损失与原观测峰诊断（全部六例）

短名	有效小时	排除有效小时	排除观测能量/%	原峰保留/%
Alissa	8641	961	34.7855	100.0000
Ian	8638	358	11.0041	97.9006
Brandon	8566	1270	25.7411	100.0000
Genoveva	8647	2023	29.6328	100.0000
Angela	8721	609	11.4317	99.9682
Dorethea	8742	630	4.5866	100.0000

两例硬候选的拒绝原因均为原观测峰保留失败，而非LP不可行；频数回退也未通过研究门槛。排除能量的分母是所有有效计量小时的已观测能量，不是填补后的全年能量。仅在完整日比较的峰值为100%时，仍可能漏掉原观测峰，故必须保留此第二张表。历史`selected_method`有四条硬候选错标为软方法，已有只改标签的勘误旁表；本轮按`candidate_method`和`statistical_candidate_adopted`读取，原结果和锁不改。

记录入口：`docs/implementation_20261002/algorithm_hard_weights/`，含`protocol.json`、`selection.json`（含来源准入拒绝）、`results.json`、`adoption_records.json`、`failures.json`、`run_manifest.json`、`independent_verification.json`与`method_identity_erratum.json`。30条方法记录和六个LP见证已有独立CDF重建检查；没有成本、投资、舒适度、储热或完整时序调度认证。

5.2　E02：三城两型号厂家工况与固定资产调度

TICA TCAV035BHE、TCAV065BHE分别取同型号有效制热量Q/输入功率P后求COP，不跨型号拼接；两型号合计165对Q/P数据，分别83、82对，空白工况保持缺失。单整模块数量固定，出水45 °C，90 kW电阻备用及100 kW接电上限；理想小时占空比例不代表厂家实测变频、除霜或启停性能。研究规模为20个相同48 m²公共原型的合成缩放，不是20栋现场建筑。

每城同一2880小时天气、20–27 °C与初末20 °C，恒温基准与全窗完美预知LP同资产、同服务。湖北条件分时电价及0.4044 kg/kWh平均因子共同用于三城，以隔离方法与天气；不是乌鲁木齐、固原当地电费或边际碳排，武汉账户资格也未核验。

表 14　六组厂家曲线对照全量结果；变化量为优化减恒温基准

城市/型号后缀	基准电量费/元	优化电量费/元	节费/%	电量变化/kWh
乌鲁木齐/035	60174.37	59375.62	1.3274	+1398.80
乌鲁木齐/065	42787.78	42313.34	1.1088	+753.51
固原/035	62513.27	61605.21	1.4526	+1548.64
固原/065	44657.69	44165.89	1.1013	+874.83
武汉/035	15384.19	15210.85	1.1268	+364.93
武汉/065	14889.78	14651.99	1.5970	+728.99

表 15　六组优化用电与相对基准的参考碳排增量

城市	型号	优化电量/kWh	碳排增量/kg
乌鲁木齐	035	79659.35	565.67
乌鲁木齐	065	56422.54	304.72
固原	035	83685.01	626.27
固原	065	59748.50	353.78
武汉	035	20948.64	147.58
武汉	065	20493.54	294.80

六组状态、服务及数值对偶检查均通过；严格下界另按有限binary64电量费LP的有理数弱对偶构造，只限定本次目标。历史比较表的对偶列仍是数值估计，不把它改称严格下界，更不推导碳排或资本支出的误差区间。

记录入口：`docs/final_optimization_20261002/heatpump/optimization_01/`，包括`optimization_comparison.csv`、`protocol.json`、`manifest.json`、`failures.json`、`independent_optimization_audit.json`和`native_lp_bound_sidecar.json`；每城每型号子目录保存调度、求解状态、汇总与对偶记录。六组全部完成，未删不利能耗结果；不含固定/需量费、资本、运维、回收期和真实设备实测节省。

5.3　E03：纯热容与配电上限的四格及状态账本

两楼各48 m^2；A的$H_A = 0.1043$ kW/K、$C_A = 0.535576$ kWh/K始终固定。B始终$H_B = 0.15645$ kW/K，仅将C_B从0.535576改为2.142304 kWh/K，这是研究设计轴，不是实测材料或另一BESTEST型号。第二轴仅移除/保留6.4796375 kW公用电上限，其值在求解前由共同天气最低温与固定H_A确定，

不随热容调参。四案都是2024年11月16日至2025年3月16日的连续2880小时、1小时步长，含起点不含终点，不拼接代表日。

正文表1已完整列出四格费用、电量、碳及峰值；以下保留约束与状态记录，防止把配电约束的作用误读为热容普遍省能规律。

表 16　四格完整状态核验与B楼热库存（相对20 °C）

案例	电上限绑定/h	B库存峰/kWh	B围护热损/kWh	求解gap/%
$C_B \times 1$、无上限	0	3.7490	5554.19	0
$C_B \times 4$、无上限	0	14.9961	6124.52	0
$C_B \times 1$、有上限	125	3.3461	5481.96	0
$C_B \times 4$、有上限	530	11.2114	5871.36	0

无上限的热容增大带来电量费-427.2017元、电量$+570.3316$ kWh、碳$+230.6421$ kg；有上限则为-276.6785元、$+335.4598$ kWh、$+135.6600$ kg。差中之差为$+150.5232$元，即配电约束下热容的省费收益减小。两楼每案内部与太阳得热各576 kWh；用解析小时温度积分计算围护热损，核对净送热加得热等于围护热损加热库存变化，而非用终点温度矩形近似。热库存是该模型的显热状态，不等于某次需求响应事件的可用储热容量。

四案RC卷积残差最大2.49×10^{-14} °C，初末库存、冷热互斥及电上限均在登记数值容差内通过。每案120秒上限、gap目标0；求解器均报告Optimal、gap0，范围是固定理想设备的有限MILP。独立卷积、账单与保存变量重建审计本身不另行证明最优性。

记录入口：`docs/final_optimization_20261002/multiscale/run_03/`，包括`protocol.json`、`protocol_lock.json`、`comparison.json`、`failures.json`、`independent_verification.json`及`analysis/case_metrics.csv`、`analysis/RESULTS.md`；各案结果保存逐楼轨迹、完整变量和独立核验。四案均保留，零失败。该设计轴对照不证明真实建筑RC预测精度、建筑聚合等效性、三城普遍规律或联合投资改善；能源费不含工程资本和全生命周期费用。

5.4　E04：72小时有限管网模式的排除与重算

实验用公开天气驱动的设计星型管网、各支路独立泵及有限稳态开启模式；每轮对不满足固定硬阈值的模式加入禁用约束后重新求解。压降阈值300 Pa/m、远端供水至少55 °C是本实验登记设计值，不称为普遍国家标准或现场验收值。未改变支路交付量，也不把压降下降说成降低运行费。

表 17　　全部两组、六轮物理反馈记录；软项与电量费分开

设计惩罚/轮	电量费元	软项元	最大压降Pa/m	最低远供°C	物理通过
关闭/1	178.64	0	1081.42	61.52	否
关闭/2	195.01	0	504.33	59.85	否
关闭/3	224.26	0	237.96	57.45	是
开启/1	178.64	3.11	1081.42	61.52	否
开启/2	195.01	3.12	504.33	59.85	否
开启/3	224.26	10.73	237.96	57.45	是

第一轮禁用模式4、9，第二轮禁用3、8，第三轮无需新增切除；两组相同物理过程，软项0.2元/（度·开启小时）只改变登记目标，不提供额外物理保证。六轮交付均191.54748 kWh。达标轮源侧热量285.9832 kWh、输配热损94.4357 kWh、泵耗0.19012 kWh，购电286.1733 kWh；开启软项后的总目标234.9943元不能当作纯电费。

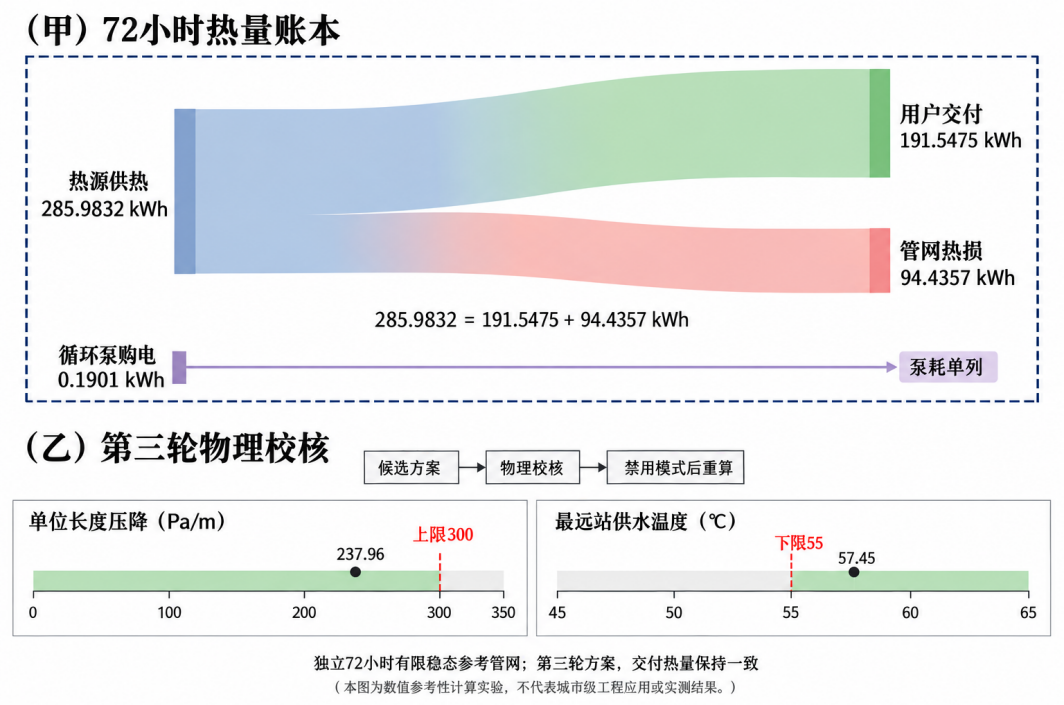

图 29　72小时参考管网的热量账本与物理校核。循环泵购电单列；流带及点位示意分配关系与阈值比较，精确值以标注为准。仅在校核失败时禁用模式后重算；第三轮通过两项阈值后结束，前两轮与费用变化保留于本节表格。

每轮时限60秒、目标gap0，均保留原求解状态、gap0与物理检查的分离记录。达标轮的历史结果属性因原浮点下界略大于可行值而保守记为缺证据，不升级为严格全局证明；物理可行审计仍通过。该有限稳态检查未覆盖冷启动、运输时滞、共泵水力平衡、RC脉冲舒适度、投资或动态故障认证。

记录入口：`docs/implementation_20261002_followup/parallel_stage_03/network/run_02/`，含`protocol.json`、`config.yaml`、`hourly.csv`、`comparison.json`、`manifest.json`与`failures.json`；`results/soft_off/history.json`和`results/soft_on/history.json`保留全部两轮拒绝及达标轮，不只展示最终通过点。

5.5　E08：四楼固定投资连续状态检查

该组是武汉已登记合成参数数学场景，以相同规划方案冻结投资后复算四个独立建筑状态，并非四楼现场测温验证。服务年为北京时间2023年7月1日

至2024年7月1日，共8784小时；声明冬季为2023年11月15日至2024年3月16日，共2928小时，均含起点、不含终点。

表 18　四楼两组连续窗口全部结果与运行层属性

项目	服务年	声明冬季	解释
小时	8784	2928	单个连续窗口
交付热量/kWh	958337.82	930728.48	四楼合计
交付冷量/kWh	1327093.47	711.22	冷服务另核
现金运行费/元	285195.84	122974.20	仅对应本窗口
求解gap/%	0.1344	0.4755	请求容差0.5%
求解时间/s	133.07	48.21	总墙钟另留存
最大RC残差/°C	1.10×10^{-13}	6.75×10^{-14}	独立逐楼复算
舒适/初末违约	0/0	0/0	登记边界内

服务年现金总成本按纠正账本为586978.7050元，其中年化资本301782.8654元已包含一次；不能再次叠加资本。声明冬季122974.2004元只作同期运行费，不折成年费用、利润或回收期。两组运行请求时限分别1800/900秒，均在0.5%登记容差内报告Optimal且状态检查通过，含固定投资标记、数值审计及新增投资零检查；仍不能说明投资计划联合最优。来源等级记录保留低等级设备参数，RC检查范围不替代热源、网络或真实建筑校准。

记录入口：原工程`outputs/competition_20261001/C4/optimized_annual_01/`与`outputs/competition_20261001/C4/optimized_season_03/`的`run_record.json`、`summary.json`、`independent_rc_audit.json`、`c6_building_audit.json`、`corrected_cost.json`及逐时轨迹。既有R8L源包`evidence/C4/ARCHIVE_INDEX.json`登记原路径、原SHA与无损gzip副本；本轮核对压缩身份及解压原身份，不修改其历史来源审核状态。

5.6　E09：三城27条固定策略的全部六方法

该组为已见三城的开发确认。每城2880小时、九策略、同八个真实代表日；六方法均先于真实策略电费/能耗完成校准。三城27条完整轨迹共77760小时，不声称减少仿真量。九策略编号0–8依次为零预热、全谷段1/3 °C、05时末端1/3 °C、13时末端1/3 °C、双末端1/3 °C；跨日状态继承、自然漂移上限27 °C、最后一小时恢复20 °C并计冷热能量。

预算均以各城同一完整季零预热电量为基线，倍率1、1.02、1.05、1.10；固定碳因子使能量/碳帽冗余。按估计费用在估计帽内选策略，费用并列容差10^{-8}元，再按最小策略编号；实际帽与服务另核，无估计可行策略不回退。遗憾只对实际可行选择计算，超帽/无选择留空；oracle只在相同九策略库内，不是全局调度最优。

表 19　六方法完整27策略估计与12次选择分母；误差列为%

方法简名	电费 MAPE	能量 MAPE	最大能量 APE	合格/12	超帽/无选
频数	0.556911	0.267761	0.718644	9	2/1
软能量	0.448316	0.044228	0.291601	11	0/1
硬W_1	0.140535	0.057807	0.315063	12	0/0
硬最小L_1	0.460634	0.043351	0.289511	12	0/0
价/时窗配对L_1	0.485844	0.050416	0.289022	12	0/0
硬W_1加候选L_1上限	0.329306	0.045621	0.290410	12	0/0

表 20　电费MAPE的逐城分解，单位为%

方法	乌鲁木齐	固原	武汉
频数	0.772160	0.094224	0.804347
软能量	0.698834	0.288920	0.357193
硬W_1	0.018529	0.341666	0.061409
硬最小L_1	0.696076	0.300437	0.385390
价/时窗配对L_1	0.523837	0.300437	0.633258
硬W_1加候选L_1上限	0.075426	0.300437	0.612056

表 21　全部城市与预算的选择；四种硬方法逐例同选，故合并显示

城市/帽倍率	频数	软能量	四种硬方法	库内oracle
乌鲁木齐/1.00	无选	无选	0	0
乌鲁木齐/1.02	8	8	8	8
乌鲁木齐/1.05	8	8	8	8
乌鲁木齐/1.10	8	8	8	8
固原/1.00	3超帽	0	0	0
固原/1.02	8	8	8	8
固原/1.05	8	8	8	8
固原/1.10	8	8	8	8
武汉/1.00	3超帽	0	0	0
武汉/1.02	4	4	4	4
武汉/1.05	8	8	8	8
武汉/1.10	8	8	8	8

频数的两次超帽分别为12.5160、12.6716 kWh，原遗憾留空；不能把九次有效遗憾为0改说成12次全部成功。四种硬法均12次实际合格且选到库内oracle，遗憾均0，故没有新候选优于硬W_1选策的证据。冻结规则按合格数、遗憾、最大能量APE等排序，仍登记价/时窗配对L_1；其最大能量APE比最小L_1仅低0.000489个百分点，电费MAPE却约为硬W_1的3.46倍。固原硬W_1电费MAPE还高于频数，逐城结果不能被总平均掩盖。代理热量守恒不等于策略电量/电费精确，价格矩和05/13时窗约束共同改变，不能据联合候选单独证明价格矩因果贡献。

独立42225项记录检查及全部27轨迹RC/服务/初末/容量/互斥检查通过；首次审计因两处约9.09×10^{-13}元的并列翻位失败，原记录保留，补充精度审计未改权重、协议或阈值。运行29.135秒、共同准备0.648秒分别记录，全真值计算计入成本；不把不同方法的依赖成本或仿真成本省略后声称提速。

记录入口：`docs/implementation_20261002_followup/parallel_stage_03/algorithm/`，含`confirmation_run_01/protocol.json`、`manifest.json`、`truth.json`、`selections.json`、`predictions.json`、`failures.json`、完整轨迹与逐日表；`analysis_01/method_summary.csv`、`budget_selections.csv`及`policy_predictions.csv`保存完整分母，`claims_reviewed.txt`、`interpretation.json`和`verification_02/audit.json`限定结论。原三城天气已用于

开发，模型为共同公开原型及方法学电价；无新留出泛化、实际投资改善或在线预测主张。

5.7 身份、复算范围与交付说明

本轮R8P对既有证据索引和新增引用文件计算SHA-256，并从保存的JSON/CSV重新汇总六楼中位数、采用分母、热容四格差值和27策略误差，核对正文及附录的舍入数值；不重新运行求解器、不累计历史测试，也不将此前通过记录改成本轮新实验。完整小时调度留在各原证据目录，附录展示所有实验单元与拒绝/超帽/无选分母，不打印数万小时流水。

正文保留`minor:supplement`标题锚点，表1仅列E01/E03；原`tab:minor-evidence`移至本附录总表，仍含管网反馈和四楼连续状态，因而主文工程反馈、创新及评分页的原交叉引用没有变成另一组实验。历史结果的原始来源、模型版本、投资范围、求解容差和适用限制随记录保留。

附录6　推荐信

推荐信

尊敬的大赛组委会、各位评审专家：

我愿推荐西南交通大学“时空质三维协同的城镇供热源荷匹配与全生命周期规划寻优模型”项目团队参加本届比赛。结合本人在燃烧技术与能源利用领域的研究，我认为该项目具有以下价值。

在学术上，项目将热源规划、热能品位匹配与逐时运行相结合，建立兼顾经济性、碳排放和供热可靠性的分析框架，为研究多热源系统的规划与运行耦合提供了有益思路。

从燃烧技术的工程应用看，项目关注燃烧供热与余热、热泵及储热的协同，可为燃烧热源的容量配置、清洁替代及调峰保供提供量化依据，有助于在系统层面协调化石能源利用与低碳转型。团队研究思路清晰，成果具有进一步深化与应用验证的潜力，我予以郑重推荐。

此致

敬礼！

推荐人：葛冰

工作单位：上海交通大学机械与动力工程学院

签名：葛冰　　日期：2026 年 9 月 29 日

联系方式：13044638798.